

UNIVERSITÀ DEGLI STUDI DI BOLOGNA

SCUOLA DI INGEGNERIA E ARCHITETTURA

Corso di Laurea in Ingegneria Informatica

Dipartimento di Informatica — Scienza e Ingegneria — DISI

TESI DI LAUREA in

Tecnologie Web T

Control Plane Integrato 5G RAN e LoRa per Ottimizzazione di Resource Management

Tesi di Laurea di:

Victoria Ulinici

Relatore:

Chiar.mo Prof.Ing. Paolo Bellavista

Correlatori:

Dr. Alessandro Aimi

Dott.ssa Sofia Montebugnoli

Anno Accademico 2023 – 2024

Indice dei Contenuti

INTRODUZIONE	3
1 O-RAN.....	5
1.2 L'evoluzione dell'O-RAN	6
1.3 Architettura e Disaggregazione HW/SW	6
1.3.1 Componenti chiave dell'architettura O-RAN	7
1.3.2 Ruolo del RAN Intelligent Controller (RIC).....	8
1.4 Near-RT RIC	9
1.4.3 Requisiti e Prestazioni del Near-RT RIC.....	13
1.5 Non-RT RIC	13
1.5.1 Le rApp: Applicazioni per il Non-RT RIC	14
1.6.1 Funzionalità e Comunicazione dell'Interfaccia E2	17
1.7 Interfaccia O1	19
1.8 Interfaccia A1.....	21
1.9 xApp.....	22
1.9.1 Interfacce e Sviluppo delle xApp	22
1.10 E2 Service Model	24
2 Tecnologie 5G e IoT	26
2.1 Evoluzione delle reti mobili e il ruolo del 5G	26
2.2 Innovazioni del 5G	28
2.3 Internet of Things (IoT) e il ruolo del 5G	29
2.4 Architettura del 5G-IoT.....	31
2.5 Sfide dell'implementazione del 5G e IoT	31
3 Tecnologie LPWAN	33
3.2 Panoramica sulle tecnologie LPWAN	33
3.3 Tecnologia LoRa.....	35
3.2.1 Architettura LoRa	36
3.3 Protocollo LoRaWAN	37
4 Orchestrazione di applicazioni cloud native	43
4.1 Introduzione alla containerizzazione	44
4.2 Docker: Il motore della containerizzazione.....	45
4.3 Kubernetes: Il sistema di orchestrazione cloud-native	47
4.4 Kubernetes nel Near-RT RIC	47
4.4.1 Helm: La gestione automatizzata del deployment	49
4.5 Ciclo di vita delle xApp in Kubernetes	50
5 Control Plane integrato per LoRa e RAN.....	53
5.1 Architettura del sistema.....	54

5.1.1 Near-RT RIC e xApp.....	55
5.1.2 E2 Termination Custom.....	56
5.1.3 LoRa Gateway.....	57
5.1.4 LoRa End-Device	57
5.2 E2SM-LoRa: Necessità e Scopo	59
6 Implementazione.....	61
6.1 Hardware Utilizzato	61
6.2 Configurazione del Cluster RIC Control Plane.....	62
6.3 Installazione del framework AppMgr.....	63
6.4 Configurazione del framework xApp	63
6.4.1 Implementazione xApp	64
6.5 Implementazione E2TermCustom	67
6.6 Implementazione LoRaGateway	71
6.7 Implementazione LoRa End-Device	73
6.8 Implementazione dell'E2SM Personalizzato.....	74
6.9 xApp Deployment: onboarding.....	75
7 Test e Risultati Sperimentali	77
7.1 Fase di Configurazione	77
7.2 Gradual Load Test: Analisi della Latenza e la Ripartizione della Trasmissione	79
7.3 Stress Test: Analisi della Latenza e la Ripartizione della Trasmissione.....	82
Conclusioni e sviluppi futuri.....	86
Indice delle figure	87
Indice dei listati.....	88
Bibliografia	89

INTRODUZIONE

L'architettura delle moderne reti di telecomunicazione si basa sulla distinzione tra Control Plane e Data Plane, due componenti fondamentali per la gestione e la trasmissione dei dati. Il Data Plane è responsabile del trasferimento effettivo dei pacchetti di dati, mentre il Control Plane si occupa della segnalazione, della configurazione e dell'allocazione delle risorse di rete. Questa distinzione è valida sia per le reti cellulari avanzate come il 5G, sia per le reti a basso consumo come LoRa, sebbene con approcci e strutture profondamente differenti.

Il 5G, con la sua architettura altamente scalabile e virtualizzata, introduce il concetto di Radio Access Network (RAN), che collega i dispositivi mobili alla rete Core e ne gestisce il traffico radio. Nelle implementazioni tradizionali, il RAN è un sistema monolitico gestito da singoli operatori, il cosiddetto black box, ma con l'avvento del paradigma O-RAN (Open Radio Access Network), l'architettura è stata ripensata per favorire modularità, apertura e interoperabilità. Un elemento centrale di questa nuova architettura è il RAN Intelligent Controller (RIC), che permette di gestire dinamicamente le risorse radio attraverso l'uso di applicazioni software intelligenti chiamate xApp e rApp. (con distinzione in base alla latenza di operazione)

All'interno del Control Plane del 5G, il Near-Real-Time RIC (Near-RT RIC) svolge un ruolo chiave, consentendo di prendere decisioni di ottimizzazione della rete con latenze comprese tra 10 millisecondi e 1 secondo. Il Near-RT RIC comunica direttamente con le xApp, applicazioni modulari che implementano algoritmi avanzati per l'allocazione delle risorse e la gestione intelligente della rete. Grazie a questa architettura, il 5G può adattarsi in tempo reale alle condizioni variabili della rete, migliorando la Qualità del Servizio e l'efficienza complessiva dell'infrastruttura.

Dall'altro lato, LoRa è una tecnologia di comunicazione a lungo raggio e basso consumo, progettata per le reti IoT. L'architettura LoRa è composta principalmente da tre elementi: i dispositivi LoRa, che trasmettono dati a bassissimo consumo; i LoRa Gateway, che ricevono i pacchetti dai dispositivi e li inoltrano alla rete core; e i Network Server, che gestiscono il routing dei pacchetti e l'instradamento delle informazioni verso le applicazioni di destinazione. Tuttavia, a differenza del 5G, LoRa non prevede un'ottimizzazione dinamica della rete, poiché il suo modello di controllo è statico e decentralizzato.

A differenza delle reti cellulari, LoRa non dispone di un Control Plane strutturato. Non esistono componenti centralizzati come il Near-RT RIC, né meccanismi di controllo avanzati per l'ottimizzazione dinamica delle risorse. Il Control Plane di LoRa si basa principalmente su configurazioni statiche che vengono applicate ai dispositivi e ai gateway LoRa prima della loro attivazione.

In un contesto in cui reti eterogenee come il 5G O-RAN e LoRa coesistono, come nel caso delle smart city, l'integrazione tra queste tecnologie può risultare vantaggiosa per affrontare scenari complessi in cui è necessario adattare dinamicamente le risorse di rete in base a dati ambientali. Se il Near-RT RIC del 5G offre un Control Plane avanzato, in grado di ottimizzare dinamicamente le risorse radio, la rete LoRa non dispone di un meccanismo equivalente. Questo crea una frammentazione nella gestione della rete, rendendo difficile un'integrazione efficiente tra le due tecnologie.

L'obiettivo di questo lavoro è progettare e implementare un Control Plane unificato, che permetta di integrare le capacità avanzate di gestione della rete del 5G O-RAN con la rete LoRa, fornendo un meccanismo per la configurazione e il monitoraggio dinamico dei dispositivi LoRa attraverso il Near-RT RIC. In particolare, il sistema proposto si basa sull'uso del Near-RT RIC come piattaforma di gestione intelligente della rete, sullo sviluppo di una xApp personalizzata per controllare i parametri di configurazione dei LoRa Gateway e sull'implementazione di un modello di servizio personalizzato

E2SM-LoRa, che definisce il formato dei messaggi per la configurazione e il monitoraggio della rete LoRa

Questa integrazione permetterà di sfruttare le capacità avanzate del 5G O-RAN per colmare il divario tra le reti cellulari e le reti LPWAN, migliorando l'allocazione delle risorse e garantendo un controllo più efficace sulla rete LoRa. L'implementazione proposta è particolarmente utile in scenari come le smart city, le reti industriali IoT e le infrastrutture per il monitoraggio ambientale, dove la coesistenza di reti diverse richiede un meccanismo di gestione unificato.

Struttura della tesi:

Il Capitolo 1 introduce il concetto di O-RAN, spiegando come l'architettura delle reti di accesso radio si stia evolvendo verso un modello più aperto, modulare e interoperabile. Vengono descritti i vantaggi della separazione tra hardware e software e l'importanza della virtualizzazione delle funzioni di rete per migliorare flessibilità ed efficienza.

Il Capitolo 2 fornisce una panoramica sulle tecnologie 5G e IoT, con particolare attenzione alle innovazioni che caratterizzano le reti mobili di nuova generazione. Vengono esaminati i principali aspetti del 5G RAN, inclusi lo slicing della rete, le funzionalità di gestione della Qualità del Servizio (QoS) e le implicazioni dell'integrazione con dispositivi IoT in scenari smart city.

Il Capitolo 3 è dedicato alle tecnologie LPWAN (Low Power Wide Area Network), con un approfondimento specifico su LoRa e LoRaWAN. Viene illustrato il funzionamento di questa tecnologia, il suo ruolo nel panorama IoT e i vantaggi offerti rispetto ad altre soluzioni di connettività per applicazioni a basso consumo energetico e lunga distanza.

Il Capitolo 4 esplora l'uso di Kubernetes e tecnologie cloud-native per l'orchestrazione delle risorse di rete. Viene illustrato come il paradigma cloud-native possa supportare l'implementazione e la gestione scalabile delle componenti software di O-RAN, con particolare attenzione alla gestione di microservizi e container per xApp.

Il Capitolo 5 descrive l'integrazione della rete LoRa nell'ecosistema O-RAN tramite il 5G RAN Intelligent Controller (Near-RT RIC) e una xApp dedicata. Viene analizzato il ruolo dell'E2 Term Custom, che funge da ponte tra O-RAN e LoRa, e l'implementazione dell'E2SM-LoRa, un modello di servizio personalizzato che definisce il formato dei messaggi per la configurazione e il monitoraggio dei dispositivi LoRa.

Il Capitolo 6 si concentra sull'implementazione pratica del framework proposto per l'integrazione tra O-RAN e LoRa. Vengono descritti i dettagli tecnici relativi allo sviluppo della xApp per il Near-RT RIC, evidenziando le funzionalità implementate per la configurazione.

Il Capitolo 7 presenta i test e i risultati sperimentali ottenuti attraverso l'implementazione del framework proposto. Vengono analizzati il comportamento della rete, la latenza, l'efficienza nella gestione delle risorse e le prestazioni complessive del sistema.

L'integrazione tra O-RAN e LoRa rappresenta un passo significativo verso l'adozione di un paradigma di gestione delle reti più flessibile e dinamico. Questo approccio consente di ottimizzare l'allocazione delle risorse in tempo reale e migliorando l'efficienza delle reti mobili. Il presente lavoro fornisce un quadro dettagliato delle tecnologie coinvolte, delle metodologie di implementazione e delle strategie di ottimizzazione adottate, contribuendo alla definizione di un modello innovativo per il Resource Management nelle reti di nuova generazione.

1 O-RAN

Negli ultimi decenni, l'evoluzione delle reti di telecomunicazione ha seguito un percorso caratterizzato dall'adozione di tecnologie sempre più avanzate e dall'introduzione di nuovi paradigmi per la gestione dell'infrastruttura di rete. Con il passaggio dalle reti 2G alle moderne reti 5G, la crescente domanda di connettività ha reso evidente la necessità di soluzioni più flessibili, scalabili e accessibili.

Le tradizionali Reti di Accesso Radio (Radio Access Networks, RAN) sono state storicamente implementate come sistemi monolitici, in cui hardware e software erano strettamente integrati e sviluppati da un numero ristretto di fornitori. Questo modello chiuso ha portato a una dipendenza significativa da soluzioni proprietarie, limitando la possibilità di innovazione, l'interoperabilità tra diversi vendor e l'adozione di nuove funzionalità [2].

Con l'avvento delle reti di quinta generazione (5G), le esigenze di maggiore flessibilità, efficienza operativa e riduzione dei costi hanno accelerato la ricerca di approcci più aperti e modulari. In questo contesto, l'Open Radio Access Network (O-RAN) emerge come un paradigma innovativo che ridefinisce l'architettura della RAN, introducendo un modello disaggregato basato su interfacce aperte, software definito e componenti interoperabili [2].

L'obiettivo principale dell'approccio O-RAN è quello di superare le limitazioni delle architetture proprietarie, permettendo agli operatori di rete di scegliere soluzioni da diversi fornitori, ridurre i costi di implementazione e gestione e, al contempo, promuovere un ambiente di innovazione più dinamico [4]. Questa trasformazione è resa possibile dall'adozione di principi quali la softwarizzazione delle funzionalità di rete, l'integrazione di algoritmi di intelligenza artificiale per l'ottimizzazione della rete e l'uso di tecnologie cloud-native per migliorare la scalabilità e l'affidabilità dell'infrastruttura [3].

L'alleanza O-RAN [30], un consorzio di operatori di rete, fornitori di tecnologia e sviluppatori open-source, gioca un ruolo centrale nella definizione degli standard e nella promozione di un'infrastruttura di rete più aperta e flessibile. Grazie agli sforzi congiunti di questa comunità, O-RAN sta guadagnando rapidamente terreno, offrendo un nuovo approccio alla progettazione e alla gestione delle reti mobili.

Questo capitolo fornirà una panoramica dettagliata di O-RAN, analizzandone i principi fondamentali e le motivazioni alla base della sua adozione. Verranno esplorati i seguenti aspetti chiave:

- Principi e Benefici di O-RAN: un'analisi delle caratteristiche principali dell'architettura O-RAN e dei vantaggi rispetto alle RAN tradizionali.
- Architettura e Disaggregazione HW/SW: una descrizione della struttura modulare di O-RAN, con un focus sulla separazione tra hardware e software e sul concetto di RAN virtualizzata.
- Componenti chiave di O-RAN: un approfondimento sulle principali unità funzionali dell'architettura, tra cui la Centralized Unit (O-CU), la Distributed Unit (O-DU) e la Radio Unit (O-RU).
- Ruolo del Near-RT RIC e del Non-RT RIC: una discussione sulla centralità dell'intelligenza di rete e sull'uso di algoritmi di machine learning per l'ottimizzazione delle prestazioni della RAN.
- Le interfacce aperte e la loro importanza: un'analisi delle interfacce standardizzate, come E2, A1 e O1, che garantiscono l'interoperabilità tra diversi fornitori e semplificano l'integrazione di nuove funzionalità.

L'obiettivo di questo capitolo è fornire una comprensione chiara e approfondita del paradigma O-RAN, mettendo in evidenza come esso stia trasformando il settore delle telecomunicazioni e quali siano le prospettive future per la sua implementazione su larga scala.

1.2 L'evoluzione dell'O-RAN

Le Reti di Accesso Radio (Radio Access Networks, RAN) stanno attraversando una profonda trasformazione, passando da implementazioni monolitiche basate su hardware proprietario a soluzioni più flessibili, innovative e personalizzabili. Questo cambiamento è reso possibile grazie alla disaggregazione, all'adozione di interfacce aperte e alla softwarizzazione delle funzionalità di rete. Un elemento chiave di questa evoluzione è l'Open Radio Access Network (O-RAN), un paradigma che ha guadagnato rapidamente consenso grazie alla creazione di un consorzio globale con un'ampia partecipazione dell'industria e l'interesse delle autorità regolatorie.

L'Alleanza O-RAN [2] propone un'architettura aperta in cui la RAN è suddivisa in diversi componenti funzionali, interconnessi tramite uno strato comune di controllo e gestione. Questa disaggregazione consente l'implementazione di logiche di controllo personalizzate attraverso applicazioni di terze parti, sviluppate da operatori di rete mobile (MNO), aziende specializzate in soluzioni RAN, la comunità open-source e nuovi attori del mercato.

O-RAN integra e amplia lo standard 3GPP(Third Generation Partnership Project) [2], definendo interfacce aperte che garantiscono l'interoperabilità tra fornitori diversi, favorendo così un ecosistema più competitivo e dinamico.

L'obiettivo principale dell'Alleanza O-RAN è favorire la concorrenza e l'innovazione, consentendo agli MNO di costruire reti più flessibili ed efficienti. Tra i principali vantaggi di questo approccio:

- Maggiore innovazione: grazie alla possibilità di sviluppare e implementare applicazioni su misura per l'ottimizzazione delle operazioni di rete.
- Riduzione dei costi: la competizione tra fornitori di software e produttori di hardware crea un ambiente più favorevole all'abbassamento dei prezzi.
- Futuro-proofing: gli MNO possono testare e convalidare nuove soluzioni e algoritmi sulla loro infrastruttura esistente, garantendo una rete sempre aggiornata e pronta per l'adozione di nuove tecnologie.

Per accelerare l'adozione del modello O-RAN, l'Alleanza ha collaborato con la Linux Foundation per creare la O-RAN Software Community (OSC) [31], un'iniziativa open-source che sviluppa implementazioni di riferimento dei componenti O-RAN. L'OSC fornisce un punto di partenza per la prototipazione di soluzioni O-RAN, che includono il Near-Real-Time RIC, il Non-Real-Time RIC, il Service Management and Orchestration (SMO), l'O-DU (O-RAN Distributed Unit) e l'O-CU (O-RAN Centralized Unit). Inoltre, distribuisce una serie di xApp, ognuna delle quali offre funzionalità avanzate, tra cui: controllo di ammissione degli utenti finali (UE), monitoraggio delle prestazioni delle stazioni base Key Performance Measurements (KPM), rilevamento di anomalie nel traffico, ottimizzazione dinamica del traffico di rete.

Grazie a questo approccio modulare e open-source, O-RAN sta plasmando il futuro delle telecomunicazioni, rendendo le reti mobili più flessibili, efficienti ed economicamente sostenibili.

1.3 Architettura e Disaggregazione HW/SW

L'Open Radio Access Network (O-RAN) rappresenta un'innovazione dirompente nel settore delle telecomunicazioni, progettata per superare i limiti delle tradizionali RAN chiuse e proprietarie. Il suo obiettivo principale è aprire le interfacce tra i componenti software, offrendo vantaggi significativi come la riduzione dei costi, una gestione dinamica del ciclo di vita dei servizi, un'accelerazione del time-to-market e uno stimolo all'innovazione.

Attualmente, i componenti della Radio Access Network (RAN) sono progettati come sistemi monolitici chiusi, vere e proprie "scatole nere" [4] dal punto di vista degli operatori. L'intero protocol stack è implementato su hardware proprietario e specializzato, vincolato a un unico fornitore. Questo approccio

limita fortemente la flessibilità e l'interoperabilità, impedendo agli operatori di personalizzare le funzionalità della rete o di adattarle alle proprie esigenze. Di conseguenza, l'architettura monolitica ostacola l'adozione di nuove applicazioni e rende complessa l'integrazione di tecnologie provenienti da fornitori diversi.

Questa visione chiusa e proprietaria è ormai messa in discussione dalle nuove esigenze di business, che richiedono reti più aperte, modulari e programmabili, capaci di adattarsi rapidamente a nuovi servizi e casi d'uso. La risposta a questa necessità è rappresentata dal modello O-RAN, promosso dall'O-RAN Alliance, che introduce un'architettura disaggregata, in cui l'hardware e il software sono separati e le interfacce sono aperte e standardizzate. Questo cambiamento consente agli operatori di scegliere componenti da diversi fornitori, aumentando la concorrenza e accelerando l'innovazione.

Tuttavia, l'adozione di O-RAN presenta alcune sfide. La necessità di garantire interoperabilità tra componenti di fornitori diversi impone la definizione di standard chiari e test rigorosi. Inoltre, la virtualizzazione della rete introduce nuove problematiche, tra cui l'efficienza energetica e la sicurezza dei sistemi. Un ulteriore aspetto critico è il passaggio da un'architettura centralizzata a una distribuita, in cui i controllori operano su diverse scale temporali (multi-time-scale control). Questa transizione richiede lo sviluppo di nuovi algoritmi di gestione, in grado di coordinare i vari cicli di controllo ed evitare instabilità o conflitti nella rete.

Al centro del concetto di O-RAN vi è la disaggregazione dei componenti della RAN, un approccio che consente agli operatori di selezionare soluzioni da fornitori diversi, riducendo la dipendenza da un unico vendor (vendor lock-in) [3] e favorendo un'architettura modulare e interoperabile. Questa modularità permette di aggiornare, migliorare o sostituire singole funzionalità di rete in modo indipendente, garantendo maggiore flessibilità e scalabilità. Inoltre, l'adozione di un'architettura aperta facilita l'integrazione di nuove tecnologie, accelerando l'innovazione e migliorando l'efficienza operativa.

Uno dei principi fondamentali di O-RAN è la virtualizzazione e softwarizzazione [4] dei suoi componenti, che vengono distribuiti su una piattaforma cloud denominata O-Cloud. Questa infrastruttura virtualizzata è solitamente distribuita su più data center e fornisce le risorse computazionali necessarie per eseguire le funzioni della rete O-RAN. L'adozione dell'O-Cloud rende la rete più scalabile, flessibile e pronta per un'orchestrazione dinamica delle risorse, rispondendo in modo efficace alle esigenze di evoluzione tecnologica. Il principio di apertura e standardizzazione delle interfacce, offre un'alternativa alle soluzioni proprietarie che vincolano gli operatori a un unico fornitore. Questa apertura consente agli MNO di selezionare le funzioni di rete in base alle loro esigenze di performance, senza dipendere da implementazioni chiuse e non interoperabili.

1.3.1 Componenti chiave dell'architettura O-RAN

Per garantire la massima flessibilità, il 3GPP ha introdotto diversi schemi di functional split nello stack protocollare della RAN, permettendo agli operatori di scegliere il punto ottimale di separazione tra le funzionalità della Centralized Unit (CU) e della Distributed Unit (DU) in base ai requisiti specifici della rete [4]

Nell'architettura O-RAN, il nodo gNB (la stazione base 5G) è suddiviso in tre unità principali[1]:

- **O-CU (Centralized Unit):** gestisce i livelli superiori dello stack protocollare della RAN, tra cui RLC (Radio Link Control), PDCP (Packet Data Convergence Protocol) e la gestione delle connessioni. È ulteriormente suddivisa in due componenti logici
 - **Control Plane (CP):** responsabile della gestione dei segnali di controllo della rete.
 - **User Plane (UP):** gestisce il traffico dati degli utenti.
- **O-DU (Distributed Unit):** si occupa dell'elaborazione a livello MAC e PHY, gestendo il controllo della trasmissione radio.

- **O-RU (Radio Unit):** comprende gli elementi hardware necessari per la trasmissione e ricezione del segnale radio verso i dispositivi utente.

Questa suddivisione consente agli operatori di rete di implementare la RAN in modo più flessibile e scalabile, ottimizzando l'allocazione delle risorse e facilitando l'adozione di nuove tecnologie.

Prima dell'avvento di O-RAN, le implementazioni di queste separazioni funzionali erano proprietarie, con interfacce definite e controllate dai fornitori, il che impediva l'interoperabilità tra componenti di fornitori diversi. In particolare, la visione O-RAN prevede due controller logici che offrono una visione astratta e centralizzata della rete, grazie a pipeline di dati che trasmettono e aggregano centinaia di Misurazioni Chiave delle Prestazioni (KPM) [1] sullo stato dell'infrastruttura di rete (ad esempio, numero di utenti, carico, throughput, utilizzo delle risorse), oltre a ulteriori informazioni contestuali provenienti da fonti esterne alla RAN.

Una volta implementato lo stack protocollare della RAN [4] in modo flessibile, attraverso i principi di softwarizzazione e apertura delle interfacce, il passo successivo è dotare la rete di intelligenza per il controllo e l'ottimizzazione della RAN. Al centro di questa trasformazione si trova il RAN Intelligent Controller (RIC), un componente essenziale che introduce intelligenza e controllo programmabile nella rete. Il RIC, insieme alle sue applicazioni avanzate, le xApp, costituisce uno strato dinamico e intelligente per il controllo delle reti O-RAN. Le xApp, eseguite sul Near-RT RIC, estendono le capacità del controllore, fornendo funzionalità specifiche per l'ottimizzazione delle prestazioni, il bilanciamento del traffico e il monitoraggio delle risorse radio [3]. Grazie alla loro natura modulare, le xApp possono essere sviluppate e integrate da diversi fornitori, migliorando ulteriormente l'interoperabilità e stimolando l'innovazione all'interno dell'ecosistema O-RAN. L'integrazione tra il RIC e le xApp rappresenta il cuore dell'approccio O-RAN, trasformando la RAN in una piattaforma programmabile, adattiva e intelligente, capace di evolversi rapidamente per rispondere alle esigenze dinamiche delle reti mobili moderne.

1.3.2 Ruolo del RAN Intelligent Controller (RIC)

L'obiettivo dell'architettura O-RAN è fornire capacità decisionali avanzate attraverso i Radio Intelligent Controllers (RIC) [4], che operano con diverse latenze:

- Il Near-RT RIC, responsabile del controllo ad anello chiuso near-real-time, che interviene con tempi compresi tra 10 millisecondi e 1 secondo, gestendo azioni come il bilanciamento del carico e il controllo della Qualità del Servizio (QoS).
- Il Non-RT RIC, che opera con tempi superiori a 1 secondo, concentrandosi su ottimizzazioni strategiche a lungo termine, come l'orchestrazione delle risorse, il policy management e l'addestramento dei modelli di Machine Learning (ML).

Come mostrato nella Figura 1.1, a livello superiore, l'architettura include il sistema di gestione e orchestrazione denominato Service Management and Orchestration (SMO) [4], che fornisce capacità di orchestrazione end-to-end (E2E), supervisionando l'intero ciclo di vita dei servizi di rete e integrando tutte le componenti della RAN. Questi RIC, oltre a gestire le applicazioni di controllo come le xApp e le rApp, forniscono funzionalità di controllo e ottimizzazione per le CU e le DU, indipendentemente dal fornitore dei componenti.

In questo modo, l'architettura O-RAN crea un sistema modulare, aperto e intelligente, capace di combinare il controllo in tempo reale con l'ottimizzazione strategica, realizzando una rete più efficiente, scalabile e pronta per le sfide delle reti oltre il 5G.

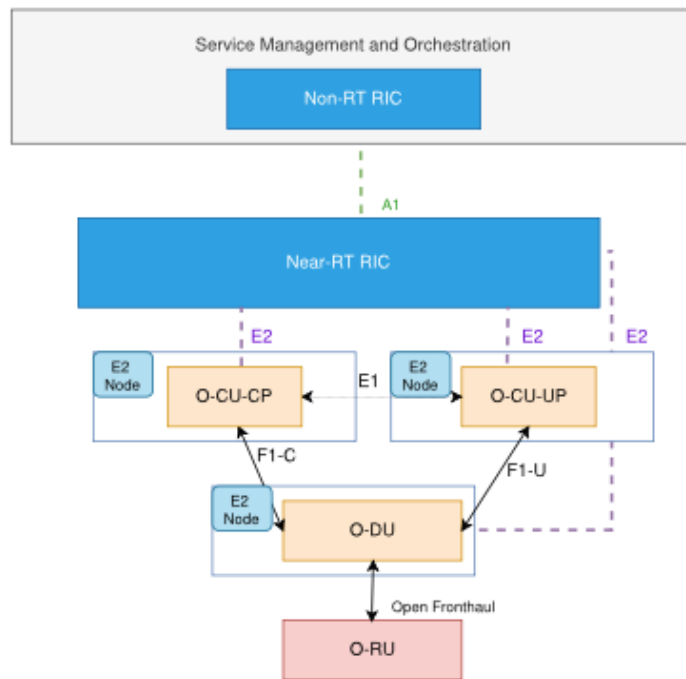


Figura 1.1 Architettura dell'O-RAN: interazione tra SMO, RIC e componenti della rete di accesso radio [4]

1.4 Near-RT RIC

Il Near-RT RIC è l'entità O-RAN responsabile dell'orchestrazione della RAN in tempo quasi reale, ovvero con un intervallo di latenza compresa tra 10 millisecondi e 1 secondo, e dell'automazione della rete. Il suo scopo principale è ospitare e facilitare l'operatività di applicazioni esterne, le xApp, per eseguire loop di controllo chiusi in tempo quasi reale, monitorare, analizzare e ottimizzare i parametri della rete, al fine di raggiungere il comportamento e le prestazioni desiderate. In sostanza, il Near-RT RIC opera in modo simile a ONOS (Open Network Operating System) nel contesto delle reti SDN (Software-Defined Networking) per le reti di trasporto [2].

Il Near-RT RIC è connesso contemporaneamente a più E2 Nodes [4], che rappresentano componenti chiave della RAN disaggregata: Open Central Unit - Control Plane (O-CU-CP); Open Central Unit - User Plane (O-CU-UP); Open Distributed Unit (O-DU).

Il Near-RT RIC ha attirato molta attenzione e contributi dai membri dell'Alleanza O-RAN [2], risultando uno dei componenti software più completi e maturi forniti dall'OSC. Interagisce con altre entità O-RAN tramite interfacce southbound e northbound per sfruttarne le informazioni e le capacità nella gestione della RAN. Di seguito, vengono dettagliate queste interazioni:

- **Nodi E2:** Il Near-RT RIC interagisce con i componenti della RAN, ad esempio O-CU-CP, O-CU-UP, O-DU, O-RU disaggregati o i nodi monolitici O-gNodeB e O-eNodeB. Raccoglie misurazioni in tempo reale e dati da questi nodi per monitorare le prestazioni della rete, il carico di traffico, i livelli di interferenza e altri parametri rilevanti. Inoltre, il Near-RT RIC comunica con i nodi E2 per configurare e regolare vari parametri della RAN, come la potenza di trasmissione, le impostazioni delle antenne, gli schemi di modulazione e gli algoritmi di scheduling, basandosi sulle decisioni prese dalle xApp.
- **SMO (Service Management and Orchestration):** Il Near-RT RIC interagisce con l'SMO, responsabile della gestione complessiva della rete e dei servizi. L'SMO fornisce funzioni di controllo e coordinamento di alto livello, mentre il Near-RT RIC agisce come un'estensione di

questo sistema, offrendo capacità di ottimizzazione e intelligenza in tempo quasi reale all'interno della RAN.

- **Non-RT RIC:** Il Near-RT RIC interagisce con il Non-RT RIC, che definisce e applica politiche, requisiti di Qualità del Servizio e vincoli normativi. Il Near-RT RIC scambia informazioni e allinea i suoi processi decisionali alle politiche descritte da questa entità, garantendo che le ottimizzazioni di rete e le allocazioni di risorse siano conformi alle regole stabilite.

Le interazioni tra il Near-RT RIC e altre entità O-RAN per lo scambio di KPM, informazioni di controllo e impostazioni di sistema creano un ecosistema collaborativo e distribuito, che abilita la programmabilità, l'automazione e l'intelligenza in tempo quasi reale all'interno della RAN. Simile a come l'ecosistema O-RAN è composto da diverse entità, anche il Near-RT RIC non è un'entità monolitica, ma una collezione di microservizi che lavorano insieme per consentire agli MNO di gestire le proprie RAN in tempo quasi reale.

1.4.1 Architettura del Near-RT RIC

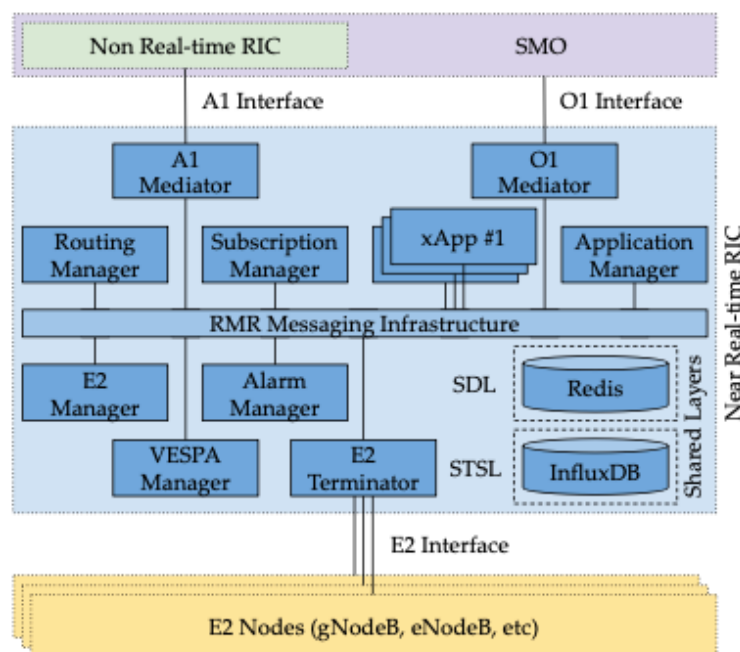


Figura 1.2 Architettura del Near-RT RIC: componenti, gestione della messaggistica e interazioni con SMO, Non-RT RIC e nodi E2 [2]

Figura 1.2 illustra i componenti del Near-RT RIC, descrivendo le funzionalità essenziali che supportano le xApp all'interno del cluster [2] [5]. Il cuore della comunicazione interna è il RIC Message Router (RMR), un'infrastruttura di messaggistica che collega tutti i componenti del Near-RT RIC, dalle xApp ai gestori delle interfacce. La libreria RMR consente lo scambio di messaggi tra applicazioni senza la necessità di conoscere dettagli tecnici come indirizzi IP o protocolli di trasporto.

La gestione dei percorsi di messaggistica è affidata al Routing Manager (RtMgr), che crea e distribuisce le politiche di instradamento sia per le xApp che per gli altri componenti del Near-RT RIC. In parallelo, il Application Manager (AppMgr) si occupa del ciclo di vita delle xApp, sovrintendendo all'installazione, alla rimozione e al monitoraggio delle applicazioni attive.

Il Subscription Manager (SubMgr) facilita l'interazione tra le xApp e i nodi E2, creando e gestendo i percorsi di sottoscrizione per lo scambio di dati. Complementare è il E2 Manager (E2Mgr), incaricato di configurare e monitorare lo stato dei nodi E2 registrati, informando le xApp in caso di anomalie.

Il E2 Terminator (E2Term) funge da ponte tra la messaggistica interna RMR e il protocollo SCTP, consentendo la comunicazione tra le xApp e i nodi E2. Il sistema di archiviazione è gestito dagli Shared Layers, composti dallo Shared Data Layer (SDL), dedicato ai dati relazionali, e dallo Shared Time Series Layer (STSL), riservato ai dati temporali.

Per la raccolta di metriche, il VESPA Manager (VesMgr) configura l'agente Virtual Event Streaming (VES), sfruttando Prometheus per ottenere statistiche dai componenti Near-RT RIC e dalle xApp. Gli allarmi vengono gestiti dall'Alarm Manager, che tramite Prometheus Alert Manager pubblica notifiche sugli eventi critici.

Infine, l'integrazione con il Non-RT RIC e l'SMO è gestita rispettivamente dall'A1 Mediator, che inoltra alle xApp le politiche strategiche ricevute dal Non-RT RIC, e dall'O1 Mediator, che espone all'SMO lo stato operativo dei componenti e delle xApp. Questa architettura integrata consente al Near-RT RIC di coordinare efficacemente le funzionalità delle xApp e l'interazione con la RAN.

Il Near-RT RIC è collegato a un Non Real-Time Radio Intelligent Controller (Non-RT RIC) tramite l'interfaccia A1 [4], che funge da interfaccia northbound e che svolge un ruolo cruciale nella gestione delle politiche e nell'arricchimento dei dati. Attraverso questa interfaccia, il Near-RT RIC può: Ricevere politiche dichiarative (declarative policies), che guidano le azioni delle xApp in base agli obiettivi definiti dall'operatore e Ottenere informazioni di arricchimento dei dati (data enrichment), come modelli predittivi o analisi storiche, che migliorano l'efficacia delle decisioni prese dalle xApp. In questo modo, il Near-RT RIC, con il supporto delle xApp, realizza un sistema di controllo intelligente e programmabile della RAN, capace di adattarsi dinamicamente alle condizioni della rete per migliorarne prestazioni, efficienza e Qualità del Servizio (QoS).

Il RIC near-real-time (near-RT) è solitamente associato a più nodi della RAN; pertanto, il controllo ad anello chiuso near-RT può influenzare la Qualità del Servizio (QoS) di centinaia o migliaia di dispositivi utente (User Equipment - UEs).

1.4.2 L'integrazione del Near-RT RIC con le xApp

All'interno del proprio application layer, il Near-RT RIC può ospitare una o più xApp, applicazioni modulari che utilizzano l'interfaccia E2 per raccogliere informazioni in near-real-time dalla RAN [4]. Queste xApp analizzano i dati ricevuti ed eseguono azioni di controllo con l'obiettivo di ottimizzare specifiche metriche di prestazione, come la Qualità del Servizio (QoS), il bilanciamento del carico o la gestione degli handover.

Come illustrato nella Figura 1.3, il rapporto tra il Near-RT RIC e i nodi E2 si articola in diverse componenti fondamentali. È importante notare che ciascun nodo E2 è associato a un solo Near-RT RIC alla volta. Il Near-RT RIC, oltre a includere una o più xApp e la terminazione dell'interfaccia E2, possiede un database che archivia i dati provenienti sia dalle xApp sia dai nodi E2. Ogni nodo E2 è composto da tre elementi principali. Il primo è un agente E2 (E2 Agent), responsabile della terminazione dell'interfaccia E2 e della gestione dell'invio e della ricezione dei messaggi E2. Il secondo elemento è costituito dalle funzioni RAN controllate dal Near-RT RIC, che forniscono servizi near-real-time. Infine, il terzo elemento comprende altre funzioni RAN, indipendenti dal controllo del Near-RT RIC, che supportano servizi differenti. Questa struttura garantisce una chiara divisione delle responsabilità

tra il Near-RT RIC e i nodi E2, facilitando il controllo delle funzioni RAN e la gestione efficiente delle interazioni tra componenti diversi dell'architettura O-RAN.

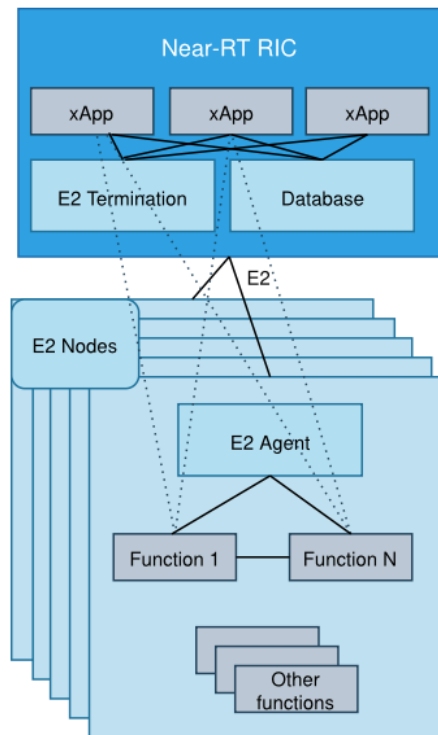


Figura 1.3 Interazione tra Near-RT RIC e nodi E2: terminazione E2, gestione delle xApp e controllo delle funzioni RAN [4]

Attraverso l'interfaccia E2, le xApp possono accedere a dati in near-real-time, sia a livello di singolo dispositivo utente (UE - User Equipment) sia a livello di cella, consentendo un controllo preciso e tempestivo delle risorse radio [3]. L'interfaccia E2 rappresenta uno degli elementi più innovativi dell'architettura O-RAN, poiché consente la comunicazione diretta tra le unità DU/CU e il Near-RT RIC. Questa connessione diretta permette il flusso continuo di dati (data streaming) dai nodi E2 alla piattaforma del RIC, con la possibilità di raccogliere dati in pochi secondi o anche meno, a differenza dei precedenti sistemi di gestione degli elementi (EMS - Element Management Systems), che normalmente effettuavano la raccolta dati a intervalli di 15-30 minuti. Grazie a questa trasmissione dati rapida e continua, il Near-RT RIC può elaborare e reagire quasi in tempo reale agli eventi della rete, migliorando l'efficacia delle decisioni e l'efficienza operativa [4].

Inoltre, l'interfaccia E2 consente alle applicazioni del Near-RT RIC (xApp) di riconfigurare i parametri operativi dei nodi E2, esercitando così un controllo diretto sulle risorse della RAN. Attraverso queste riconfigurazioni, le xApp possono ottimizzare le risorse della rete per raggiungere obiettivi specifici, come: miglioramento dell'efficienza energetica, ad esempio regolando la potenza trasmessa o spegnendo celle inutilizzate e ottimizzazione della Qualità dell'Esperienza (QoE) per singole applicazioni, come il video streaming o il gaming [4].

Tutto questo avviene in modalità near-real-time, ossia con tempi di risposta inferiori a 1 secondo, rendendo l'architettura O-RAN estremamente reattiva e adattiva rispetto alle condizioni dinamiche della rete.

1.4.3 Requisiti e Prestazioni del Near-RT RIC

Il Near-RT RIC deve soddisfare diversi requisiti fondamentali per garantire il funzionamento ottimale della rete O-RAN [4]. Primo, per definizione, il Near-RT RIC deve supportare i requisiti di latenza necessari per l'ottimizzazione near-real-time, con tempi compresi tra 10 millisecondi e 1 secondo.

Secondo, dal punto di vista architetturale, il Near-RT RIC deve utilizzare una connessione dedicata E2 che identifichi in modo univoco ciascun nodo E2, permettendo così una connessione diretta e uno scambio continuo di informazioni. Inoltre, il Near-RT RIC deve supportare connessioni E2 provenienti da più nodi E2 contemporaneamente, implementando quindi un'architettura one-to-many.

Il Near-RT RIC è inoltre agnostico rispetto alla tecnologia di accesso radio (RAT), il che significa che può supportare diversi tipi di RAT, come LTE, 5G NR, e future tecnologie di rete. Per facilitare questa interoperabilità, il flusso di lavoro del Near-RT RIC prevede la raccolta, dai nodi E2, dell'elenco delle funzioni che supportano i servizi RIC e i relativi modelli di servizio E2 (E2SM).

Terzo, il Near-RT RIC deve essere in grado di ospitare applicazioni modulari, note come xApps, ciascuna delle quali è progettata per gestire una funzione specifica della RAN su uno specifico nodo E2. Le xApps, attraverso l'interfaccia E2, ricevono dati in tempo reale e possono applicare azioni di controllo per ottimizzare le prestazioni della rete.

Quarto, il Near-RT RIC deve essere progettato per garantire resilienza. L'architettura O-RAN prevede che, in caso di guasto dell'interfaccia E2 o del Near-RT RIC, i nodi E2 continuino a funzionare in modo indipendente, garantendo la continuità del servizio.

L'OSC fornisce un'implementazione open-source del Near-RT RIC [2], seguendo un'architettura a microservizi basata su tecnologie cloud. Il Near-RT RIC è un cluster Kubernetes specializzato che utilizza il motore Docker per i container. In questo modo, ogni componente del Near-RT RIC è un pod Kubernetes isolato che esegue uno o più container Docker, le cui risorse, porte e interfacce sono descritte utilizzando Helm Charts (dettagliaremo queste tecnologie cloud più avanti).

In sintesi, il Near-RT RIC è il fulcro del controllo intelligente near-real-time della RAN, agendo come intermediario tra le politiche strategiche definite dal Non-RT RIC e le azioni operative eseguite dai nodi E2, e contribuendo così all'ottimizzazione continua e dinamica delle prestazioni di rete.

1.5 Non-RT RIC

Il secondo elemento chiave dell'architettura O-RAN è il framework SMO (Service Management and Orchestration). Questo componente è responsabile della gestione di tutte le procedure di orchestrazione, gestione e automazione per monitorare e controllare i componenti della RAN. In particolare, l'SMO ospita il Non-RT RIC [1] e fornisce un insieme di interfacce (descritte in dettaglio nelle Sezioni Successive) che supportano l'interazione tra i diversi componenti della rete, oltre a offrire funzionalità di raccolta dati per facilitare il monitoraggio e il controllo della rete tramite algoritmi di intelligenza artificiale (AI) e apprendimento automatico (ML).

Vale la pena menzionare che, al momento della stesura, le specifiche O-RAN non forniscono linee guida rigide riguardanti la separazione tra le funzionalità dell'SMO e del Non-RT RIC. Tuttavia, le specifiche raggruppano tali funzionalità in tre insiemi distinti. Il primo insieme (blocco evidenziato in arancione nella Figura 1.4) identifica le funzionalità e le interfacce ancorate al Non-RT RIC. Il secondo insieme (blocco evidenziato in verde) identifica le funzionalità ancorate al di fuori del Non-RT RIC. Il terzo insieme (blocco evidenziato in giallo) raggruppa funzionalità che non sono ancora ancorate a un componente specifico dell'SMO o che si estendono su più componenti.

Il RIC non-real-time è un componente del framework di Service Management and Orchestration (SMO), come illustrato nella Figura 1.1, e integra il RIC near-real-time per il funzionamento intelligente e l'ottimizzazione della RAN su una scala temporale superiore a un secondo. Analogamente al RIC near-real-time, consente il controllo ad anello chiuso della RAN, ma su scale temporali superiori a un secondo [1]. Oltre a ciò, il RIC non-RT supporta l'esecuzione di applicazioni di terze parti, denominate rApp, che forniscono servizi a valore aggiunto per supportare e facilitare l'ottimizzazione e il funzionamento della RAN. Tra i servizi offerti dalle rApp vi sono la definizione di linee guida per le politiche, l'arricchimento delle informazioni, la gestione delle configurazioni e l'analisi dei dati.

Attraverso il ciclo di controllo non-real-time, il RIC non-RT fornisce indicazioni, informazioni di arricchimento e gestione dei modelli di Machine Learning (ML) per il RIC near-RT. Le applicazioni rApp, eseguite sul Non-RT RIC, sfruttano le funzionalità esistenti del Non-RT RIC e della piattaforma di Service Management and Orchestration (SMO) per migliorare il controllo e l'ottimizzazione della RAN attraverso un processo decisionale avanzato basato sui dati.

Tra le funzionalità messe a disposizione dal Non-RT RIC e dalla SMO, vi sono [4]: raccolta ed esposizione dei dati (Data Collection and Exposure), che consente alle rApp di accedere a informazioni aggiornate sullo stato della rete e un SDK (Software Development Kit) dedicato allo sviluppo di modelli di Machine Learning (ML), che permette di creare algoritmi avanzati per il controllo e l'ottimizzazione della rete.

1.5.1 Le rApp: Applicazioni per il Non-RT RIC

Le rApp hanno il compito di fornire un livello aggiuntivo di intelligenza per il controllo della RAN, concentrandosi su analisi e ottimizzazioni su scala più ampia e a lungo termine. Il loro ambito di intervento include:

- Gestione delle risorse radio (Radio Resource Management - RRM), con lo scopo di ottimizzare l'utilizzo dello spettro e migliorare l'efficienza della rete.
- Analisi dei dati (Data Analytics), per monitorare le prestazioni, individuare tendenze e prevedere potenziali problemi.
- Fornitura di informazioni arricchite (Enrichment Information) per le xApp, che operano sul Near-RT RIC con cicli di controllo molto più rapidi.

L'integrazione tra rApp e xApp è fondamentale per l'architettura O-RAN, poiché le rApp, grazie alla loro visione più ampia e all'analisi a lungo termine, forniscono al Near-RT RIC dati e modelli utili a migliorare l'efficacia delle azioni eseguite dalle xApp nei cicli di controllo near-real-time. Sebbene le rApp possano supportare funzionalità di controllo simili a quelle delle xApp, come l'ottimizzazione del traffico, il controllo della schedulazione e la gestione degli handover, operano su scale temporali più ampie e sono progettate per definire politiche di controllo di livello superiore. Rispetto alle xApp, le rApp influenzano un numero maggiore di utenti e nodi di rete, agendo su una dimensione più strategica e globale [4].

Come mostrato nella Figura 1.4, il RIC non-RT ospita inoltre la terminazione dell'interfaccia R1, che collega le rApp al sistema, consentendo loro di accedere ai servizi di gestione ed esposizione dei dati, alle funzionalità di Intelligenza Artificiale (AI) e Machine Learning (ML), nonché alle interfacce A1, O1 e O2 attraverso l'infrastruttura interna di messaggistica [1].

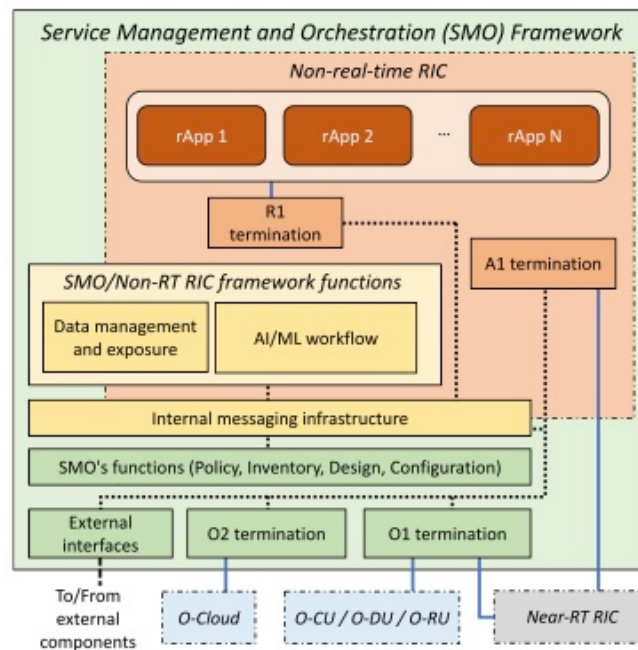


Figura 1.4 Architettura del framework SMO e Non-RT RIC: gestione delle rApp, infrastruttura di messaggistica e interfacce di terminazione [1]

Per garantire un'architettura flessibile, in cui il comportamento di ogni componente e funzionalità della rete possa essere regolato in tempo reale per soddisfare le intenzioni e gli obiettivi degli operatori, il RIC non-real-time (non-RT) fornisce due principali servizi di gestione e orchestrazione: la gestione della rete basata su intenti (intent-based network management) e l'orchestrazione intelligente (intelligent orchestration). L'orchestrazione intelligente è fondamentale nell'architettura O-RAN, poiché facilita lo sviluppo, il deployment, l'esecuzione e la manutenzione dell'intelligenza di rete. Tuttavia, l'integrazione di un numero crescente di xApp e rApp, eseguite in diversi RIC e punti della rete, rende il controllo più complesso, richiedendo soluzioni capaci di coordinare e orchestrare queste applicazioni. In questo contesto, il RIC non-RT assume il ruolo centrale nell'orchestrare l'intelligenza della rete, garantendo che le xApp e rApp selezionate siano adeguate agli obiettivi dell'operatore e soddisfino i requisiti richiesti. Inoltre, il RIC non-RT si occupa di assicurare che queste applicazioni siano istanziate nel punto corretto della rete per controllare gli elementi della RAN indicati negli intenti.

In sintesi, il Non-RT RIC, con le sue rApp, svolge un ruolo cruciale nell'architettura O-RAN, contribuendo a una gestione della rete più intelligente, adattiva e basata sui dati, e supportando il Near-RT RIC con informazioni strategiche per un controllo più efficace della rete[4]. Il RIC non-RT garantisce che le xApp e rApp ricevano i dati pertinenti e siano sufficientemente robuste da evitare conflitti. Questi conflitti possono emergere quando più applicazioni cercano di controllare simultaneamente le stesse funzionalità o parametri della rete. L'orchestrazione, dunque, non solo coordina l'esecuzione delle applicazioni, ma previene anche interferenze che potrebbero compromettere le prestazioni complessive della rete.

1.6 Interfaccia E2

Un aspetto fondamentale dell'architettura O-RAN è l'apertura delle interfacce e la definizione di specifiche che consentono la riconfigurazione completa dei nodi della RAN in modo interoperabile, ovvero senza vincoli di fornitore (vendor lock-in).

L'interfaccia E2 è stata progettata per soddisfare questo principio di apertura e per operare con tempi inferiori a 1 secondo, consentendo così l'implementazione di un sistema di ottimizzazione ad anello chiuso (closed-loop). Questa ottimizzazione riguarda parametri della RAN particolarmente sensibili alla latenza, come quelli dei livelli PHY (Physical Layer) e MAC (Medium Access Control) [4].

L'interfaccia E2 è un canale aperto che collega il Near-Real-Time RIC (Near-RT RIC) ai nodi E2, i quali comprendono Distributed Unit (DU), Central Unit (CU) ed eNB LTE conformi allo standard O-RAN [1]. Questa interfaccia è fondamentale per consentire al Near-RT RIC di gestire le risorse radio (Radio Resource Management - RRM) e controllare le funzionalità dei nodi E2. Inoltre, l'E2 permette di raccogliere metriche della RAN, sia a intervalli regolari sia in risposta a eventi specifici, supportando il monitoraggio continuo e l'ottimizzazione delle prestazioni della rete.

La raccolta delle informazioni può avvenire in due modalità principali. La prima è la raccolta periodica, in cui i dati vengono inviati a intervalli regolari per analizzare le prestazioni della rete nel tempo. La seconda è la raccolta basata su eventi, attivata al verificarsi di condizioni specifiche, come il superamento di una soglia di traffico o un'anomalia nel comportamento della rete. Tali procedure possono riguardare specifiche entità della rete, come celle, slice, classi di Qualità del Servizio (QoS) o singoli dispositivi utente (User Equipment, UE) [1].

L'interfaccia E2 gestisce la comunicazione tra il Near-RT RIC e i nodi E2 mediante messaggi standardizzati, classificati in due principali tipologie: le segnalazioni di controllo e i set di azioni. Le segnalazioni di controllo vengono utilizzate per modificare il comportamento della rete, ad esempio per gestire il bilanciamento del carico o l'esecuzione degli handover. I set di azioni, invece, definiscono l'insieme delle operazioni che il nodo E2 deve eseguire in risposta a eventi specifici, come la riconfigurazione delle risorse radio o l'applicazione di nuove politiche di rete.

Oltre a consentire l'invio di comandi e istruzioni, l'interfaccia E2 permette ai nodi E2 di comunicare al Near-RT RIC l'elenco delle funzionalità supportate. Questa capacità di autodichiarazione consente al RIC di adattare le proprie politiche di controllo alle capacità effettive dei nodi. Tale scambio di informazioni è essenziale per garantire l'adattamento dinamico delle politiche di rete, ottimizzando il comportamento del Near-RT RIC in base alle risorse disponibili. Inoltre, facilita l'interoperabilità tra componenti forniti da produttori diversi, promuovendo un ecosistema multi-vendor conforme agli standard O-RAN.

Per garantire l'interoperabilità, l'interfaccia E2 deve essere: Aperta e basata su specifiche standard, dotata di un modello di scambio dati (exchange model) chiaramente definito e implementato per la comunicazione tra nodi E2 e Near-RT RIC [4].

L'implementazione di questa interfaccia è fondamentale per l'attuazione di un controllo intelligente, adattivo e in tempo reale della rete, rendendo l'architettura O-RAN altamente reattiva e capace di adattarsi rapidamente alle condizioni dinamiche della rete.

Come mostrato nella Figura 1.5 l'interfaccia E2 è strutturata logicamente in due protocolli principali: E2 Application Protocol (E2AP) e E2 Service Model (E2 SM) [1]. L'E2AP è un protocollo procedurale di base che coordina la comunicazione tra il Near-RT RIC e i nodi E2, fornendo un insieme essenziale di servizi, come illustrato nella Figura 6. I messaggi E2AP possono includere diversi modelli di servizio E2 SM, ciascuno dei quali implementa funzionalità specifiche, come la segnalazione delle metriche della RAN o il controllo dei parametri della rete. L'interfaccia E2 utilizza il protocollo SCTP (Stream Control Transmission Protocol) per il trasporto dei dati.

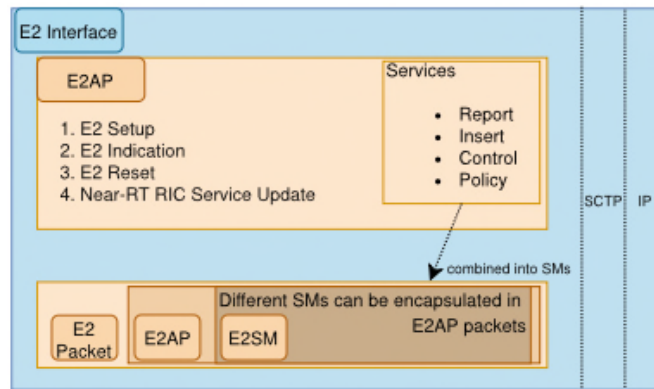


Figura 1.5 E2 Interface: Struttura dell'E2AP e incapsulamento degli E2SM [4]

Ogni nodo E2 espone una serie di funzioni RAN, cioè i servizi e le capacità che supporta. Ad esempio, le DU (Distributed Units) di fornitori diversi possono offrire controlli differenti a seconda dei parametri regolabili e delle funzionalità disponibili, oltre alla capacità di raccogliere e segnalare metriche di prestazione diverse. L'interfaccia E2 utilizza un meccanismo di publish-subscribe, in cui i nodi E2 pubblicano i propri dati e le xApp nel RIC near-RT possono sottoscrivere a una o più di queste funzioni RAN. Questo meccanismo permette di separare chiaramente le capacità di ciascun nodo e di definire in modo preciso l'interazione delle xApp con la RAN, migliorando la modularità e l'efficienza del sistema.

A livello più basso, il protocollo E2AP si occupa della gestione dell'interfaccia, gestendo operazioni come l'inizializzazione, il ripristino e la segnalazione degli errori dell'interfaccia E2, oltre agli aggiornamenti dei servizi del RIC near-real-time (near-RT). Tra questi aggiornamenti vi è lo scambio dell'elenco delle funzioni RAN supportate dal nodo E2 [1].

1.6.1 Funzionalità e Comunicazione dell'Interfaccia E2

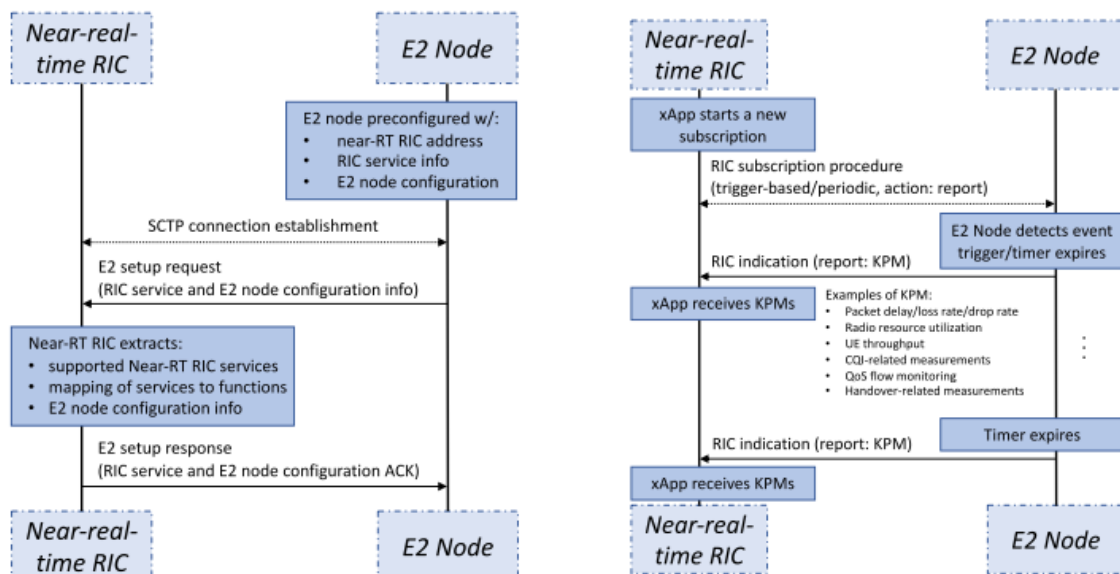


Figura 1.6 Procedura di sottoscrizione e gestione degli eventi tra Near-RT RIC ed E2 Node [1]

Un esempio della procedura di inizializzazione dell'interfaccia E2 viene schematizzato nella Figura 1.6. Il processo si articola in più fasi [1]: In primo luogo, viene stabilita la connessione SCTP (Stream Control Transmission Protocol) tra il RIC near-RT e il nodo E2, con il nodo E2 che conosce l'indirizzo IP e la porta dell'endpoint E2 del RIC near-RT. Successivamente, il nodo E2 invia una richiesta di configurazione (E2 setup request), elencando le funzioni RAN e le configurazioni supportate, insieme agli identificatori del nodo stesso. Il RIC near-RT elabora queste informazioni e risponde con un messaggio di conferma (E2 setup response).

La procedura di E2 Setup ha lo scopo di stabilire l'interfaccia E2 tra il Near-RT RIC e un nodo E2. Durante questa procedura, vengono scambiate due tipologie principali di informazioni: da un lato, l'elenco dei servizi supportati dal Near-RT RIC con la relativa mappatura tra i servizi e le funzioni presenti nel nodo E2, come indicato nella Figura 3; dall'altro, vengono fornite le informazioni dettagliate sulla configurazione del nodo E2 [4].

Una volta stabilita la connessione, il protocollo E2AP fornisce quattro servizi principali, che possono essere combinati per implementare i modelli di servizio E2SM (E2 Service Models)[1]. Questi servizi, illustrati anche nella Figura 7, sono:

La procedura di RIC Indication consente al nodo E2 di inviare al Near-RT RIC i messaggi associati ai servizi Report e Insert, in conformità con una precedente richiesta di sottoscrizione (RIC Subscription) effettuata dal RIC.

Il servizio E2 Report prevede l'invio di messaggi di indicazione (RIC Indication messages) dal nodo E2 al RIC near-real-time (near-RT) contenenti dati e telemetria. Per attivare questo servizio, una xApp nel Near-RT RIC effettua una sottoscrizione (subscription) a una funzione offerta dal nodo E2. Durante la negoziazione della sottoscrizione, la xApp può specificare gli eventi trigger, ossia le condizioni che devono attivare l'invio dei report, la frequenza temporale (time scale) con cui il nodo E2 deve inviare i messaggi di report. In base alla periodicità definita, viene impostato un timer nel nodo E2, e al suo scadere viene inviato un messaggio di report al RIC. Il messaggio di RIC Indication generato è di tipo report

Il servizio E2 Insert, invece, riguarda l'invio di messaggi dal nodo E2 a una xApp nel RIC near-RT per notificare il verificarsi di un evento specifico all'interno del nodo E2. Anche in questo caso, il servizio si attiva tramite una sottoscrizione effettuata dalla xApp, che richiede al nodo E2 di notificare il verificarsi di specifici eventi. Questo servizio è spesso utilizzato per ricevere notifiche su condizioni che richiedono un'azione immediata, come il superamento di soglie prestazionali o la disponibilità di nuove risorse nella rete. Ad esempio, il nodo potrebbe segnalare che un dispositivo utente (UE) ha segnalato l'opportunità di eseguire un handover. Anche questo servizio si attiva tramite la sottoscrizione della funzione da parte di una xApp e implica l'invio di un messaggio di RIC Indication, questa volta di tipo insert. In questo scenario, il trigger è associato a una procedura di gestione delle risorse radio (RAN radio resource management) che viene sospesa al momento dell'invio del messaggio insert. Viene inoltre avviato un timer di attesa: se il RIC non risponde prima della scadenza del timer, il nodo E2 può riprendere la procedura sospesa o interromperla definitivamente.

Il servizio E2 Control consente al RIC near-real-time (near-RT) di inviare comandi di controllo ai nodi E2. Questo servizio può essere avviato in due modi: autonomamente dal Near-RT RIC, per eseguire azioni di controllo programmate, oppure in risposta alla ricezione di un messaggio E2 INSERT, quando il nodo E2 segnala un evento che richiede l'intervento del RIC. La procedura del servizio di controllo si basa su uno scambio a due messaggi: Una RIC Control Request, inviata dal Near-RT RIC al nodo E2, contenente i comandi di riconfigurazione per modificare specifici parametri esposti dalle funzioni RAN del nodo E2, Una RIC Control Acknowledge, inviata dal nodo E2 al RIC, come conferma dell'avvenuta esecuzione del comando, del rifiuto dell'operazione o segnalare eventuali errori. Questo servizio è essenziale per consentire al RIC di modificare in tempo reale i parametri della RAN, migliorando l'efficienza e l'adattività della rete.

L'ultimo servizio, denominato E2 Policy riguarda invece la definizione di politiche di gestione delle risorse radio che il nodo E2 deve eseguire autonomamente. L'attivazione di questo servizio richiede una procedura di sottoscrizione (subscription) in cui vengono specificati: Un evento trigger, che definisce la condizione che attiva quando il nodo E2 deve applicare la politica, Una policy, ossia una serie di regole che il nodo E2 deve seguire per la gestione delle risorse radio (Radio Resource Management - RRM). Il servizio E2 POLICY permette di automatizzare l'applicazione di politiche per la gestione della rete, rendendo i nodi E2 autonomi nell'esecuzione di azioni prestabilite in risposta a eventi specifici, senza richiedere un intervento continuo da parte del RIC.

In sintesi, questi quattro servizi E2 (REPORT, INSERT, CONTROL, POLICY) costituiscono il meccanismo chiave per la comunicazione tra il Near-RT RIC e i nodi E2, abilitando un controllo dinamico e adattivo della rete, essenziale per le moderne architetture O-RAN basate su intelligenza artificiale e cicli di controllo ad anello chiuso (closed-loop control). Questi servizi (E2 Report, E2 Insert, E2 Control, E2 Policy) possono essere combinati per creare un modello di servizio (Service Model - E2SM). Il modello di servizio è incluso come payload all'interno di uno dei messaggi del protocollo E2AP, come mostrato nella Figura 6. [1] [4]

La procedura di E2 Reset, invece, permette di ripristinare l'interfaccia E2, potendo essere iniziata sia dal Near-RT RIC sia dal nodo E2. È importante sottolineare che tutte le informazioni precedentemente scambiate, comprese quelle relative alla procedura di E2 Setup, agli aggiornamenti di configurazione e agli aggiornamenti dei servizi, rimangono memorizzate dopo il ripristino. In caso di modifiche ai servizi supportati, il nodo E2 utilizza la procedura di Near-RT RIC Service Update per notificare il Near-RT RIC. Questa procedura segnala sia l'aggiornamento dell'elenco dei servizi supportati, sia l'eventuale modifica della mappatura tra servizi e funzioni. L'invio dell'aggiornamento può essere avviato su richiesta del Near-RT RIC, attraverso un messaggio denominato RIC Service Query. Infine, la procedura di E2 Node Configuration Update viene utilizzata dal nodo E2 per comunicare al Near-RT RIC eventuali modifiche alla propria configurazione interna o alla connessione E2.

Il contenuto effettivo del messaggio è codificato utilizzando la notazione ASN.1 (Abstract Syntax Notation One), che definisce in modo rigoroso i tipi di dati, come numeri e coppie chiave-valore (key-value pairs), garantendo così l'interoperabilità tra i diversi componenti della rete.

Queste procedure costituiscono l'insieme delle operazioni fondamentali per mantenere la comunicazione tra il Near-RT RIC e i nodi E2, garantendo l'aggiornamento continuo delle informazioni, la coesione operativa e l'efficienza del controllo nella rete O-RAN. Infine, il Near-RT RIC è connesso al livello di Service Management & Orchestration (SMO) tramite l'interfaccia O1, utilizzata per: Gestione degli elementi (Element Management), come la configurazione e il monitoraggio dei componenti, Orchestrazione dei servizi, per la gestione del ciclo di vita delle applicazioni e delle risorse di rete, che dettaglieremo nella sezione successiva.

1.7 Interfaccia O1

L'interfaccia O1 è un elemento fondamentale dell'architettura O-RAN, progettata per consentire la gestione, il monitoraggio e la configurazione dei componenti della RAN attraverso il framework Service Management and Orchestration (SMO). La sua funzione principale è quella di fornire una connessione standardizzata tra l'SMO e i vari componenti O-RAN, come il Near-RT RIC, il Non-RT RIC, l'O-Cloud, le O-CU, le O-DU e le O-RU.

L'interfaccia O1 svolge principalmente due compiti essenziali. Il primo è la gestione del ciclo di vita dei componenti O-RAN, il che include operazioni come l'installazione, l'aggiornamento, la configurazione, il monitoraggio e la dismissione delle risorse. Il secondo compito è la raccolta e l'esposizione dei dati sulle prestazioni (Performance Management), come il monitoraggio delle

metriche di traffico, l'identificazione di anomalie e l'analisi degli indicatori chiave di prestazione (Key Performance Indicators - KPI).

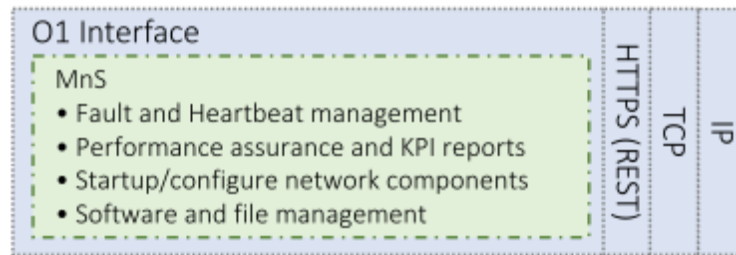


Figura 1.7 Funzionalità e protocolli dell'interfaccia O1 per la gestione e supervisione della rete (MnS) [1]

Come evidenziato nella Figura 1.7, l'interfaccia O1 si basa su servizi di gestione degli elementi (Management Services - MnS), definiti secondo il modello FCAPS (Fault, Configuration, Accounting, Performance, Security), una struttura standard per la gestione delle reti. All'interno dell'interfaccia O1, i principali servizi MnS includono:

- Gestione delle configurazioni (O1-CM): Permette la configurazione iniziale e dinamica dei componenti O-RAN, facilitando la distribuzione delle configurazioni tramite file ConfigMap o API REST.
- Monitoraggio delle prestazioni (O1-PM): Consente la raccolta di metriche in tempo reale, come throughput, latenza e qualità delle connessioni.
- Gestione degli allarmi (O1-FM): Raccoglie segnalazioni sugli errori di sistema e sugli eventi critici, notificando l'SMO per consentire una risposta rapida.
- Gestione della sicurezza (O1-SEC): Fornisce controlli di accesso, auditing e monitoraggio degli eventi di sicurezza.

Il flusso di comunicazione tramite l'interfaccia O1 segue una struttura chiara e ben definita. Quando un componente della RAN (ad esempio una O-DU) viene avviato, l'SMO tramite l'interfaccia O1 esegue una procedura di configurazione iniziale, inviando parametri operativi e politiche di gestione. Successivamente, il componente monitora il proprio stato operativo e trasmette all'SMO, attraverso O1, rapporti periodici e segnalazioni di eventi critici, come guasti o degradi delle prestazioni. In caso di allarme, l'SMO può inviare tramite O1 istruzioni per la risoluzione del problema, come il riavvio del componente o l'applicazione di una nuova configurazione.

L'interfaccia O1 trova applicazione in molteplici scenari pratici. Ad esempio, può essere utilizzata per eseguire aggiornamenti software su una O-DU senza interrompere il servizio, gestire le configurazioni delle celle in base ai modelli di traffico oppure automatizzare la risposta agli allarmi, come l'attivazione di una cella di riserva in caso di guasto. Un altro esempio è l'integrazione con sistemi di analisi predittiva: l'SMO, analizzando i dati sulle prestazioni ricevuti tramite O1, può prevedere possibili degradi della rete e applicare azioni correttive preventive.

L'interfaccia O1 riveste un ruolo cruciale nell'ecosistema O-RAN perché garantisce una gestione centralizzata e standardizzata dei componenti della rete. Inoltre, separando nettamente le funzioni di gestione e controllo operativo, favorisce l'interoperabilità tra componenti di fornitori diversi, contribuendo allo sviluppo di un ecosistema multi-vendor aperto e scalabile. In sintesi, l'interfaccia O1 è la spina dorsale delle operazioni di gestione della rete nell'architettura O-RAN, poiché assicura una gestione efficiente e dinamica del ciclo di vita dei componenti, supportando l'implementazione di reti 5G e future sempre più automatizzate, intelligenti e resilienti. [1][3][4][5]

1.8 Interfaccia A1

L'interfaccia A1 è una componente essenziale dell'architettura O-RAN che permette la comunicazione tra il Non-Real-Time RIC (Non-RT RIC) e il Near-Real-Time RIC (Near-RT RIC). Il suo scopo principale è supportare la gestione strategica della rete, facilitando l'integrazione di algoritmi di intelligenza artificiale (AI) e apprendimento automatico (ML) per ottimizzare il funzionamento della RAN.

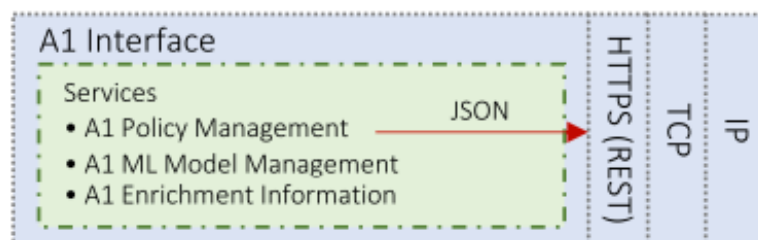


Figura 1.8 Panoramica dell'interfaccia A1 [1]

Come si può notare nella Figura 1.8 l'interfaccia A1 svolge due funzioni fondamentali. La prima è la gestione delle politiche (A1 Policies), attraverso la quale il Non-RT RIC trasmette al Near-RT RIC direttive che guidano il comportamento delle xApp. Queste politiche stabiliscono obiettivi di rete, come la riduzione del consumo energetico o il miglioramento della Qualità del Servizio (QoS). La seconda funzione è l'arricchimento dei dati (A1 Enrichment Information), con cui il Non-RT RIC fornisce al Near-RT RIC informazioni contestuali, come analisi predittive o dati storici, per supportare decisioni più accurate e migliorare l'efficacia degli algoritmi AI/ML utilizzati dalle xApp.

L'interfaccia A1 è composta da tre elementi principali. Il primo è l'A1 Mediator, che si trova all'interno del Near-RT RIC e gestisce la ricezione delle politiche e delle informazioni di arricchimento, distribuendole alle xApp. Il secondo è l'A1 Policies Repository, un archivio interno al Near-RT RIC che conserva le politiche ricevute e le rende disponibili alle xApp. Il terzo è l'A1 Interface Client, il componente del Non-RT RIC responsabile dell'invio delle politiche e delle informazioni di arricchimento.

Il flusso di comunicazione tramite l'interfaccia A1 segue un processo preciso. Il Non-RT RIC analizza i dati storici e i modelli predittivi per definire una serie di politiche. Queste vengono inviate al Near-RT RIC tramite l'A1 Mediator, che le inoltra alle xApp pertinenti. Le xApp utilizzano le politiche per guidare il proprio comportamento operativo, ad esempio regolando l'allocazione delle risorse radio. Inoltre, il Non-RT RIC può trasmettere informazioni di arricchimento per supportare l'addestramento dei modelli AI/ML e migliorare l'accuratezza delle previsioni.

L'interfaccia A1 trova applicazione in diversi scenari pratici. Un esempio è l'ottimizzazione delle risorse radio, dove il Non-RT RIC invia una politica per ridurre il consumo energetico nelle celle a bassa attività, consentendo al Near-RT RIC di abbassare la potenza trasmessa. Un altro caso è la gestione della Qualità del Servizio (QoS), in cui il Non-RT RIC stabilisce una politica per garantire la priorità al traffico voce rispetto al traffico dati. Inoltre, l'interfaccia A1 supporta l'apprendimento automatico distribuito, permettendo al Near-RT RIC di utilizzare modelli AI/ML aggiornati con dati storici per migliorare continuamente le prestazioni di rete.

L'interfaccia A1 è fondamentale per l'architettura O-RAN perché separa chiaramente i cicli di controllo: il Non-RT RIC si occupa delle decisioni strategiche a lungo termine, mentre il Near-RT RIC esegue il controllo operativo in tempo quasi reale. Questa separazione consente di integrare algoritmi AI/ML per ottimizzare la rete in modo dinamico, adattandosi alle condizioni operative in tempo reale. Inoltre, l'adozione di uno standard aperto per l'interfaccia A1 garantisce l'interoperabilità tra componenti di fornitori diversi, favorendo un ecosistema multi-vendor.

In conclusione, l'interfaccia A1 rappresenta un collegamento strategico tra il controllo strategico e quello operativo della rete, contribuendo a rendere l'architettura O-RAN più intelligente, adattiva e pronta a soddisfare le esigenze delle reti 5G e future. [1][3][4][5]

1.9 xApp

Dal punto di vista funzionale, le xApp sono microservizi discreti [2] che implementano logiche di business ben definite per gestire le RAN. Questa logica può includere la raccolta e l'elaborazione di dati dai nodi E2, il calcolo di metriche per generare report o attivare allarmi, e il controllo di vari aspetti dei nodi E2 in base a un determinato algoritmo. Dal punto di vista dell'implementazione, le xApp sono pod Kubernetes che operano all'interno del cluster Near-RT RIC, ognuno dei quali può contenere uno o più container Docker. Le applicazioni eseguite nei container Docker implementano la logica aziendale delle xApp, sfruttando una libreria del framework xApp disponibile in Python, C++, Go o Rust. Questa libreria fornisce alle xApp una serie di API per sfruttare le funzionalità offerte dai diversi componenti del Near-RT RIC.

Le xApp sono i componenti principali del RIC near-RT. Una xApp è un componente plug-and-play [1] che implementa logiche personalizzate, ad esempio per l'analisi dei dati della RAN e il controllo della RAN. Le xApp ricevono dati e telemetria dalla RAN e inviano comandi di controllo tramite l'interfaccia E2. Il descrittore di una xApp (ad esempio, un file YAML o JSON) contiene le informazioni necessarie per la gestione della xApp, tra cui: Policy di autoscaling; Configurazioni per il deployment; Procedure di eliminazione; Informazioni sugli aggiornamenti.

Uno degli obiettivi principali delle xApp è ottimizzare la gestione delle risorse radio (Radio Resource Management - RRM) nei nodi E2.

1.9.1 Interfacce e Sviluppo delle xApp

Interfacce Principali delle xApp (Figura 1.9) [2]:

- **Configurazione O1-CM:** Durante l'avvio, il file descrittivo della xApp fornisce parametri di configurazione iniziali per interagire con il Near-RT RIC e parametri opzionali per personalizzare il funzionamento della xApp. Questo file viene caricato come ConfigMap nel pod Kubernetes, consentendo agli utenti del Near-RT RIC (ad esempio, amministratori di sistema e operatori di rete) di modificare i parametri opzionali durante il runtime.
- **Messaggistica RMR:** Permette alle xApp di comunicare tra loro e con i componenti del Near-RT RIC tramite una libreria di messaggistica a bassa latenza. RMR utilizza un paradigma di tipo publish-subscribe, consentendo alle xApp di ignorare gli indirizzi IP di altri pod Kubernetes e di scambiare messaggi in base ai tipi di messaggi. Inoltre, RMR serve come API attraverso cui le xApp ricevono le politiche A1 dal Non-RT RIC tramite l'A1 Mediator.
- **Archiviazione SDL/STSL:** Fornisce alle xApp l'accesso al database condiviso all'interno del Near-RT RIC, facilitando operazioni di lettura/scrittura per l'archiviazione persistente e astruendo i dettagli di implementazione specifici del database sottostante. Gestisce anche i processi di autenticazione e autorizzazione per accedere al database, garantendo che le xApp rimangano portabili e senza stato.
- **HTTP/REST:** Fornisce alle xApp callback REST per gestire richieste HTTP, consentendo agli sviluppatori di personalizzare le risposte ai controlli di "liveness" e "readiness" di Kubernetes in base ai requisiti delle xApp. Inoltre, è possibile creare callback REST per esporre informazioni interne sulla logica di business della xApp e rispondere a comandi e parametri esterni, consentendo agli utenti di interagire direttamente con le xApp tramite HTTP.

- **Sottoscrizione E2:** Consente alle xApp di ottenere informazioni dalla RAN e di controllarne il funzionamento. Le xApp possono sottoscrivere a metriche e aggiornamenti di un determinato set di nodi E2 per l'elaborazione successiva, l'analisi dei dati e il controllo dei nodi E2 in base alla loro logica aziendale, passando parametri o attivando le funzioni RAN supportate espone dai loro modelli di servizio.

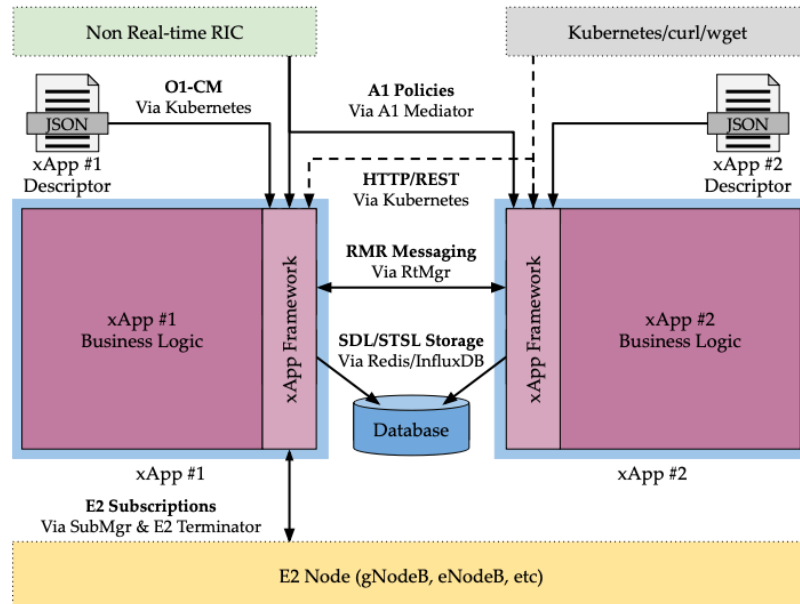


Figura 1.9 Flusso di comunicazione tra xApp, Near-RT RIC e E2 Node [2]

Una xApp richiede un descrittore valido fornito tramite O1-CM per funzionare [2], in quanto contiene la configurazione iniziale necessaria per abilitare il deployment della xApp e l'interazione con altri componenti del Near-RT RIC, se necessario. Inoltre, il descrittore della xApp istruisce l'AppMgr su come installare la xApp, compreso il recupero delle immagini Docker da un registro Docker accessibile, la configurazione del pod Kubernetes e la notifica ad altri componenti del Near-RT RIC della creazione di una nuova xApp (ad esempio, RtMgr e SubMgr) per consentire alla nuova xApp di sfruttare le loro capacità.

Per implementare in modo efficace azioni di controllo e applicare politiche, il modello di servizio E2SM (E2 Service Model) utilizza una combinazione dei servizi dell'interfaccia E2 descritti in precedenza. La Figura 1.9 illustra un esempio di scambio di messaggi tra un nodo E2 e una xApp nel RIC near-real-time (near-RT) durante una procedura tipica di controllo. Il processo inizia quando la xApp sottoscrive una funzione RAN Control esposta dal nodo E2. Durante questa fase di sottoscrizione, la xApp può definire un evento di attivazione (trigger) o un timer. Quando il timer scade o si verifica la condizione definita nel trigger, il nodo E2 invia un messaggio di tipo E2 Insert al RIC. Ad esempio, questo può accadere quando una metrica monitorata dal nodo (come il gNB) supera una determinata soglia. Questo evento è noto come Evento A3 secondo le specifiche 3GPP [1]. In tale scenario, il nodo E2 invia un messaggio di richiesta di controllo per l'handover, includendo l'identificativo della cella target, sospende la procedura di gestione delle risorse radio e avvia un timer. A questo punto possono verificarsi tre esiti:

- Il RIC near-RT risponde con un messaggio di E2 Control rifiutando la richiesta di controllo.
- Il RIC near-RT accetta la richiesta e invia un'azione di controllo, ad esempio indicando l'ID della cella target selezionata dalla xApp.
- Se il timer scade prima che il nodo E2 riceva una risposta dal RIC, la procedura può: continuare autonomamente, secondo la configurazione del nodo E2 oppure essere definitivamente interrotta, a seconda della procedura e delle impostazioni specifiche del nodo E2.

Questa architettura a controllo chiuso consente una gestione dinamica e adattiva delle risorse radio, migliorando l'efficienza della rete e la qualità dell'esperienza utente. Tutte queste operazioni avvengono tramite meccanismi ben definiti, basati su una sintassi strutturata secondo uno schema JSON, capace di esprimere politiche e intenzioni ad alto livello (high-level intents). Le politiche, i modelli di ML e le informazioni di arricchimento possono riferirsi sia a gruppi di utenti (UEs - User Equipments) sia a singoli utenti specifici.

Il RIC near-RT offre servizi e API per la gestione automatizzata del ciclo di vita delle xApp, che comprende: Onboarding (registrazione e caricamento); Deploy (distribuzione); Termination.

Nell'OSC, anche le xApp sono microservizi cloud-native [2], ovvero pod Kubernetes basati su container Docker, che operano all'interno del cluster Near-RT RIC. Pertanto, è fondamentale che gli sviluppatori di xApp comprendano i fondamenti di queste tecnologie cloud e le utilizzino in modo appropriato. Inoltre, l'OSC ha creato un SDK per le xApp, che facilita lo sviluppo di xApp in diversi linguaggi e l'interazione con i componenti del Near-RT RIC. Questo SDK verrà utilizzato nel corso di questo documento per sviluppare e gestire le xApp.

Nonostante gli sforzi di standardizzazione e sviluppo di O-RAN, la creazione di un SDK per sviluppare xApp in diversi linguaggi di programmazione e una crescente comunità di sviluppatori accademici e industriali, il processo di creazione delle xApp rimane complesso. Dal punto di vista dell'implementazione, le xApp sono microservizi altamente complessi che interagiscono con diversi componenti del Near Real-Time RAN Intelligent Controller (Near-RT RIC) tramite API e protocolli differenti, rendendo difficile per i principianti iniziare a prototipare le proprie xApp.

I pochi lavori esistenti sullo sviluppo di xApp affrontano considerazioni specifiche, come i flussi di dati tra entità O-RAN, interazioni con le stazioni base o la progettazione di agenti di Deep Reinforcement Learning (DRL), senza fornire un contesto su tutte le funzionalità e capacità disponibili per gli sviluppatori di xApp. Inoltre, alcuni aspetti delle xApp sono ancora in fase di standardizzazione attiva, come i flussi di lavoro AI/ML, mentre altri sono stati lasciati per ulteriori studi, come la sicurezza all'interno del Near-RT RIC, o sono stati completamente rivisti nelle recenti versioni di O-RAN, ad esempio gli abbonamenti alle informazioni dalla RAN.

1.10 E2 Service Model

L'O-RAN Alliance ha sviluppato quattro modelli di servizio E2 (E2SM - E2 Service Models) [4] per fornire uno scambio di informazioni standardizzato e strutturato tra i nodi E2 e il Near-RT RIC. I modelli di servizio E2 (E2SM - E2 Service Models) vengono realizzati attraverso l'interfaccia E2, che consente lo scambio di dati e comandi tra il Near-RT RIC e i nodi E2, implementando diverse funzionalità per la raccolta di informazioni, il controllo delle risorse e l'ottimizzazione della rete.

Il modello **E2SM KPM (Key Performance Measurements)** [4] è progettato per la raccolta delle metriche di prestazione della rete RAN. Questo avviene attraverso il servizio E2 REPORT, che consente ai nodi E2 di inviare dati relativi alle prestazioni al Near-RT RIC. Durante la procedura di E2 Setup, il nodo E2 dichiara l'elenco delle metriche disponibili (KPMs), che possono includere parametri come RSSI, SNR, throughput, latenza e altri KPI di rete. Una volta stabilita la connessione, una xApp nel Near-RT RIC può inviare un messaggio di sottoscrizione (subscription) specificando le metriche di interesse e il tipo di reporting desiderato. I report possono essere generati periodicamente o su base event-triggered. Il nodo E2 trasmette quindi i dati tramite messaggi di tipo E2 Indication, consentendo al Near-RT RIC di analizzare le informazioni e prendere decisioni per l'ottimizzazione della rete.

Il modello **E2SM NI (Network Interface)** [4] è progettato per trasferire i messaggi di report relativi a specifiche interfacce di rete dai nodi E2 al Near-RT RIC. Durante la procedura di sottoscrizione, il nodo E2 comunica al Near-RT RIC quali interfacce supporta per la raccolta dei dati. Una volta stabilita la

sottoscrizione, i report sulle prestazioni delle interfacce vengono inviati al Near-RT RIC attraverso l'interfaccia E2, permettendo il monitoraggio continuo dello stato della rete.

Il modello **E2SM RC (RAN Control)** [4] introduce un meccanismo di controllo avanzato per l'ottimizzazione delle risorse radio (RRM - Radio Resource Management) nei nodi E2. Questo controllo avviene tramite il servizio E2 CONTROL, che può essere attivato autonomamente dal Near-RT RIC o in risposta a un messaggio E2 INSERT proveniente dal nodo E2.

L'interazione tra il Near-RT RIC e il nodo E2 avviene attraverso due tipi di messaggi principali:

- RIC Control Request: il Near-RT RIC invia al nodo E2 istruzioni specifiche per la riconfigurazione delle risorse radio, come ad esempio la modifica della potenza di trasmissione, la variazione dei parametri di scheduling o la riallocazione delle risorse.
- RIC Control Acknowledge: il nodo E2 conferma l'avvenuta esecuzione delle istruzioni ricevute.

Questo modello è particolarmente utile nelle reti 5G, dove il Near-RT RIC può implementare controlli ad anello chiuso (closed-loop control) per massimizzare le prestazioni della rete e garantire una migliore qualità del servizio.

Il modello **E2SM CCC (Cell Configuration and Control)** [4] si occupa della gestione delle configurazioni a livello di cella (cell-level) e slice (slice-level) nei nodi RAN. Sul piano del controllo, questo modello è responsabile dell'allocazione delle risorse per le slice (slice resource allocation control), regolando le risorse radio assegnate a ciascuna slice. Questa funzione include il controllo dei blocchi di risorse (Resource Blocks) dedicati al uplink e al downlink, nonché la gestione delle risorse assegnate alle slice a livello di DU (Distributed Unit). Grazie a questo modello, il Near-RT RIC può gestire dinamicamente le risorse di rete per garantire il miglior compromesso tra prestazioni e utilizzo efficiente delle risorse disponibili.

2 Tecnologie 5G e IoT

Le reti di telecomunicazione hanno subito un'evoluzione significativa nel corso delle ultime decadi, passando da sistemi analogici con funzionalità limitate a reti digitali ad alte prestazioni, in grado di supportare una vasta gamma di servizi e applicazioni. Il 5G rappresenta la più recente generazione di reti mobili e introduce un'architettura altamente innovativa, progettata per migliorare la velocità di connessione, ridurre la latenza e supportare un numero crescente di dispositivi connessi.

Oltre alle prestazioni superiori rispetto al 4G LTE, il 5G si distingue per la capacità di abilitare nuovi scenari applicativi, come la comunicazione ultra-affidabile a bassa latenza (URLLC), le comunicazioni massive tra macchine (mMTC) e il miglioramento della banda larga mobile (eMBB). Queste caratteristiche rendono il 5G una tecnologia abilitante per la trasformazione digitale in molteplici settori, tra cui la manifattura, la sanità, l'automotive e le smart cities.

Parallelamente, l'Internet of Things (IoT) è una delle principali aree di applicazione del 5G. Grazie alla capacità di gestire milioni di dispositivi connessi in maniera efficiente e a basso consumo energetico, il 5G consente di realizzare ecosistemi IoT altamente scalabili e performanti. L'integrazione tra 5G e IoT sta rivoluzionando la gestione delle infrastrutture urbane, l'automazione industriale e l'agricoltura di precisione, abilitando nuovi modelli operativi basati su reti intelligenti e servizi digitalizzati.

Tuttavia, nonostante il grande potenziale, l'implementazione del 5G e dell'IoT presenta una serie di sfide tecnologiche e operative, tra cui la gestione della sicurezza, la scalabilità della rete, l'ottimizzazione delle risorse e l'integrazione con le architetture esistenti.

In questo capitolo verranno approfonditi i principali aspetti legati al 5G e all'IoT, analizzandone il funzionamento, i benefici e le sfide legate alla loro implementazione. In particolare:

- Fornirà una panoramica dell'evoluzione delle reti mobili, analizzando il passaggio dalle generazioni precedenti (1G-4G) al 5G e illustrando le innovazioni chiave introdotte da questa nuova tecnologia.
- Esplorerà l'architettura del 5G, spiegando il funzionamento del network slicing, del massive MIMO e delle altre tecnologie abilitanti.
- Approfondirà il ruolo del 5G nell'IoT, evidenziando come questa tecnologia permetta di supportare miliardi di dispositivi connessi in maniera efficiente.
- Descriverà l'architettura del 5G-IoT, spiegando i diversi livelli tecnologici e le modalità di interazione tra i dispositivi connessi.
- Fornirà una panoramica sulle prospettive future di queste tecnologie, considerando le evoluzioni previste per il 6G e il loro impatto su settori emergenti come le reti non terrestri e l'intelligenza artificiale applicata alle telecomunicazioni.

Questo capitolo fornirà quindi una visione completa dell'ecosistema 5G e IoT, illustrando le potenzialità, i limiti e le opportunità offerte da questa rivoluzione tecnologica.

2.1 Evoluzione delle reti mobili e il ruolo del 5G

Le reti mobili hanno subito un'evoluzione significativa nel corso delle generazioni, offrendo servizi sempre più avanzati e migliorando continuamente le capacità di connessione. Ogni generazione ha rappresentato un passo avanti nelle prestazioni, introducendo nuove funzionalità e ampliando le possibilità di utilizzo della rete mobile.

Come illustrato nella Figura 2.1, il 5G rappresenta la nuova generazione di reti mobili, progettata per superare i limiti del 4G LTE e abilitare un'ampia gamma di innovazioni tecnologiche. A differenza

delle reti 3G e 4G, il 5G non solo offre connessioni più veloci e affidabili, ma introduce anche il supporto per tecnologie avanzate come la comunicazione su lunghe distanze e le reti a basso consumo energetico (LPWA - Low Power Wide Area), precedentemente incompatibili con le generazioni precedenti.

- **1G (Anni '80):** la prima generazione di reti mobili, basata su tecnologia analogica, ha introdotto le prime chiamate vocali senza fili. Tuttavia, offriva una qualità audio scadente e bassi livelli di sicurezza.
- **2G (Anni '90):** ha segnato il passaggio dai segnali analogici a quelli digitali, consentendo l'invio di SMS e MMS. Sebbene il suo utilizzo sia stato progressivamente dismesso in molte regioni, tra cui Europa e Nord America, il 2G è ancora diffuso in diverse aree in via di sviluppo.
- **3G (Primi anni 2000):** con l'introduzione degli standard UMTS (Universal Mobile Telecommunication System), ha reso possibile la navigazione web da dispositivi mobili e lo streaming di musica e video. Ancora oggi il 3G è utilizzato in diverse parti del mondo.
- **4G (Dal 2010 circa):** ha rappresentato un grande salto qualitativo, migliorando notevolmente velocità di connessione e larghezza di banda. Questa generazione ha abilitato applicazioni avanzate come streaming video in alta definizione, gaming online e utilizzo intensivo di app mobile, garantendo prestazioni elevate anche senza connessione Wi-Fi.
- **5G (Dal 2019 circa):** l'ultima evoluzione delle reti mobili, attualmente in fase di implementazione su scala globale. Il 5G supera i limiti del 4G, offrendo: velocità di trasmissione dati molto elevate, minore latenza, maggiore capacità di connessione simultanea per dispositivi IoT e smart devices. Il 5G sfrutta tecnologie avanzate come Massive MIMO (Multiple Input Multiple Output), beamforming e network slicing, che migliorano ulteriormente capacità, copertura ed efficienza della rete.



Figura 2.1 Evoluzione delle generazioni di rete mobile: dal 1G al 5G [7]

Le reti di quinta generazione (5G) introducono servizi e casi d'uso con requisiti estremamente stringenti in termini di throughput, latenza, affidabilità, sicurezza e privacy. Queste esigenze sono soddisfatte dai sistemi wireless 5G, che integrano nuove tecnologie come il massive Multiple Input Multiple Output (MIMO) [4], le comunicazioni su onde millimetriche e nella banda sub-terahertz, l'elaborazione del segnale supportata dal machine learning (ML) e la pianificazione, ottimizzazione e diagnostica della rete abilitate dall'ML. Tuttavia, l'adozione di queste tecnologie aumenta notevolmente la complessità, che i sistemi di gestione e orchestrazione degli operatori devono affrontare per mantenere prestazioni elevate e garantire il corretto funzionamento della rete.

Con l'evoluzione delle reti verso il 6G, emergono nuovi casi d'uso che metteranno ulteriormente alla prova l'architettura O-RAN. Tecnologie come le reti non terrestri (NTN), le comunicazioni sub-terahertz e le applicazioni immersive per il metaverso, basate su AR (Augmented Reality) e VR (Virtual Reality), richiederanno reti ancora più adattive, intelligenti e resilienti. In questo contesto, il paradigma O-RAN, con la sua apertura e modularità, rappresenta una piattaforma ideale per sperimentare e implementare le innovazioni necessarie a supportare le reti oltre il 5G.

Per soddisfare i requisiti aziendali e di comunicazione, le reti cellulari stanno attraversando un cambiamento paradigmatico, guidato da concetti come la softwarizzazione, la disaggregazione, l'uso di interfacce aperte e di hardware generico e programmabile, in sostituzione delle tradizionali architetture basate su hardware specializzato. Questo cambiamento apre la strada a reti più flessibili, agili, cloud-native, interoperabili, guidate dai dati e ottimizzate tramite algoritmi di Machine Learning (ML).

Nei prossimi paragrafi verranno esplorate nel dettaglio le tecnologie chiave del 5G e il loro impatto sull'IoT, analizzando le opportunità e le sfide che questa rivoluzione comporta.

2.2 Innovazioni del 5G

Con la crescente digitalizzazione dei processi e dei prodotti, le organizzazioni stanno orientandosi sempre più verso l'innovazione, adottando nuove tecnologie per trasformare i propri modelli operativi. L'analisi dei dati in tempo reale consente ai decisori di prendere decisioni aggiornate, basate sulle informazioni degli utenti, rendendo i processi aziendali più agili e reattivi.

Un'innovazione fondamentale abilitata dal 5G è il Network Slicing [7], una tecnologia che permette di creare partizioni virtuali della rete, chiamate slices, ognuna delle quali è dedicata a uno specifico servizio o applicazione. Il Network Slicing, completamente supportato dal 5G, consente a sensori e dispositivi di accedere a risorse dedicate della rete per soddisfare le loro esigenze operative. Ogni slice è completamente isolata dalle altre, garantendo sicurezza e indipendenza operativa. Questa separazione è resa possibile grazie alla funzionalità di CUPS (Control Plane and User Plane Separation), che suddivide il piano di controllo (Control Plane) dal piano utente (User Plane) e introduce un ulteriore livello, il piano di gestione (Management Plane).

Il 5G non è solo un miglioramento delle prestazioni rispetto alle generazioni precedenti, ma introduce un'architettura completamente nuova, progettata per soddisfare una vasta gamma di applicazioni con requisiti molto differenti. Il **3GPP (3rd Generation Partnership Project)**, l'organizzazione che definisce gli standard delle telecomunicazioni mobili, ha suddiviso le funzionalità del 5G in tre categorie principali [12]:

- **Enhanced Mobile Broadband (eMBB):** Il 5G eMBB rappresenta l'evoluzione naturale delle reti mobili per l'accesso a Internet ad alta velocità, migliorando significativamente la capacità e la velocità di trasmissione dei dati rispetto al 4G LTE (che arriva fino a 1 Gbps). Il suo obiettivo principale è fornire una connettività ultraveloce e affidabile in scenari ad alta densità di traffico, come aree urbane, eventi affollati e trasporti pubblici, per supportare più utenti simultaneamente in ambienti congestionati [12].
- **Ultra-Reliable Low Latency Communications (URLLC):** Il 5G URLLC è progettato per applicazioni critiche che richiedono estrema affidabilità e latenza ultra-bassa. La riduzione della latenza sotto il millisecondo (1 ms) contro i 30-50 ms tipici del 4G LTE, e un'affidabilità del 99,999% per garantire trasmissioni senza errori anche in ambienti critici, permettono l'uso del 5G in scenari in cui ogni millisecondo è fondamentale [12].
- **Massive Machine Type Communications (mMTC):** Il 5G mMTC è progettato per connettere un numero elevatissimo di dispositivi, supportando oltre 1 milione di dispositivi per km², garantendo connettività su larga scala in modo efficiente ed economico, riducendo il consumo energetico e ottimizzando la gestione della rete. Questa tecnologia è ideale per l'Internet of Things (IoT), dove milioni di sensori, dispositivi e macchine comunicano in tempo reale con la rete [12].

Il 5G introduce una serie di caratteristiche innovative progettate per migliorare l'efficienza, la flessibilità e le capacità della rete [7]:

- **Supporto per più tecnologie:** Il 5G è in grado di integrare diverse tecnologie di comunicazione, come LoRa, che è specificamente progettata per dispositivi a basso consumo energetico, fondamentali nell'ecosistema IoT.
- **Mobilità senza interruzioni (Seamless Mobility):** Consente il passaggio fluido tra diverse reti senza perdita di connessione.

- Flessibilità: L'architettura del 5G è modulare e programmabile, capace di adattarsi a diversi casi d'uso.
- KPI avanzati (Advanced KPIs): Offre indicatori di prestazione chiave più sofisticati per monitorare l'efficienza della rete.
- Servizi multimediali in tempo reale e non: Supporta la trasmissione simultanea di contenuti in real-time e non real-time, migliorando l'esperienza utente.
- Personalizzazione delle reti: Grazie al Network Slicing, il 5G può creare sezioni di rete virtuali dedicate a servizi specifici.
- Efficienza nell'uso delle risorse (Resource Efficiency): Riduce lo spreco di risorse grazie a un'allocazione intelligente e dinamica.

2.3 Internet of Things (IoT) e il ruolo del 5G

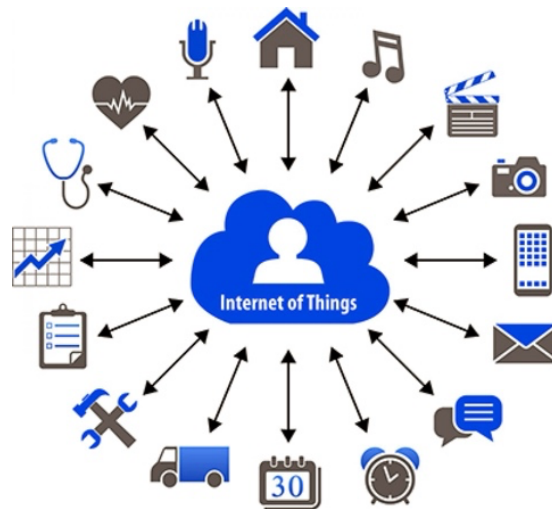


Figura 2.2 Ecosistema dell'Internet of Things (IoT): interconnessione di dispositivi e servizi
<https://www.federprivacy.org/informazione/sapresti-rispondere/cosa-significa-internet-of-things>

Il termine **Internet of Things (IoT)** si riferisce generalmente a scenari in cui la connettività di rete e la capacità computazionale si estendono a oggetti, sensori e dispositivi di uso quotidiano non normalmente considerati computer, consentendo a questi dispositivi di generare, scambiare e consumare dati con un intervento umano minimo [32]. Tuttavia, non esiste una definizione univoca e universalmente accettata del concetto di IoT, poiché le sue implementazioni possono variare a seconda del contesto tecnologico e applicativo.

Sistemi IoT avanzati, come veicoli connessi, sistemi di traffico intelligente e sensori integrati in strade e ponti, stanno accelerando lo sviluppo delle smart city [32], contribuendo alla riduzione della congestione urbana e del consumo energetico. Grazie alla capacità di raccogliere e analizzare grandi quantità di dati in tempo reale, le reti IoT rappresentano una risorsa fondamentale per l'ottimizzazione delle infrastrutture urbane e industriali.

I sistemi IoT adottano diversi modelli di comunicazione, ognuno con caratteristiche specifiche. Quattro modelli comuni, descritti dall'Internet Architecture Board [32], includono: Device-to-Device, in cui i dispositivi comunicano direttamente tra loro, Device-to-Cloud, in cui i dispositivi inviano dati a piattaforme cloud per l'elaborazione, Device-to-Gateway, che prevede l'uso di un gateway per mediare la comunicazione, e Back-End Data-Sharing, che consente la condivisione di dati tra più sistemi. Questi modelli evidenziano la flessibilità con cui i dispositivi IoT possono connettersi e fornire valore agli utenti.

L'adozione del 5G avrà un impatto significativo sul nostro modo di interagire con la tecnologia, abilitando connessioni più rapide, stabili e reattive. Una delle trasformazioni più rilevanti è rappresentata dall'Internet of Things (IoT) [7], che consente ai dispositivi di connettersi e comunicare tra loro attraverso la rete 5G. Questo permetterà la creazione di un ecosistema interconnesso, come mostrato nella Figura 2.2, in cui un'ampia varietà di applicazioni—dai dispositivi domestici alle infrastrutture urbane—potrà sfruttare le potenzialità del 5G per migliorare efficienza, automazione e qualità della vita. Oltre agli smartphone, esistono numerosi altri dispositivi, come sensori e contatori [7], che operano all'interno dell'ecosistema dell'Internet of Things (IoT) e richiedono connessioni con bassa velocità di trasmissione dati e latenza moderata.

Tutti i dispositivi possono comunicare con l'infrastruttura di rete e trasmettere i dati per ulteriori elaborazioni. Questa comunicazione può avvenire sia direttamente tra i dispositivi, con o senza l'uso di gateway, sia attraverso l'infrastruttura di rete. L'obiettivo delle reti è trasferire tutti i dati raccolti dai dispositivi IoT verso le applicazioni, che li elaboreranno per offrire servizi e funzionalità avanzate. Molte applicazioni IoT richiederanno una comunicazione bidirezionale, ossia sia un uplink, per inviare dati alla rete, sia un downlink, per ricevere comandi e aggiornamenti, consentendo così il monitoraggio e il controllo remoto dei dispositivi.

In questo contesto, entra in gioco la tecnologia LoRa (Long Range), che è progettata per dispositivi a basso consumo energetico. LoRa è una componente essenziale delle reti IoT, poiché permette la comunicazione tra dispositivi su lunghe distanze consumando pochissima energia. Questa tecnologia è particolarmente adatta per applicazioni che richiedono trasmissioni di piccoli pacchetti di dati a intervalli regolari, come i sensori ambientali, i contatori intelligenti e i dispositivi per la smart city.

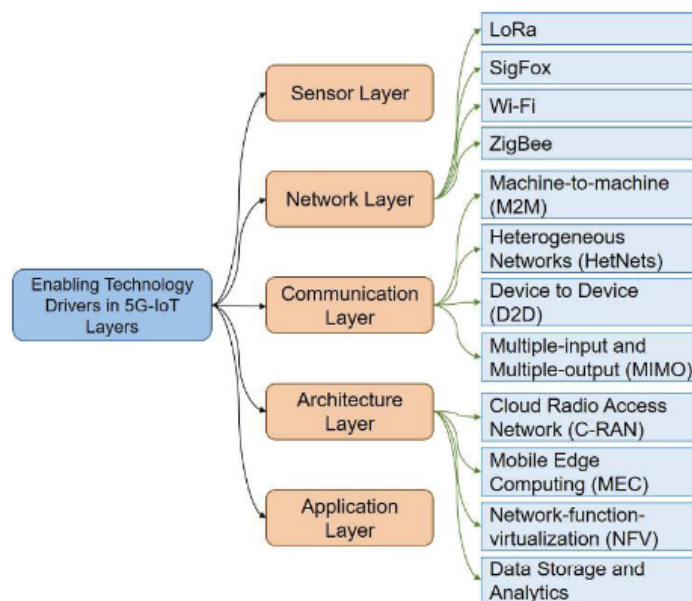


Figura 2.3 Architettura a strati del 5G IoT: livelli funzionali e tecnologie abilitanti [6]

L'Internet of Things (IoT) svolge un ruolo centrale in questa trasformazione, consentendo l'interconnessione di oggetti fisici attraverso le tecnologie dell'informazione e comunicazione (ICT). L'ecosistema IoT, alimentato dalla rete 5G, crea un ambiente in cui dispositivi intelligenti possono comunicare, scambiarsi dati e interagire automaticamente, migliorando l'efficienza dei processi in settori come la sanità, l'industria, l'agricoltura e la logistica [7].

I dispositivi IoT (Internet of Things) possono essere connessi alla rete utilizzando le tecnologie LPWAN (Low Power Wide Area Networks), come ZigBee, LoRa, SigFox, Wi-Fi e NB-IoT, progettate per la comunicazione su lunghe distanze con un basso consumo energetico.

2.4 Architettura del 5G-IoT

L'architettura del 5G IoT è suddivisa in cinque livelli principali, come mostrato nella Figura 2.3 [7]:

- **IoT Sensor Layer** (Livello dei Sensori): Questo livello è composto dai sensori presenti nei dispositivi IoT, responsabili della raccolta dei dati e della loro trasmissione al livello successivo. I sensori possono operare in modalità di comunicazione bidirezionale, consentendo il dialogo diretto tra i dispositivi e la rete, realizzando così una comunicazione MTC (Machine-to-Machine).
- **Network Layer** (Livello di Rete): Questo livello comprende le tecnologie LPWAN, come ZigBee, LoRa, SigFox, Wi-Fi e NB-IoT, che offrono comunicazioni a lungo raggio con basso consumo energetico. In particolare:
 - LoRa (Long Range) permette la trasmissione sicura di dati su lunghe distanze, mantenendo un basso consumo energetico. In Europa, LoRa opera nella banda di frequenza compresa tra 915 MHz e 928 MHz.
- **Communication Layer** (Livello di Comunicazione): Questo livello rappresenta la spina dorsale della comunicazione 5G, utilizzando la tecnologia RAT (Radio Access Technology) per trasmettere i dati delle applicazioni IoT. Il 5G impiega antenne MIMO (Multiple Input Multiple Output), che aumentano la capacità di trasmissione gestendo più flussi di dati contemporaneamente e ridimensionano la latenza, grazie a una trasmissione più rapida e a un minor consumo energetico.
- **Architecture Layer** (Livello Architetturale): Questo livello è costituito da infrastrutture per il cloud computing e l'analisi dei dati (Data Analysis). La tecnologia cloud consente agli utenti di accedere a servizi scalabili con una gestione semplificata e una maggiore efficienza operativa.
- **Application Layer** (Livello Applicativo): Qui si trovano le applicazioni finali che utilizzano i dati raccolti, come:
 - Smart Cities: Sistemi intelligenti per la gestione del traffico, dell'illuminazione e della sicurezza.
 - Smart Buildings: Edifici intelligenti dotati di sistemi automatizzati per il risparmio energetico.
 - Smart Factories: Fabbriche connesse per il monitoraggio e l'automazione dei processi produttivi.

2.5 Sfide dell'implementazione del 5G e IoT

Tuttavia, l'implementazione della tecnologia IoT presenta diverse sfide [7], tra cui: Costo dei dispositivi e Durata della Batteria. Il costo è un fattore determinante per la diffusione su larga scala delle applicazioni IoT, soprattutto per i mercati di massa e le applicazioni ad alto volume. Dispositivi più economici facilitano l'adozione su vasta scala. La durata della batteria è una delle principali sfide per i dispositivi IoT, poiché quasi tutti questi dispositivi funzionano a batteria. L'obiettivo è garantire una durata della batteria di diversi anni, ma questo dipende fortemente dalle modalità d'uso del dispositivo e dall'efficienza energetica della rete.

La copertura è un requisito essenziale per molte applicazioni, soprattutto in settori come il trasporto su auto, treni e altri mezzi mobili. Una connessione stabile e continua è fondamentale per il monitoraggio in tempo reale e per i servizi avanzati.

La connettività interna agli edifici è cruciale per molte applicazioni, soprattutto nel settore dei trasporti e della logistica, dove i dispositivi devono poter comunicare efficacemente anche in ambienti chiusi, come stazioni, porti e magazzini.

L'ecosistema IoT comprende una vasta gamma di dispositivi con requisiti e funzionalità diverse, rendendo complessa l'integrazione e la gestione di tutti questi elementi in un'unica rete. La capacità della rete di supportare un numero sempre maggiore di dispositivi è essenziale per il successo delle implementazioni su larga scala. Con la crescita esponenziale dell'IoT, le reti devono essere progettate per gestire un'enorme quantità di connessioni simultanee.

Una delle sfide più critiche nell'implementazione dell'IoT (Internet of Things) è la necessità di garantire la comunicazione tra miliardi, e in futuro anche trilioni, di dispositivi. Questo rappresenta un problema significativo, poiché il protocollo IPv4 non dispone di un numero sufficiente di indirizzi IP per connettere tutti questi nuovi dispositivi alla rete. Il limite principale dell'IPv4 è il numero di indirizzi univoci disponibili, pari a circa 4,3 miliardi, un valore decisamente insufficiente per supportare l'espansione globale dell'IoT, che prevede la connessione di una quantità enorme di sensori, dispositivi intelligenti e macchine interconnesse.

Per risolvere questo problema, entra in gioco il protocollo IPv6, progettato per offrire un numero praticamente illimitato di indirizzi IP. L'IPv6 espande la lunghezza degli indirizzi da 32 bit (come in IPv4) a 128 bit, consentendo di assegnare un indirizzo univoco a un numero enorme di dispositivi. Grazie a questa espansione, l'IPv6 è in grado di supportare 340 trilioni di trilioni di indirizzi, rendendo possibile l'interconnessione su vasta scala richiesta dall'ecosistema IoT. Di conseguenza, è evidente che non ci sarà un futuro per l'IoT senza IPv6, poiché solo questo protocollo può soddisfare l'immensa domanda di indirizzi IP generata dalla connessione di miliardi di dispositivi.

Attualmente, la transizione verso l'IPv6 è facilitata dal fatto che la maggior parte dei sistemi operativi moderni è in grado di utilizzare indirizzi IPv6, spesso in modalità di coesistenza (dual-stack) con l'IPv4, consentendo una migrazione graduale e senza interruzioni tra i due protocolli. In sintesi, l'IPv6 non è solo una soluzione tecnica, ma un elemento fondamentale per supportare la crescita dell'IoT, garantendo scalabilità, interoperabilità e la possibilità di creare un mondo sempre più connesso e intelligente.

Un'altra sfida rilevante per l'IoT (Internet of Things) riguarda la capacità di archiviazione necessaria per gestire le connessioni di miliardi di sensori attraverso Internet. L'enorme volume di dati generato dalle applicazioni IoT deve essere spesso archiviato in tempo reale, e questo può facilmente diventare un collo di bottiglia per l'implementazione pratica dei sistemi IoT, ostacolando la scalabilità e l'efficienza operativa.

Queste minacce rendono evidente l'importanza di adottare misure di sicurezza avanzate e strategie di protezione dei dati per garantire che i sistemi IoT siano non solo efficienti, ma anche affidabili, sicuri e resilienti.

3 Tecnologie LPWAN

L'evoluzione delle reti di comunicazione per l'Internet of Things (IoT) ha portato alla necessità di soluzioni di connettività efficienti, scalabili e a basso consumo energetico. Sebbene il 5G rappresenti un progresso significativo per molte applicazioni, non tutte le esigenze dell'IoT possono essere soddisfatte esclusivamente da questa tecnologia. In particolare, le applicazioni che richiedono comunicazione su lunghe distanze, bassi consumi energetici e costi ridotti necessitano di soluzioni alternative.

In questo contesto, le tecnologie LPWAN (Low Power Wide Area Network) emergono come un complemento essenziale al 5G, offrendo connettività affidabile per dispositivi distribuiti su vaste aree geografiche. LPWAN permette la comunicazione tra dispositivi con bassissimo consumo energetico e costi operativi contenuti, rendendola ideale per smart cities, monitoraggio ambientale, agricoltura di precisione e gestione delle infrastrutture industriali.

Tra le principali tecnologie LPWAN, spiccano due grandi categorie: LPWAN cellulari e LPWAN non cellulari. Queste due famiglie di tecnologie coesistono e si completano a vicenda, offrendo soluzioni adatte a scenari e casi d'uso differenti. In particolare, LoRaWAN ha guadagnato popolarità grazie alla sua scalabilità, flessibilità e capacità di integrazione con il 5G, contribuendo alla creazione di infrastrutture di connettività ibride che combinano i vantaggi di entrambe le tecnologie.

Tuttavia, l'integrazione di LPWAN con il 5G presenta sfide e opportunità. Se da un lato il 5G può fungere da backhaul per reti LPWAN, garantendo una gestione centralizzata ed efficiente dei dispositivi, dall'altro la convergenza di queste tecnologie solleva questioni legate alla sicurezza, scalabilità e standardizzazione.

Questo capitolo esplorerà il ruolo delle tecnologie LPWAN nell'ecosistema IoT, le possibilità di integrazione con il 5G, e le principali piattaforme di gestione. In particolare, verranno trattati i seguenti aspetti:

- Caratteristiche principali delle reti LPWAN, con un'analisi delle differenze rispetto alle reti cellulari tradizionali e una classificazione delle soluzioni disponibili tra LPWAN cellulari e non cellulari.
- Funzionamento della tecnologia LoRa (Long Range), con un approfondimento sul livello fisico, il meccanismo di modulazione Chirp Spread Spectrum (CSS) e i parametri chiave come Spreading Factor (SF), Bandwidth (BW) e consumo energetico.
- Protocollo LoRaWAN, che regola la comunicazione tra dispositivi LoRa, gateway e server di rete, includendo un'analisi delle tre classi di dispositivi LoRaWAN (Classe A, Classe B e Classe C) e delle problematiche legate alla gestione della rete e della scalabilità.
- ChirpStack, una piattaforma open-source per la gestione delle reti LoRaWAN, i cui componenti principali (Gateway Bridge, Network Server e Application Server) verranno esaminati, con particolare attenzione al suo ruolo nell'integrazione con 5G e cloud computing.

3.2 Panoramica sulle tecnologie LPWAN

Per garantire una comunicazione efficiente nelle reti IoT, la tecnologia LPWAN (*Low Power Wide Area Network*) si afferma come una soluzione complementare alla rete 5G, offrendo vantaggi quali ampia copertura, basso consumo energetico e costi ridotti. Sebbene il 5G garantisca connessioni ad alta velocità e una grande larghezza di banda, la sua infrastruttura è principalmente progettata per supportare una vasta gamma di scenari aziendali, migliorare la capacità di trasporto della rete e fornire servizi sicuri per reti industriali private. Tuttavia, il 5G da solo potrebbe non essere sufficiente a soddisfare tutte le esigenze dell'IoT massivo, che richiede connettività a basso consumo su vaste aree. In questo

contesto, le tecnologie LPWAN risultano particolarmente adatte grazie alla loro capacità di connettere dispositivi distribuiti su ampie zone geografiche con un consumo energetico minimo.

Tra le soluzioni LPWAN, LoRaWAN si distingue per il suo basso consumo energetico, la capacità di coprire lunghe distanze, l'elevata scalabilità e la semplicità di implementazione. Queste caratteristiche la rendono una tecnologia ideale per integrare la copertura offerta dalla rete 5G, garantendo un accesso stabile e affidabile anche in scenari in cui la connettività tradizionale risulterebbe onerosa o poco efficiente.

Le tecnologie **LPWAN** si suddividono in due principali categorie [12]:

- LPWAN cellulari: comprendono tecnologie come Narrowband IoT (NB-IoT), enhanced Machine Type Communication (eMTC) ed extended coverage Global System for Mobile Communication IoT (EC-GSM-IoT). Queste soluzioni, standardizzate dal 3GPP, operano su bande licenziate e sono progettate per integrarsi con le reti cellulari esistenti.
- LPWAN non cellulari: includono tecnologie come LoRaWAN e Sigfox, che funzionano indipendentemente dalle reti mobili e richiedono un'infrastruttura dedicata per l'accesso alla rete e la gestione del traffico dati.

Nonostante le differenze strutturali, le LPWAN non cellulari si sono imposte sul mercato grazie a una maggiore efficienza energetica, costi ridotti e l'uso di bande di frequenza non licenziate. Tra queste, LoRaWAN è la più diffusa, con previsioni che indicano che, entro il 2026, rappresenterà oltre un quarto di tutte le connessioni LPWAN, secondo ABI Research [12]. Per rendere LoRaWAN più interoperabile con le infrastrutture di rete esistenti, sono stati condotti numerosi studi per valutarne l'integrazione con il 5G. In particolare, quattro possibili modelli di integrazione:

- Attraverso la rete di accesso 3GPP, sfruttando direttamente l'infrastruttura delle reti mobili.
- Tramite una rete di accesso non 3GPP non affidabile, mantenendo un'infrastruttura separata ma con connessioni al core network cellulare.
- Come parte dell'eNodeB, integrando il supporto a LoRaWAN direttamente nelle stazioni base mobili.
- Virtualmente come parte della rete core, implementando la gestione di LoRaWAN all'interno della core network del 5G.

Inoltre, lo studio [9] propone l'implementazione di una stazione base virtuale (*Virtual Base Station*) per LoRaWAN, incorporando le funzionalità protocollari degli eNodeB direttamente all'interno dei gateway LoRaWAN, semplificando così la gestione della rete e migliorando la compatibilità con le infrastrutture cellulari esistenti.

Le reti cellulari e LoRaWAN coesisteranno sotto forma di reti ibride nell'era del 5G e oltre. Tuttavia, dotato di una rete core flessibile, agile e potente, il 5G può servire a più di un semplice backhaul per LoRaWAN. L'integrazione LoRaWAN-5G è destinata ad evolversi dalle reti di accesso alle reti core, come le LPWAN cellulari, e alla convergenza fisso-mobile, consentendo una comunicazione efficiente con il core network 5G unificato. Ci sono tre motivazioni principali dietro l'integrazione di LoRaWAN-5G con un core network convergente: 1) Dal punto di vista degli MNO, massimizzare il ritorno sugli investimenti nell'infrastruttura, andando oltre il ruolo di semplici fornitori di backhaul nel vasto mercato delle LPWAN non cellulari. Invece, essi potranno evolvere le proprie reti cellulari per espandere il proprio business. Con una rete core convergente, gli MNO avranno la possibilità di prendere in carico l'operatività di LoRaWAN. 2) Dal punto di vista dei clienti, l'integrazione di LoRaWAN e 5G con un core convergente è economicamente vantaggiosa, poiché evita il doppio dispiegamento e la gestione separata di LoRaWAN e 5G. 3) Da un punto di vista tecnico, un core convergente può migliorare le capacità di LoRaWAN, ad esempio in termini di sicurezza, capacità e possibilità di roaming [12]

3.3 Tecnologia LoRa

LoRa (Long Range) è una tecnologia di modulazione a radiofrequenza proprietaria sviluppata dalla società Semtech [10], concepita per le reti LPWAN (Low Power Wide Area Network). Grazie alla sua capacità di garantire comunicazioni a lungo raggio con un consumo energetico estremamente ridotto, LoRa si è affermata come una delle soluzioni più adatte per le applicazioni nell'ambito dell'Internet of Things (IoT). La sua efficienza e versatilità la rendono ideale per l'interconnessione di dispositivi su vasta scala, con particolare applicazione nei settori della logistica, dell'industria 4.0, delle smart cities e dell'agricoltura intelligente.

La tecnologia LoRa opera sulle bande ISM (Industrial, Scientific and Medical) [12] non licenziate, con una larghezza di banda tipica di 125 kHz, contribuendo così a ridurre i costi operativi e a facilitare l'adozione su ampia scala senza la necessità di licenze specifiche. Il termine LoRa si riferisce comunemente a due livelli distinti della comunicazione: il livello fisico (Physical Layer), che utilizza la modulazione Chirp Spread Spectrum (CSS) per la trasmissione del segnale radio, e il livello MAC (MAC Layer), definito dal protocollo LoRaWAN, che stabilisce l'architettura della rete e i meccanismi di gestione dell'accesso per i dispositivi.

La tecnologia LoRa si basa sulla modulazione **Chirp Spread Spectrum (CSS)** [15], una tecnica di trasmissione che garantisce elevata copertura, robustezza alle interferenze e bassi consumi energetici. Il CSS è stato storicamente utilizzato nei settori militare e aerospaziale, poiché permette la trasmissione di dati su lunghe distanze mantenendo un'elevata affidabilità del segnale anche in condizioni di forte rumore elettromagnetico. La tecnologia LoRa rappresenta la prima implementazione commerciale di CSS a basso costo, consentendone la diffusione su larga scala e l'applicazione in ambito IoT.

La trasmissione mediante CSS avviene attraverso segnali chirp, ovvero segnali che variano la loro frequenza in modo continuo all'interno di un intervallo predefinito. Questa modulazione consente di espandere il segnale su una banda di frequenza più ampia, riducendo il rischio di interferenze e migliorando il rapporto segnale-rumore. Inoltre, i dati vengono codificati direttamente nei chirp, rendendoli facilmente distinguibili anche in presenza di elevato rumore di fondo. Rispetto ad altre modulazioni, come il Frequency Shift Keying (FSK) [14], il CSS garantisce una maggiore resistenza alle interferenze e una copertura più estesa senza comprometterne l'efficienza energetica.

La comunicazione tra i dispositivi LoRa e i gateway avviene su canali di frequenza multipli, con parametri di trasmissione configurabili che influenzano direttamente la qualità del collegamento. Uno degli elementi chiave di questa tecnologia è lo Spreading Factor (SF), che determina il tempo necessario per trasmettere un messaggio e la portata del segnale. LoRa supporta sei valori distinti di SF, compresi tra 7 e 12: un valore più elevato consente una maggiore distanza di trasmissione, ma riduce la velocità di trasmissione dei dati, mentre un valore inferiore permette una comunicazione più rapida, ma su distanze più brevi.

Un altro parametro fondamentale è la larghezza di banda (Bandwidth, BW) [8], che può essere configurata su 125 kHz, 250 kHz o 500 kHz. Una BW più stretta incrementa la sensibilità del ricevitore, migliorando la copertura a lungo raggio, ma riduce la velocità di trasmissione. Generalmente, per la banda ISM 868 MHz, viene adottata una larghezza di banda di 125 kHz, in quanto rappresenta il compromesso ideale tra portata del segnale ed efficienza della comunicazione.

Questi parametri possono essere adattati dinamicamente in base alle condizioni operative e alle esigenze della rete, ottimizzando il bilanciamento tra affidabilità della trasmissione, consumo energetico e capacità di gestire un elevato numero di dispositivi connessi simultaneamente.

L'adozione della tecnologia LoRa, rispetto ad altre soluzioni di comunicazione LPWAN, offre numerosi vantaggi in termini di copertura, efficienza energetica e resistenza alle interferenze. La sua

ampia portata consente la copertura di vaste aree con un numero limitato di gateway, rendendo la tecnologia particolarmente adatta a scenari con dispositivi distribuiti in modo capillare. La resistenza alle interferenze, derivante dall'utilizzo della modulazione CSS, garantisce comunicazioni stabili anche in ambienti ad alta densità di segnali radio, mentre il basso consumo energetico consente ai dispositivi di funzionare per diversi anni con una singola batteria.

La flessibilità operativa della tecnologia LoRa si manifesta anche nella possibilità di configurare dinamicamente parametri come lo Spreading Factor, la larghezza di banda e la potenza di trasmissione, permettendo di ottimizzare la rete in base alle specifiche esigenze applicative. Questa caratteristica la rende idonea per un'ampia gamma di scenari d'uso, tra cui il monitoraggio ambientale, la gestione delle risorse idriche, il controllo dell'illuminazione pubblica nelle smart cities, il tracciamento di merci nella logistica e la gestione intelligente delle coltivazioni nell'agricoltura di precisione.

Grazie alla combinazione tra efficienza energetica, ampia copertura e costi di implementazione ridotti, LoRa si è affermata come una delle tecnologie LPWAN più diffuse, contribuendo in modo significativo allo sviluppo di reti IoT su larga scala e alla realizzazione di infrastrutture di connettività resilienti e scalabili.

3.2.1 Architettura LoRa

Il sistema di comunicazione distribuita basato su LoRa è progettato per garantire basso consumo energetico e trasmissioni a lungo raggio, risultando particolarmente adatto alle reti LPWAN (Low Power Wide Area Network). Questo sistema è strutturato in quattro livelli principali [16], ciascuno con funzioni specifiche per gestire la raccolta, l'elaborazione e l'utilizzo dei dati trasmessi dai dispositivi IoT.

Il livello di accesso (Access Layer) rappresenta la base dell'architettura e comprende dispositivi con sensori, moduli di raccolta e trasmissione dati. È responsabile dell'acquisizione delle informazioni dall'ambiente e della loro trasmissione alla rete. I nodi sensore, posizionati nei punti di monitoraggio, raccolgono dati quali temperatura, umidità, pressione e altre grandezze fisiche. I dati acquisiti vengono quindi inviati a un modulo di trasmissione, che li inoltra ai gateway attraverso la tecnologia LoRa. Il modulo di trasmissione si attiva solo quando necessario, contribuendo a ridurre il consumo energetico complessivo del sistema.

Il livello di convergenza (Convergence Layer) ha il compito di aggregare e inoltrare i dati ricevuti dai dispositivi finali. È costituito da gateway LoRa, stazioni base e switch di rete. I gateway ricevono i dati dai dispositivi e li trasferiscono ai livelli superiori per un'ulteriore elaborazione. La comunicazione tra i gateway e i nodi finali avviene generalmente tramite una topologia a stella, in cui un singolo gateway può gestire più dispositivi IoT in un'area estesa. Oltre alla raccolta dei dati, il livello di convergenza svolge una funzione di ottimizzazione e regolazione dinamica della trasmissione, coordinando i dispositivi per garantire un utilizzo efficiente dello spettro radio.

Il livello centrale (Core Layer) fornisce il supporto alla rete e si occupa dell'elaborazione dei dati raccolti. Questo livello è generalmente basato su un'architettura di cloud computing, che permette di elaborare grandi volumi di dati con efficienza e scalabilità. L'adozione di tecnologie cloud offre numerosi vantaggi, tra cui alta densità di elaborazione, ottimizzazione delle risorse, gestione centralizzata e riduzione del consumo energetico. Inoltre, il cloud computing garantisce la scalabilità della rete, consentendo l'integrazione di nuovi dispositivi senza compromettere le prestazioni del sistema. La gestione centralizzata dei dati consente inoltre di implementare funzionalità avanzate di analisi e intelligenza artificiale per migliorare le capacità predittive e di automazione delle applicazioni IoT.

Il **livello applicativo (Application Layer)** costituisce l'interfaccia tra il sistema e l'utente finale, fornendo servizi on-demand e strumenti di gestione dei dati. Questo livello è composto da client software, applicazioni mobili e piattaforme web che consentono agli utenti di monitorare i dati raccolti, configurare i dispositivi e ricevere notifiche in tempo reale. Le informazioni provenienti dal livello centrale vengono presentate in modo accessibile, attraverso dashboard intuitive e strumenti di analisi. Nell'ambito delle applicazioni industriali, il livello applicativo può includere sistemi di gestione per reti di distribuzione dell'energia, monitoraggio ambientale, gestione delle infrastrutture urbane e controllo della logistica.

All'interno di questa architettura, ogni livello svolge un ruolo chiave nel garantire efficienza, scalabilità e affidabilità nella comunicazione tra dispositivi IoT. L'integrazione tra i livelli consente di ottimizzare il flusso dei dati, riducendo il consumo energetico e migliorando le prestazioni della rete, rendendo LoRa una tecnologia strategica per lo sviluppo di infrastrutture IoT di nuova generazione.

3.3 Protocollo LoRaWAN

LoRaWAN è un protocollo di rete aperto sviluppato dalla LoRa Alliance, un'organizzazione no-profit dedicata alla standardizzazione e all'evoluzione dell'ecosistema LoRaWAN. Questo protocollo è stato progettato per estendere le capacità della tecnologia LoRa, fornendo un'infrastruttura scalabile e interoperabile in grado di supportare la connettività IoT su larga scala.

La LoRa Alliance [10] distingue chiaramente LoRa come livello fisico (Physical Layer), responsabile della modulazione del segnale radio, e LoRaWAN come livello MAC (MAC Layer), che definisce l'architettura di rete e le regole di accesso per i dispositivi connessi, come mostrato anche nella Figura 3.2. LoRaWAN è stato sviluppato per garantire una gestione efficiente della rete, consentendo la connessione simultanea di migliaia di dispositivi su ampie aree geografiche, con un numero limitato di infrastrutture di rete.

Le reti di comunicazione IoT possono adottare diverse topologie, tra cui quella a maglia (mesh) e quella a stella (star topology). Molte reti tradizionali utilizzano un'architettura mesh, in cui i nodi finali inoltrano i dati provenienti da altri nodi, ampliando la portata della rete. Tuttavia, questa configurazione introduce una serie di complessità, tra cui la riduzione della capacità complessiva e un aumento del consumo energetico, poiché ogni nodo deve ricevere e ritrasmettere informazioni che non sempre sono direttamente pertinenti alla propria operatività.

3.3.1 Architettura di rete LoRaWAN

Come rappresentato nella figura 3.1, l'architettura LoRaWAN è composta da tre componenti principali:

- **Dispositivi finali (End-Devices, ED):** Si tratta di sensori e attuatori a basso consumo energetico che raccolgono e trasmettono dati utilizzando la modulazione LoRa [8].
- **Gateway (GW):** Dispositivi intermedi che ricevono i pacchetti inviati dagli end-devices e li inoltrano al Network Server. Poiché un messaggio LoRa può essere ricevuto da più gateway, la ridondanza nella rete è un elemento chiave per garantire l'affidabilità della trasmissione [8].

- **Network Server (NS):** Il componente centrale della rete, responsabile della gestione della comunicazione tra i dispositivi finali e le applicazioni. Il Network Server si occupa della deduplicazione dei pacchetti, della gestione della sicurezza, dell'allocazione delle risorse radio e della trasmissione dei dati all'application server [8].

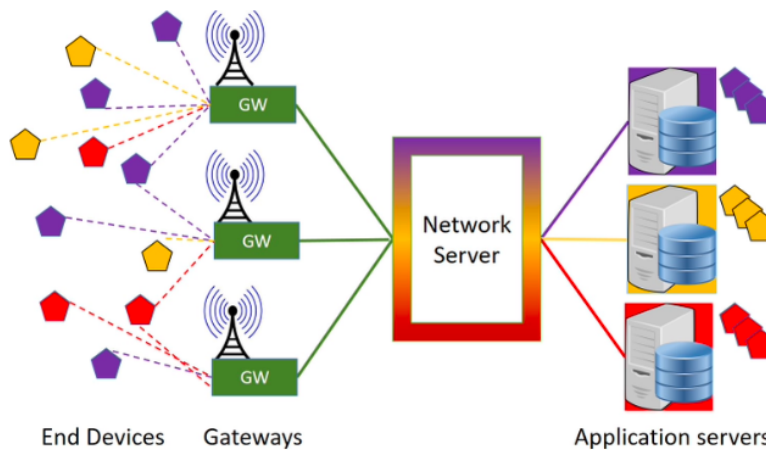


Figura 3.1 Architettura di rete LoRaWAN: interazione tra dispositivi finali, gateway, network server e application server [8]

L'architettura di rete LoRaWAN si basa su una topologia a stella (star topology) [10], composta da server LoRaWAN centralizzati e gateway multipli, in cui i dispositivi IoT comunicano direttamente con i gateway, che a loro volta inoltrano i dati al Network Server. Questo approccio permette di centralizzare la gestione della rete, semplificando l'elaborazione e l'instradamento dei dati. Il flusso operativo del protocollo segue tre fasi principali: i dispositivi IoT trasmettono i dati ai gateway, i gateway raccolgono le informazioni e le inoltrano al network server, il quale filtra e processa i dati prima di inviarli all'application server per l'elaborazione finale.

A differenza delle tradizionali reti cellulari, i dispositivi finali (end-devices) non sono associati a un gateway specifico per accedere alla rete. Infatti, i gateway agiscono semplicemente come ripetitori del livello di collegamento (link layer relay), inoltrando i pacchetti ricevuti dai dispositivi finali al Network Server (NS) tramite una connessione Ethernet o 3G, dopo aver aggiunto informazioni sulla qualità del canale (come RSSI e SNR)[8]. Va notato che, almeno logicamente, i gateway risultano trasparenti per i dispositivi finali.

LoRaWAN è una tecnologia wireless progettata per garantire un'ampia copertura geografica con un'infrastruttura di rete relativamente limitata, permettendo la connessione di un numero elevato di dispositivi IoT. Tuttavia, la scalabilità e l'affidabilità della rete dipendono da diversi fattori, tra cui la densità dei dispositivi connessi, le condizioni ambientali e la configurazione della rete stessa. Un'elevata concentrazione di dispositivi nella stessa area può aumentare significativamente le interferenze, compromettendo la qualità della comunicazione. Inoltre, ostacoli fisici come edifici e rilievi naturali possono attenuare il segnale, riducendone la portata.

Per garantire prestazioni ottimali e una buona scalabilità, è fondamentale una pianificazione attenta della rete, con una distribuzione strategica dei gateway in modo da massimizzare la copertura e minimizzare il rischio di collisioni. Nei gateway LoRaWAN, la ricezione simultanea di pacchetti da molti dispositivi sullo stesso canale può causare collisioni e perdita di dati, con un impatto negativo sulle prestazioni complessive. Il problema si aggrava quando più dispositivi utilizzano lo stesso Spreading Factor (SF) e trasmettono sulla stessa frequenza, rendendo difficile la corretta decodifica dei segnali e riducendo l'affidabilità della rete.

Uno dei principali fattori che influisce negativamente sulle prestazioni di LoRaWAN è la perdita di pacchetti a livello Media Access Control (MAC), dovuta soprattutto alle collisioni tra trasmissioni simultanee. Ciò è strettamente legato all'uso del protocollo di accesso ALOHA [10], che consente ai dispositivi di trasmettere senza coordinazione. Sebbene questo approccio semplifichi l'implementazione della rete, aumenta il rischio di collisioni, specialmente in ambienti ad alta densità di dispositivi. Inoltre, l'assenza di un meccanismo centralizzato di controllo non tiene conto di fenomeni come il fading del canale, il controllo dinamico della potenza di trasmissione e la gestione delle interferenze, peggiorando ulteriormente l'affidabilità del sistema.

Per migliorare le prestazioni della rete [8], è essenziale introdurre strategie di ottimizzazione a livello MAC che riducano il numero di collisioni e migliorino l'efficienza della trasmissione dati. Una soluzione efficace è l'allocazione dinamica dello Spreading Factor (SF), che distribuisce i dispositivi su diversi livelli di modulazione, riducendo così il rischio di interferenze.

Un altro elemento chiave per l'ottimizzazione della rete è il Adaptive Data Rate (ADR), un meccanismo che regola automaticamente la velocità di trasmissione e la potenza del segnale in base alle condizioni di rete e alla qualità del collegamento. Ad esempio, un SF elevato consente di aumentare la portata del segnale e comunicare su distanze maggiori, ma riduce il data rate e incrementa il Time-on-Air (ToA). Un tempo di trasmissione più lungo può aumentare la probabilità di collisioni tra pacchetti trasmessi simultaneamente. Il ruolo dell'ADR è quindi cruciale nell'adattare i parametri di trasmissione alle esigenze di ogni dispositivo, contribuendo anche a prolungarne la durata della batteria.

Grazie alla gestione dinamica delle risorse e all'uso dell'ADR, LoRaWAN è in grado di offrire una rete scalabile e adattabile a scenari con differenti livelli di densità di dispositivi. Inoltre, l'espansione della rete può essere gestita con l'aggiunta di nuovi gateway, aumentando la capacità complessiva e riducendo il carico su quelli esistenti. Studi dimostrano che una distribuzione ottimale dei gateway può incrementare la capacità della rete fino a 6-8 volte [14], mantenendo un elevato livello di Quality of Service (QoS).

Un altro aspetto essenziale nella gestione delle reti LoRaWAN è il consumo energetico. Poiché LoRaWAN è pensato per applicazioni IoT a basso consumo, i dispositivi trasmettono in modalità asincrona, attivandosi solo quando devono inviare dati. Questo approccio massimizza l'autonomia operativa, riducendo il numero di trasmissioni e ottimizzando l'uso dell'energia. A differenza delle reti mesh o cellulari, che richiedono una sincronizzazione periodica tra i nodi, LoRaWAN non necessita di una sincronizzazione continua, evitando sprechi di energia. Tuttavia, questa caratteristica introduce latenza nelle comunicazioni, rendendo la tecnologia meno adatta per applicazioni che richiedono aggiornamenti in tempo reale.

Dal punto di vista della capacità di rete, la topologia a stella di LoRaWAN impone requisiti specifici ai gateway, che devono gestire un elevato numero di dispositivi connessi. Se un gateway ha una capacità limitata, può subire un sovraccarico di traffico, con conseguente aumento della latenza e riduzione delle prestazioni. Per mitigare questo problema, i gateway LoRaWAN utilizzano transceiver multicanale e multi-modem, che permettono la ricezione simultanea di pacchetti su più canali di frequenza. L'ADR gioca un ruolo fondamentale anche in questo contesto, ottimizzando la velocità di trasmissione e adattando i parametri radio alle condizioni della rete.

La capacità di una rete LoRaWAN dipende da diversi fattori, tra cui il numero di canali disponibili, il data rate, il Time-on-Air (ToA), la lunghezza del payload e la frequenza delle trasmissioni dei dispositivi. Ottimizzando questi parametri, è possibile migliorare la scalabilità della rete e garantire un uso più efficiente delle risorse disponibili.

Grazie a una gestione attenta delle risorse radio e all'uso di tecniche avanzate di allocazione dello spettro, LoRaWAN è in grado di supportare un numero elevato di dispositivi con un'infrastruttura relativamente ridotta. L'impiego di gateway ad alte prestazioni, combinato con meccanismi di adattamento dinamico delle trasmissioni, permette di ridurre le interferenze e migliorare la qualità del servizio, garantendo al tempo stesso un consumo energetico contenuto. Per questo motivo, LoRaWAN rappresenta una delle soluzioni più avanzate e affidabili per la realizzazione di reti IoT su vasta scala, offrendo un equilibrio ottimale tra efficienza energetica, copertura estesa e scalabilità.

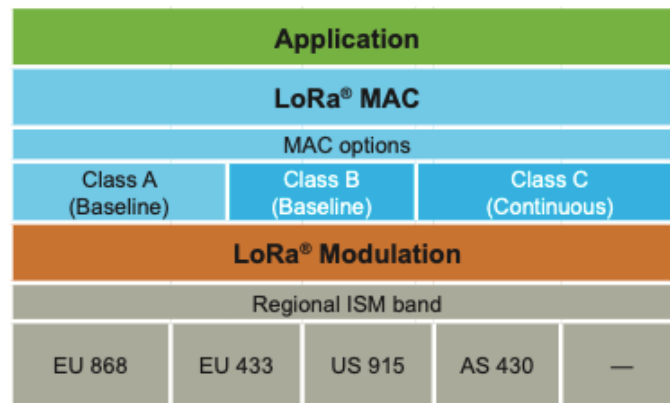


Figura 3.2 Struttura dello stack di protocollo LoRaWAN [14]

3.3.2 Classi di dispositivi LoRaWAN

Per adattarsi alle esigenze di diverse applicazioni IoT, LoRaWAN supporta tre classi di dispositivi finali: **Classe A**, **Classe B** e **Classe C**, mostrati nella Figura 3.2. Ogni classe offre un diverso equilibrio tra consumo energetico e disponibilità per la ricezione dei messaggi in **downlink**, influenzando la reattività e l'efficienza della rete [14]

Classe A – Dispositivi bidirezionali a basso consumo: I dispositivi di Classe A rappresentano l'opzione più efficiente dal punto di vista energetico. Essi trasmettono dati in uplink in base alle proprie necessità, con una leggera variazione casuale (protocollo ALOHA). Dopo ogni trasmissione, il dispositivo apre due brevi finestre di ricezione per eventuali messaggi in downlink. Se il Network Server (NS) deve inviare un messaggio in un momento diverso, dovrà attendere la successiva trasmissione uplink del dispositivo. Questa configurazione è ideale per applicazioni IoT a basso consumo energetico, dove la comunicazione in downlink è necessaria solo immediatamente dopo un uplink. Tuttavia, la latenza per la ricezione dei messaggi è elevata, poiché un dispositivo non può ricevere dati se non ha prima trasmesso.

Classe B – Dispositivi bidirezionali con finestre di ricezione programmate: I dispositivi di Classe B migliorano la reattività ai messaggi in downlink rispetto alla Classe A, grazie all'apertura di finestre di ricezione aggiuntive in orari programmati. Questa sincronizzazione avviene tramite un beacon inviato dal gateway, che permette al Network Server di sapere esattamente quando il dispositivo è in ascolto. Grazie a questa caratteristica, i dispositivi di Classe B riducono il tempo di attesa per ricevere messaggi dal server, rendendoli adatti per applicazioni che richiedono una comunicazione più tempestiva, pur mantenendo un consumo energetico relativamente contenuto rispetto alla Classe C.

Classe C – Dispositivi bidirezionali con finestre di ricezione sempre aperte: I dispositivi di Classe C massimizzano la disponibilità per il downlink, mantenendo la finestra di ricezione quasi sempre aperta, chiudendola solo durante la trasmissione di un messaggio in uplink. Questo consente una comunicazione immediata e a bassa latenza, rendendo la Classe C ideale per applicazioni che richiedono risposte in tempo reale, come il controllo remoto e l'automazione industriale. Tuttavia, il principale svantaggio è l'elevato consumo energetico, poiché il dispositivo deve rimanere costantemente in

ascolto. Per questo motivo, la Classe C è consigliata principalmente per dispositivi alimentati da rete elettrica o con requisiti di latenza stringenti.

3.4 ChirpStack: piattaforma di gestione per LoRaWAN

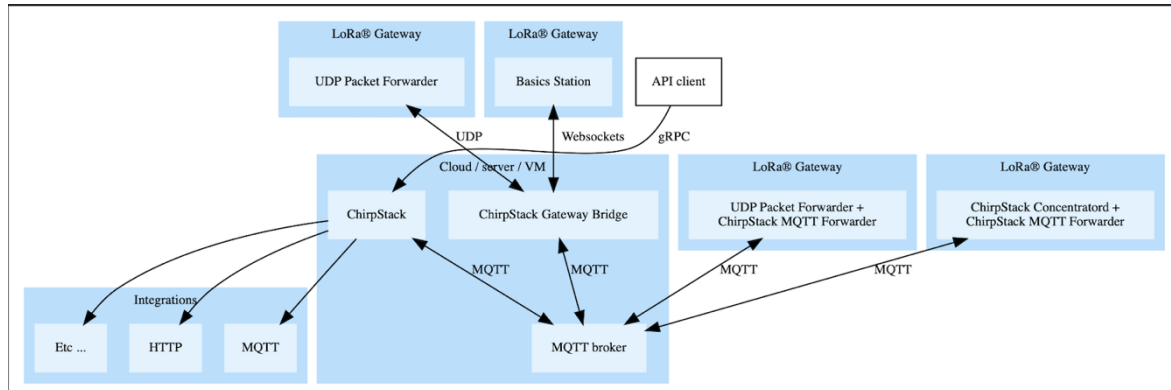


Figura 3.3 Architettura ChirpStack per la gestione delle reti LoRaWAN [18]

ChirpStack [14] è una piattaforma open-source progettata per la gestione di reti LoRaWAN, come mostrato nella Figura 3.3, fornendo tutti i componenti necessari per implementare un'infrastruttura scalabile e flessibile per dispositivi IoT. Grazie alla sua modularità, ChirpStack permette l'elaborazione e l'instradamento dei dati tra dispositivi LoRaWAN, gateway e applicazioni, consentendo una gestione efficiente della rete e un'integrazione con altri sistemi informatici.

L'architettura di ChirpStack è composta da diversi moduli interconnessi che svolgono specifiche funzioni. I dispositivi LoRaWAN trasmettono i dati ai gateway, che inoltrano i pacchetti ricevuti al ChirpStack Gateway Bridge. Quest'ultimo funge da intermediario tra i gateway e il resto della rete, traducendo i dati in un formato compatibile con il ChirpStack Network Server. Quest'ultimo è responsabile dell'elaborazione dei pacchetti, gestendo il tasso di dati adattivo (Adaptive Data Rate, ADR), la deduplicazione dei messaggi ricevuti da più gateway e l'autenticazione dei dispositivi. Una volta elaborati, i pacchetti vengono inoltrati al ChirpStack Application Server, che si occupa della decodifica dei dati e della loro distribuzione verso applicazioni esterne tramite API REST.

La piattaforma ChirpStack presenta numerose funzionalità avanzate che la rendono una soluzione ideale per la gestione di reti LoRaWAN su larga scala. Tra queste vi è il supporto al roaming, che consente ai dispositivi di connettersi a più gateway anche appartenenti a reti diverse, garantendo una maggiore continuità del servizio. La gestione delle chiavi di sicurezza è un ulteriore aspetto fondamentale, in quanto ChirpStack implementa protocolli di crittografia end-to-end per proteggere le comunicazioni tra i dispositivi e i server di rete. Inoltre, la piattaforma supporta diversi backend di archiviazione, tra cui PostgreSQL e Redis [21], permettendo una gestione efficiente dei dati raccolti. Un altro elemento distintivo di ChirpStack è la sua capacità di integrarsi con servizi cloud come AWS IoT Core, Google Cloud IoT e Microsoft Azure IoT Hub, ampliando le possibilità di analisi e gestione delle informazioni trasmesse dai dispositivi IoT.

L'installazione e la configurazione di ChirpStack possono essere effettuate su server locali o in ambienti cloud, a seconda delle esigenze operative. Il processo di implementazione prevede l'installazione dei principali componenti software, tra cui il ChirpStack Network Server e il ChirpStack Application Server, seguita dalla configurazione del Gateway Bridge per permettere la connessione tra i gateway LoRaWAN e il sistema. Una volta completata questa fase, i servizi di ChirpStack possono essere avviati e monitorati attraverso l'interfaccia web dedicata, che consente di visualizzare in tempo reale lo stato della rete e dei dispositivi connessi.

Dal punto di vista della compatibilità hardware, ChirpStack supporta una vasta gamma di gateway LoRaWAN, tra cui dispositivi prodotti da Kerlink, RAK Wireless, The Things Industries e Multitech [20]. Inoltre, la piattaforma utilizza protocolli di comunicazione standard come MQTT per la gestione dei dati, gRPC per la comunicazione tra i vari componenti e API REST per l'integrazione con applicazioni di terze parti [17]

Un aspetto rilevante riguarda il confronto tra ChirpStack e altre soluzioni LoRaWAN, come The Things Stack (TTS) e Lorient [19]. A differenza di TTS e Lorient, che offrono modelli commerciali con servizi cloud, ChirpStack rappresenta un'alternativa open-source completamente personalizzabile, permettendo agli utenti di gestire la rete in locale senza costi aggiuntivi. Inoltre, la possibilità di implementare ChirpStack all'interno di un'infrastruttura ibrida LoRaWAN-5G conferisce alla piattaforma un ulteriore vantaggio in termini di scalabilità e interoperabilità.

L'integrazione di ChirpStack con il 5G rappresenta un'opportunità significativa per il settore delle telecomunicazioni e dell'Internet of Things. Grazie all'impiego del 5G Edge Computing, ChirpStack può beneficiare di una riduzione della latenza nella trasmissione dei dati, migliorando le prestazioni complessive della rete. L'integrazione con il core network 5G garantisce inoltre una maggiore sicurezza, facilitando la gestione centralizzata dei dispositivi LoRaWAN in un'unica infrastruttura ibrida. Questa sinergia tra LoRaWAN e 5G è particolarmente rilevante per scenari applicativi come le smart cities, l'agricoltura intelligente e il monitoraggio industriale, dove la combinazione di tecnologie LPWAN e reti mobili di ultima generazione consente di ottimizzare il consumo energetico e migliorare la copertura del segnale.

In conclusione, ChirpStack si configura come una soluzione estremamente versatile per la gestione di reti LoRaWAN, offrendo una piattaforma open-source scalabile, sicura e facilmente integrabile con altri sistemi informatici e di telecomunicazione. L'adozione di ChirpStack in contesti industriali e urbani in combinazione con il 5G rappresenta un'importante evoluzione verso la creazione di reti IoT ibride, capaci di garantire elevate prestazioni e un'efficiente gestione dei dispositivi connessi.

4 Orchestrazione di applicazioni cloud native

L'evoluzione delle architetture software ha portato a una crescente adozione delle tecnologie basate su container e orchestrazione cloud-native, rivoluzionando la gestione delle applicazioni distribuite. In passato, le infrastrutture IT si basavano su server fisici e successivamente su macchine virtuali (VM), ma l'aumento della complessità delle applicazioni e la necessità di scalabilità dinamica hanno evidenziato i limiti di questi approcci. Oggi, con l'avvento delle architetture microservizi e delle reti virtualizzate, l'adozione di strumenti avanzati per l'automazione del deployment, la gestione efficiente delle risorse e la scalabilità on-demand è diventata essenziale.

La containerizzazione ha rivoluzionato il modo in cui le applicazioni vengono sviluppate, distribuite ed eseguite. I container offrono un ambiente isolato e leggero, che include tutto il necessario per eseguire un'applicazione: codice, runtime, librerie e configurazioni. Questo garantisce portabilità, efficienza nell'uso delle risorse e scalabilità, rendendo il deployment delle applicazioni più rapido e coerente su diverse piattaforme.

Docker è il motore di containerizzazione più diffuso, fornendo un framework standardizzato per la creazione, il packaging e l'esecuzione di container. Attraverso Dockerfile, è possibile definire in modo dichiarativo l'ambiente di esecuzione di un'applicazione, rendendo il deployment più prevedibile e riproducibile su qualsiasi infrastruttura compatibile con Docker. Sebbene Docker semplifichi la gestione dei singoli container, non offre strumenti nativi per orchestrare applicazioni distribuite composte da più container eseguiti su diversi nodi. In scenari complessi, dove più container devono interagire tra loro e adattarsi dinamicamente al carico di lavoro, è necessario un sistema di orchestrazione avanzato come Kubernetes (K8s). Kubernetes è una piattaforma open-source per l'orchestrazione di container, progettata per automatizzare il deployment, la gestione e il scaling delle applicazioni containerizzate su cluster distribuiti. Fornisce funzionalità avanzate per il bilanciamento del carico, l'auto-ripristino, la scalabilità dinamica e la gestione delle configurazioni. Grazie a queste funzionalità, Kubernetes è diventato la piattaforma di riferimento per la gestione di infrastrutture cloud-native e ambienti containerizzati complessi, come quelli utilizzati nelle reti 5G e nelle architetture O-RAN.

Nel contesto delle reti mobili di nuova generazione, l'Open Radio Access Network (O-RAN) introduce un'architettura aperta e disaggregata, in cui i componenti software vengono eseguiti su piattaforme cloud-native. Uno degli elementi chiave di questa architettura è il Near-RT RIC (Near-Real-Time RAN Intelligent Controller), un modulo software che gestisce l'allocazione dinamica delle risorse radio attraverso l'esecuzione di applicazioni modulari chiamate xApp.

Kubernetes gioca un ruolo fondamentale nell'orchestrazione delle xApp all'interno del Near-RT RIC, garantendo un'infrastruttura scalabile e resiliente, capace di adattarsi ai cambiamenti dinamici del traffico di rete; gestione automatizzata delle xApp, controllandone il deployment, il failover e l'aggiornamento senza downtime; isolamento e modularità, assicurando che ogni xApp operi indipendentemente dalle altre, senza interferenze; integrazione con il framework O-RAN, facilitando l'interoperabilità tra le xApp e i moduli di gestione della rete.

Le xApp vengono eseguite come Pod Kubernetes e sono gestite attraverso strumenti di automazione come Helm, che consente un deployment strutturato e riproducibile delle applicazioni. Kubernetes fornisce inoltre un meccanismo di bilanciamento del carico per ottimizzare la distribuzione delle richieste tra le diverse istanze delle xApp e garantire una gestione efficiente del traffico dati.

Grazie a queste capacità, Kubernetes si posiziona come tecnologia abilitante per l'evoluzione delle reti mobili, fornendo la flessibilità e la scalabilità necessarie per gestire l'enorme complessità del 5G e delle future reti 6G.

In questo capitolo, verranno trattati i seguenti aspetti chiave:

- Introduzione alla containerizzazione e ai vantaggi dei container rispetto alle macchine virtuali
- Docker: funzionamento, gestione delle immagini e creazione di container
- Il ruolo di Kubernetes nell'orchestrazione di container e nella gestione di infrastrutture distribuite
- L'architettura di Kubernetes: Pod, ReplicaSet, Service, Deployment, Namespace e Helm
- L'utilizzo di Kubernetes nel Near-RT RIC per la gestione delle xApp e delle risorse di rete
- Il ciclo di vita delle xApp in Kubernetes: onboarding, deployment, aggiornamenti e decommissioning
- L'automazione della gestione delle xApp tramite Helm e AppMgr

L'obiettivo di questo capitolo è fornire una panoramica completa sulle tecnologie di containerizzazione e orchestrazione nel cloud, con un focus specifico sul loro utilizzo nelle reti mobili e nelle architetture O-RAN. L'integrazione tra Kubernetes e le tecnologie cloud-native rappresenta un passo cruciale per realizzare infrastrutture più flessibili, scalabili e automatizzate, in grado di supportare le reti di nuova generazione.

4.1 Introduzione alla containerizzazione

La tecnologia dei container crea un livello astratto che incapsula il livello software al di sopra del sistema operativo necessario al funzionamento del componente applicativo, disaccoppiandolo dall'infrastruttura sottostante. Questo permette il funzionamento su qualsiasi piattaforma che supporta la tecnologia dei container. Un container non è altro che un pacchetto di codice software contenente il codice di un'applicazione, le sue librerie e altre dipendenze che costituiscono l'ambiente di esecuzione dell'applicazione. In questo modo, un container astrae dal sistema operativo e dall'infrastruttura fisica sottostante, consentendo di portare un'applicazione containerizzata senza doversi preoccupare del sistema operativo sottostante, pur mantenendo il controllo su scalabilità e runtime.

Uno dei principali vantaggi dei container è la loro dimensione ridotta. A differenza delle macchine virtuali (VM), che includono un intero sistema operativo guest e occupano diversi gigabyte di spazio, i container condividono il kernel del sistema operativo host e contengono solo le librerie e le dipendenze necessarie per eseguire l'applicazione. Questo riduce notevolmente le dimensioni dei container, portandole all'ordine di grandezza dei megabyte.

L'idea alla base della progettazione dei container è quella di realizzare un ambiente isolato destinato all'esecuzione di una singola applicazione, limitando l'uso delle risorse il più possibile, come evidenziato nella Figura 4.1. In questo modo, a parità di hardware, è possibile distribuire un numero significativamente maggiore di container rispetto alle VM.

Negli ultimi anni, la containerizzazione è diventata una delle tecnologie chiave nello sviluppo e nella distribuzione di applicazioni software, grazie alla sua capacità di garantire portabilità, scalabilità ed efficienza. In questo contesto, **Docker** rappresenta il principale motore di containerizzazione, consentendo la creazione, il packaging e l'esecuzione di applicazioni all'interno di ambienti isolati e riproducibili.

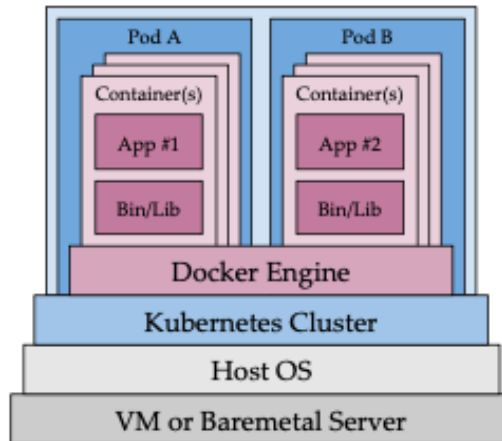


Figura 4.1 Struttura a livelli per l'esecuzione delle xApp su Kubernetes, con pod Docker e Near-RT RIC su VM o server bare metal [2]

4.2 Docker: Il motore della containerizzazione

Docker [22] nasce con l'obiettivo di risolvere le problematiche legate all'incoerenza degli ambienti di sviluppo e produzione. Tradizionalmente, il software veniva eseguito direttamente sul sistema operativo ospitante, rendendo difficile garantire che un'applicazione funzionasse nello stesso modo su macchine diverse, a causa di dipendenze software variabili, configurazioni incompatibili e differenze tra le piattaforme di esecuzione. Docker supera questi problemi fornendo un ambiente standardizzato, in cui le applicazioni vengono eseguite in container isolati, indipendenti dall'infrastruttura sottostante.

Docker si basa sul concetto di container, un'unità leggera ed eseguibile che include tutto il necessario per far funzionare un'applicazione: codice, runtime, librerie di sistema, dipendenze e configurazioni. I container condividono il kernel del sistema operativo, a differenza delle macchine virtuali (VM) che necessitano di un sistema operativo completo per ogni istanza. Questo approccio riduce significativamente il consumo di risorse e migliora le prestazioni.

```

1  # Start by building from a base image
2  FROM python:3.8-alpine
3
4  # Create temporary variable with a path
5  ARG dir=/tmp
6
7  # Run shell command to install dependencies
8  RUN apk update && apk add gcc musl-dev bash
9
10 # Copy files from host machine to the image
11 COPY src/ ${dir}/src
12 COPY init/ ${dir}/init
13 COPY setup.py ${dir}/
14
15 # Install the Python xApp
16 RUN pip3 install ${dir}
17
18 # Set location of xApp configuration file
19 ENV CONFIG_FILE=${dir}/config_file.json
20
21 # Starting the container running our xApp
22 CMD run-xapp

```

Figura 4.2 Dockerfile per il deployment di una xApp [2]

Ogni container viene creato a partire da un'immagine Docker, un pacchetto immutabile che contiene tutte le istruzioni necessarie per avviare e configurare un'applicazione. Le immagini Docker sono costruite a partire da uno o più layer (strati), che rappresentano le diverse parti di un sistema operativo e delle dipendenze applicative. Questa struttura stratificata permette di ottimizzare lo storage e il deployment, riutilizzando i layer comuni tra più immagini.

Le immagini Docker vengono generate utilizzando un **Dockerfile**, un file di testo che descrive in modo dichiarativo tutti i passaggi necessari per creare l'immagine, come illustrato nella Figura 4.2. Ogni riga di un Dockerfile rappresenta un comando che viene eseguito per costruire un nuovo layer.

L'adozione di Docker ha rivoluzionato la gestione delle applicazioni, offrendo diversi vantaggi rispetto ai tradizionali ambienti di sviluppo e deployment:

Portabilità: poiché i container includono tutte le dipendenze necessarie, le applicazioni possono essere eseguite in modo coerente su qualsiasi piattaforma che supporti Docker, indipendentemente dal sistema operativo o dall'infrastruttura sottostante.

Scalabilità: Docker facilita il deployment di applicazioni scalabili grazie all'integrazione con strumenti di orchestrazione come Kubernetes, che automatizzano il bilanciamento del carico e la distribuzione dei container su più nodi.

Efficienza nell'uso delle risorse: a differenza delle macchine virtuali, che richiedono un intero sistema operativo per ogni istanza, i container condividono il kernel del sistema operativo, riducendo il consumo di memoria e CPU.

Rapidità di sviluppo e distribuzione: grazie ai container, gli sviluppatori possono lavorare su ambienti isolati e identici a quelli di produzione, evitando problemi legati a differenze nelle configurazioni locali.

Versionamento e riproducibilità: le immagini Docker possono essere versionate e archiviate in registri remoti (ad esempio, Docker Hub o registri privati), consentendo di riprodurre esattamente la stessa versione di un'applicazione in qualsiasi momento.

Modularità: Docker incoraggia l'adozione di architetture basate su microservizi, in cui ogni componente dell'applicazione può essere eseguito in un container separato, migliorando la manutenibilità e la scalabilità.

Le immagini Docker possono essere archiviate in registri remoti per facilitarne la distribuzione e l'accesso da più macchine. Il registro pubblico più diffuso è Docker Hub, ma esistono anche registri privati come Amazon Elastic Container Registry (ECR), Google Container Registry (GCR) e Harbor.

Sebbene Docker sia un sistema potente per la gestione dei container, la sua capacità di orchestrazione su larga scala è limitata. Per questo motivo, è spesso combinato con strumenti come Kubernetes, che fornisce funzionalità avanzate per la gestione di cluster di container.

Kubernetes utilizza Docker (o altri runtime container) per eseguire e orchestrare container distribuiti su più nodi, garantendo alta disponibilità, bilanciamento del carico e auto-ripristino in caso di errori. In un'infrastruttura Kubernetes, Docker è responsabile dell'esecuzione dei container all'interno dei Pod, mentre Kubernetes gestisce il ciclo di vita e la scalabilità delle applicazioni.

4.3 Kubernetes: Il sistema di orchestrazione cloud-native

Negli ultimi anni, la virtualizzazione e la containerizzazione hanno trasformato radicalmente l'architettura delle reti di telecomunicazione, introducendo un livello di flessibilità, scalabilità ed efficienza senza precedenti. In questo contesto, Kubernetes (K8s) [23] si è affermato come il framework di riferimento per l'orchestrazione dei container, offrendo un'infrastruttura robusta per la gestione automatizzata di applicazioni distribuite su larga scala.

Kubernetes è una piattaforma open-source progettata per orchestrare l'intero **ciclo di vita** delle applicazioni containerizzate, garantendo **affidabilità, alta disponibilità e scalabilità** attraverso un approccio completamente automatizzato. La sua architettura si basa su una configurazione dichiarativa, in cui lo stato desiderato del sistema viene definito dall'utente e mantenuto in modo autonomo dal framework.

Il funzionamento di Kubernetes si basa sulla definizione di **oggetti**, entità persistenti che descrivono lo stato del cluster e ne regolano il comportamento. Questi oggetti consentono di specificare:

- Quali applicazioni devono essere eseguite e su quali nodi del cluster.
- Le risorse assegnate a ciascuna applicazione, come CPU e memoria.
- Le configurazioni e i parametri ambientali passati ai container in esecuzione.
- Le politiche di gestione del ciclo di vita, inclusi i criteri di riavvio, aggiornamento e ripristino in caso di guasto.
- Le regole di networking e bilanciamento del traffico tra le applicazioni.

Uno degli aspetti fondamentali di Kubernetes è la sua capacità di riconciliare continuamente lo stato corrente del sistema con lo stato desiderato. Quando un oggetto Kubernetes viene creato, aggiornato o eliminato, il framework si occupa automaticamente di applicare le modifiche necessarie per raggiungere la configurazione dichiarata.

Ad esempio, se viene definito un oggetto che specifica l'esecuzione di un determinato container, Kubernetes si assicurerà che esso venga avviato e rimanga costantemente attivo. Se il container si arresta in modo imprevisto, il sistema provvede a riavviarlo o a crearne una nuova istanza su un nodo disponibile. Allo stesso modo, se un aggiornamento modifica lo stato desiderato di un'applicazione, Kubernetes applica automaticamente le modifiche in modo controllato, senza interrompere il servizio.

Questa capacità di autoregolazione e auto-ripristino rappresenta il fulcro dell'orchestrazione fornita da Kubernetes, consentendo di gestire infrastrutture complesse in modo affidabile, scalabile e resiliente, senza la necessità di interventi manuali.

4.4 Kubernetes nel Near-RT RIC

Nel contesto di O-RAN (Open Radio Access Network), Kubernetes rappresenta una soluzione chiave per gestire le xApp– applicazioni modulari che ottimizzano e migliorano la gestione delle risorse radio nel Near-RT RIC (Near-Real-Time RAN Intelligent Controller). Il Near-RT RIC è un componente essenziale dell'architettura O-RAN, responsabile dell'orchestrazione intelligente della rete mobile attraverso l'uso di algoritmi avanzati, machine learning e automazione basata su software.

L'adozione di Kubernetes nel Near-RT RIC porta diversi vantaggi:

- Scalabilità dinamica: le xApp possono essere scalate automaticamente in base al carico di rete e alle esigenze operative, garantendo prestazioni ottimali in ogni scenario.
- Affidabilità e tolleranza ai guasti: Kubernetes rileva automaticamente i problemi nei container e li riavvia o ridistribuisce su nodi funzionanti, mantenendo l'alta disponibilità del sistema.

- Gestione semplificata del ciclo di vita delle xApp: grazie a strumenti come Helm, è possibile automatizzare il deployment e la configurazione delle xApp in modo rapido e coerente.
- Separazione logica e isolamento: attraverso i namespace, Kubernetes consente di segmentare le diverse componenti del Near-RT RIC, migliorando l'organizzazione e la sicurezza.
- Automazione e self-healing: Kubernetes è in grado di garantire che lo stato del sistema rimanga coerente con le configurazioni desiderate, riducendo la necessità di intervento manuale.

Grazie a queste capacità, Kubernetes non è solo un'opzione per la gestione delle xApp, ma un vero e proprio abilitatore dell'architettura O-RAN, contribuendo a una rete più agile, efficiente e intelligente.

L'integrazione con Kubernetes consente inoltre di facilitare l'interoperabilità tra diversi fornitori di telecomunicazioni, allineandosi ai principi fondamentali di O-RAN Alliance, che promuove l'adozione di interfacce aperte e standardizzate per migliorare la modularità e la competizione nel mercato delle reti radio.

Kubernetes fornisce una serie di concetti fondamentali per la gestione e l'orchestrazione di applicazioni containerizzate, garantendo un'infrastruttura scalabile e resiliente. Nel contesto del **Near-RT RIC**, queste risorse consentono di gestire le **xApp** in modo efficiente, assicurando un controllo completo sul loro ciclo di vita e sulla loro interazione con l'ambiente circostante.

- **Pod**: l'unità atomica di Kubernetes. Il Pod rappresenta l'unità base di esecuzione in Kubernetes e racchiude uno o più container che condividono lo stesso ambiente di rete e storage. La sua funzione è quella di fornire un contesto isolato per l'esecuzione delle applicazioni, garantendo coerenza e indipendenza dai vincoli hardware e software del sistema sottostante. Nel Near-RT RIC, le xApp vengono eseguite all'interno di Pod Kubernetes, assicurando una separazione chiara tra le diverse applicazioni e agevolando la gestione della loro scalabilità. Ogni Pod è progettato per essere altamente disponibile e sostituibile in caso di malfunzionamenti, garantendo continuità operativa.
- **Service**: esposizione e bilanciamento del traffico. Poiché i Pod possono essere dinamicamente creati e distrutti, Kubernetes introduce il concetto di Service, un'astrazione che consente di esporre un insieme di Pod come un unico punto di accesso stabile. In questo modo, le applicazioni possono comunicare tra loro senza doversi preoccupare delle variazioni nella disposizione dei Pod all'interno del cluster. I Service garantiscono un bilanciamento del traffico efficace, consentendo alle xApp di interagire con altri componenti del Near-RT RIC senza dipendere direttamente dagli indirizzi IP dei Pod sottostanti. Questo approccio migliora la resilienza e la modularità dell'infrastruttura.
- **ReplicaSet**: garanzia di alta disponibilità. Per assicurare la disponibilità continua delle applicazioni, Kubernetes utilizza i ReplicaSet, meccanismi che monitorano costantemente lo stato dei Pod e ne mantengono attivo un numero predefinito. Se un Pod viene terminato o diventa instabile, Kubernetes provvede automaticamente a crearne una nuova istanza, assicurando che il servizio rimanga operativo senza interruzioni. Nel contesto delle xApp, ReplicaSet è essenziale per mantenere in esecuzione le applicazioni critiche e prevenire disservizi dovuti a guasti o carichi di lavoro variabili.
- **Deployment**: gestione del ciclo di vita delle applicazioni. L'aggiornamento e la manutenzione delle applicazioni containerizzate avvengono attraverso i Deployment, una risorsa Kubernetes che gestisce la creazione, l'aggiornamento e il rollback delle applicazioni. Questo meccanismo permette di distribuire nuove versioni delle xApp in modo graduale, minimizzando i tempi di inattività e riducendo i rischi di interruzione del servizio. Grazie ai Deployment, le applicazioni possono essere aggiornate senza causare downtime, con la possibilità di ripristinare rapidamente una versione precedente in caso di problemi. Questo garantisce una gestione più efficiente e sicura dell'infrastruttura software.

- **Namespace:** separazione logica delle componenti. Kubernetes permette di suddividere le risorse in Namespace, offrendo un livello di isolamento logico che facilita la gestione di ambienti complessi. Nel Near-RT RIC, questa segmentazione consente di organizzare le diverse componenti della piattaforma in spazi dedicati, migliorando l'ordine, la sicurezza e il controllo sugli accessi. Il cluster Kubernetes del Near-RT RIC separa i suoi pod in diversi namespace:
 - *ricinfra*: Contiene i pod che forniscono l'infrastruttura di supporto per il funzionamento del cluster Kubernetes.
 - *ricplt*: Contiene i pod dei componenti O-RAN nella piattaforma Near-RT RIC, come AppMgr, RtMgr, SubMgr, ecc.
 - *ricxapp*: Contiene tutti i pod delle xApp installate.

Questa struttura consente di mantenere separati i diversi livelli dell'architettura, semplificando la gestione e migliorando la scalabilità del sistema.

- **Helm Charts:** automazione del deployment e della configurazione. La gestione delle applicazioni su Kubernetes richiede strumenti avanzati per garantire un deployment strutturato, riproducibile e facilmente configurabile. Helm è il principale sistema di gestione dei pacchetti per Kubernetes, progettato per semplificare l'installazione, l'aggiornamento e la configurazione delle applicazioni containerizzate. Grazie a Helm, è possibile definire e distribuire rapidamente applicazioni complesse utilizzando i Helm Charts, pacchetti che contengono tutti i file di configurazione necessari per il corretto funzionamento delle risorse Kubernetes.

4.4.1 Helm: La gestione automatizzata del deployment

Nel contesto del Near-RT RIC, Helm [2] assume un ruolo centrale nella gestione delle xApp, consentendo di eseguire un deployment automatizzato e standardizzato. Questo processo è orchestrato da AppMgr, il modulo responsabile dell'onboarding, dell'aggiornamento e della rimozione delle xApp. Grazie a questa integrazione, le applicazioni vengono installate e configurate in modo coerente e uniforme, indipendentemente dall'ambiente in cui vengono eseguite.

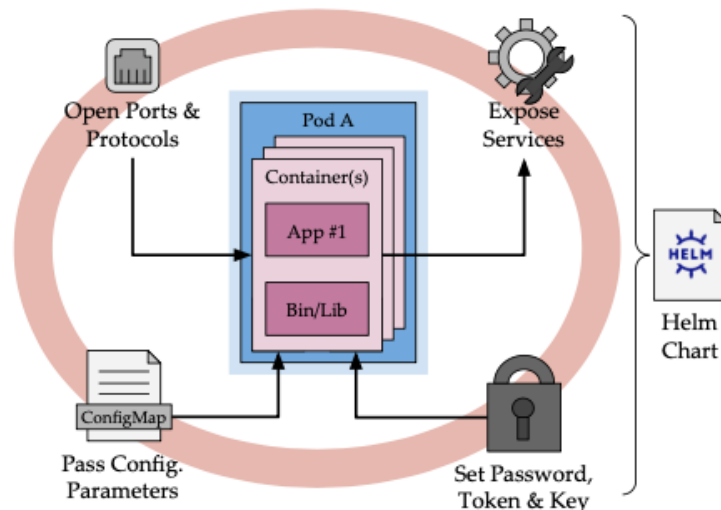


Figura 4.3 Distribuzione e gestione di una xApp con Helm Chart [2]

L'adozione di Helm consente di ridurre significativamente la complessità operativa associata alla gestione delle applicazioni Kubernetes. In particolare, Helm fornisce un modello dichiarativo per definire il comportamento delle applicazioni e garantisce che ogni deployment avvenga in maniera controllata e replicabile.

Come si può osservare nella Figura 4.3, un Helm Chart è composto da più file di configurazione, che descrivono le diverse proprietà e i requisiti delle applicazioni containerizzate. Tra questi, il file `values.yaml` svolge un ruolo chiave, permettendo agli utenti di personalizzare i parametri di configurazione prima della distribuzione nel cluster. Questo meccanismo fornisce un alto grado di flessibilità e consente agli sviluppatori di adattare rapidamente le applicazioni a contesti diversi senza dover modificare il codice sorgente.

Nell'ambiente O-RAN, la gestione delle xApp tramite Helm è supportata dall'OSC (O-RAN Software Community), che fornisce Helm Charts preconfigurati per facilitare il deployment dei componenti core del Near-RT RIC. Tuttavia, ogni sviluppatore di xApp è responsabile della creazione e della gestione dei propri Helm Charts, garantendo la compatibilità con il framework Kubernetes.

Per ottimizzare e velocizzare questo processo, l'OSC mette a disposizione `dms_cli`, uno strumento che automatizza la creazione degli Helm Charts per le xApp, semplificando la configurazione e la distribuzione delle applicazioni nel Near-RT RIC. Questo approccio favorisce una maggiore standardizzazione e riduce il rischio di errori manuali, contribuendo a un'adozione più efficiente della containerizzazione e dell'orchestrazione su larga scala.

Grazie all'uso di Helm, il deployment delle xApp nel Near-RT RIC diventa più efficiente, affidabile e scalabile, migliorando la gestione dell'infrastruttura Kubernetes e garantendo un controllo più granulare sulle configurazioni applicative.

4.5 Ciclo di vita delle xApp in Kubernetes

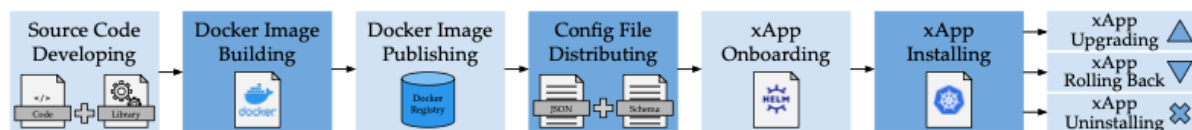


Figura 4.4 Ciclo di vita delle xApp in Kubernetes [2]

Le xApp all'interno del Near-RT RIC seguono un ciclo di vita strutturato che comprende diverse fasi, come rappresentato nella Figura 4.4, dall'onboarding iniziale fino alla loro eventuale rimozione, passando attraverso il deployment, l'esecuzione e gli aggiornamenti periodici. L'obiettivo di questo modello è garantire che le applicazioni siano sempre disponibili, scalabili e facilmente gestibili, senza richiedere interventi manuali da parte degli operatori.

Per realizzare questa gestione in modo efficiente, il Near-RT RIC sfrutta un'infrastruttura basata su Kubernetes, combinata con strumenti di automazione come Helm. L'adozione di Helm consente di standardizzare e semplificare il deployment e la configurazione delle xApp, eliminando la necessità di definire manualmente le risorse Kubernetes e affidando invece la gestione delle applicazioni a un sistema dichiarativo e riproducibile.

Nel Near-RT RIC, l'intero ciclo di vita delle xApp è supervisionato da AppMgr (RIC Application Manager), il modulo che si occupa di orchestrare l'installazione, l'aggiornamento e la rimozione delle applicazioni nel cluster Kubernetes. Questo processo avviene in modo automatizzato, basandosi su file di configurazione in formato JSON (`descriptor.json` e `schema.json`), che contengono tutte le informazioni necessarie per la gestione della xApp senza dover intervenire direttamente sulle configurazioni di Kubernetes.

Dopo la fase di sviluppo descritta nel Capitolo 1, si passa alla creazione dell'immagine Docker della xApp. Per fare ciò, lo sviluppatore deve preparare un **Dockerfile**, che contiene le istruzioni necessarie per la costruzione dell'immagine. Questo file definisce l'ambiente in cui la xApp verrà eseguita,

specificando la versione del linguaggio, le librerie richieste e le directory necessarie. Una volta scritto il Dockerfile, l'immagine Docker viene generata attraverso il comando `docker build`. In questa fase è essenziale includere tutte le dipendenze richieste dalla xApp per garantire un funzionamento corretto una volta che sarà eseguita all'interno del Near-RT RIC.

Completata la creazione dell'immagine Docker, si procede con la pubblicazione su un **Docker Registry**. Prima della pubblicazione, l'immagine viene testata localmente per verificare che la xApp funzioni correttamente in un ambiente containerizzato. Successivamente, lo sviluppatore assegna un'etichetta all'immagine e la carica nel registry designato, che può essere pubblico o privato. Questo passaggio è necessario affinché il Near-RT RIC possa recuperare l'immagine e avviare i container della xApp. Una volta completata la pubblicazione, lo sviluppatore verifica la disponibilità dell'immagine nel registry per assicurarsi che sia accessibile e pronta per l'uso.

Dopo la pubblicazione dell'immagine Docker, è necessario condividere i file di configurazione della xApp. In particolare, devono essere forniti il **descriptor.json**, che contiene i metadati e i parametri di controllo richiesti dalla xApp, e lo **schema.json**, che definisce la struttura dei parametri e ne consente la validazione. Il descrittore della xApp specifica il nome, la versione e le opzioni di configurazione disponibili, mentre lo schema permette al sistema di verificare che i parametri siano corretti prima dell'installazione della xApp nel Near-RT RIC.

4.6 Automazione del ciclo di vita delle xApp con `dms_cli`

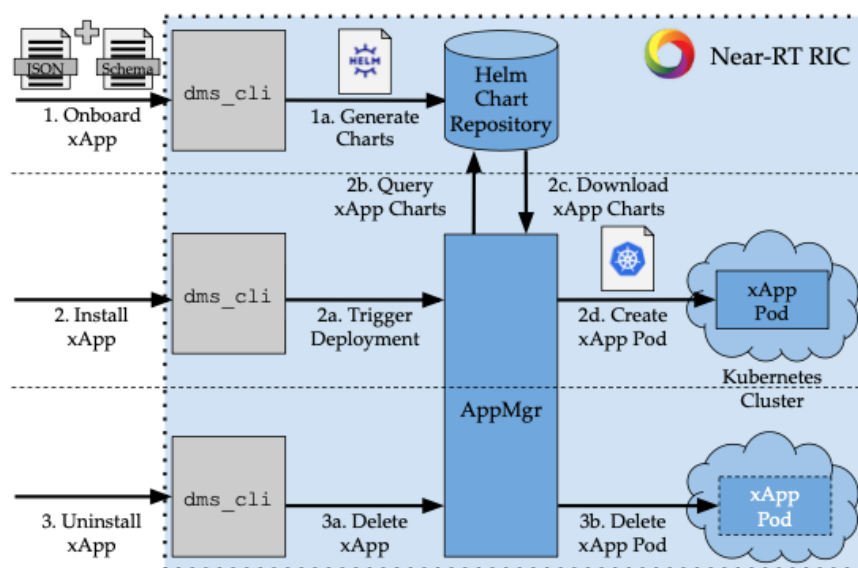


Figura 4.5 Onboarding, installazione e disinstallazione delle xApp nel Near-RT RIC con `dms_cli` [2]

L'**onboarding**[2] di una xApp consiste nel processo di registrazione e integrazione dell'applicazione all'interno del Near-RT RIC. Durante questa fase, lo sviluppatore della xApp fornisce un insieme di file JSON che descrivono i metadati dell'applicazione, incluse informazioni come il nome, la versione, le dipendenze e le risorse necessarie per l'esecuzione. AppMgr si occupa di verificare la compatibilità della xApp con il sistema, assicurandosi che rispetti le specifiche del Near-RT RIC e che possa interagire correttamente con gli altri componenti della piattaforma. Una volta completata questa fase, l'applicazione viene preparata per essere distribuita nel cluster Kubernetes.

Il **deployment** della xApp è un processo completamente automatizzato, reso possibile grazie a Helm. Invece di dover scrivere manualmente i file di configurazione YAML per definire le risorse Kubernetes necessarie, come Pod, Service e altri componenti, il deployment viene gestito attraverso Helm Charts,

pacchetti preconfigurati che generano automaticamente tutte le risorse necessarie sulla base delle informazioni fornite nei file JSON. Questo approccio consente di distribuire la xApp in modo coerente e riproducibile, riducendo la complessità operativa e minimizzando il rischio di errori.

Durante l'**installazione**, Kubernetes si occupa di avviare le istanze della xApp nei nodi disponibili del cluster, distribuendo le risorse in base alla capacità del sistema e garantendo che l'applicazione sia operativa e pronta a comunicare con gli altri componenti della rete. Grazie alla natura modulare di Kubernetes, l'installazione delle xApp può essere scalata in base alle necessità operative, permettendo di eseguire più istanze della stessa applicazione su diversi nodi per garantire una maggiore disponibilità e bilanciamento del carico.

Dopo l'installazione, la gestione continua delle xApp include anche le operazioni di **aggiornamento**. Come qualsiasi altra applicazione, anche le xApp necessitano di aggiornamenti periodici per migliorare le prestazioni, correggere eventuali bug o introdurre nuove funzionalità. Kubernetes offre strumenti avanzati per eseguire aggiornamenti senza downtime, garantendo la continuità del servizio attraverso i cosiddetti rolling updates. Questo meccanismo permette di sostituire progressivamente le istanze della xApp con una nuova versione, evitando interruzioni nel funzionamento dell'applicazione. L'aggiornamento avviene attraverso Helm, che distribuisce la nuova versione della xApp e monitora costantemente il suo stato di salute. Se la nuova versione funziona correttamente, le vecchie istanze vengono progressivamente rimosse. In caso di problemi, Kubernetes è in grado di eseguire un rollback automatico, ripristinando rapidamente la versione precedente per evitare interruzioni nel servizio.

Quando una xApp non è più necessaria o deve essere sostituita, viene avviato il processo di decommissioning, che prevede la **rimozione** dell'applicazione dal Near-RT RIC in modo sicuro e controllato. La rimozione di una xApp è un'operazione critica, poiché richiede l'eliminazione di tutte le risorse associate senza compromettere l'integrità del cluster Kubernetes. Durante questa fase, Kubernetes interrompe progressivamente le istanze della xApp, assicurandosi che la terminazione avvenga in modo controllato per evitare interruzioni nei servizi dipendenti. Successivamente, vengono deallocate le risorse che erano state assegnate alla xApp, come CPU, memoria e storage, e viene completata la deregistrazione della xApp dal sistema di gestione centrale. Infine, le configurazioni e i dati persistenti associati all'applicazione vengono eliminati, garantendo che non rimangano tracce della xApp all'interno del cluster. Questo processo consente di mantenere l'infrastruttura Kubernetes sempre ottimizzata, evitando sprechi di risorse e assicurando che il sistema sia pronto per accogliere nuove applicazioni.

L'adozione di Kubernetes per la gestione del ciclo di vita delle xApp nel Near-RT RIC rappresenta un passo fondamentale verso un'infrastruttura più scalabile, resiliente e automatizzata. Grazie all'integrazione con Helm e AppMgr, il deployment e la gestione delle xApp avvengono in maniera completamente automatizzata, riducendo il carico operativo e garantendo una gestione più efficiente delle applicazioni. Questo approccio elimina la necessità di configurazioni manuali, offrendo agli sviluppatori un sistema dichiarativo in cui è sufficiente fornire i file JSON con le specifiche della xApp affinché il sistema si occupi di tutto il resto.

L'uso combinato di Kubernetes e Helm consente al Near-RT RIC di adattarsi in modo dinamico alle esigenze della rete, ottimizzando l'utilizzo delle risorse e garantendo un'infrastruttura altamente scalabile e flessibile. L'approccio dichiarativo adottato facilita la gestione centralizzata delle xApp, consentendo agli operatori di rete di concentrarsi sull'ottimizzazione continua dei servizi senza dover intervenire manualmente sulle configurazioni delle applicazioni. Questa architettura consente di migliorare significativamente l'efficienza operativa del Near-RT RIC, riducendo i tempi di deployment e aggiornamento delle applicazioni e garantendo un controllo più efficace sulle risorse della rete.

Grazie a questa infrastruttura, il Near-RT RIC è in grado di evolversi rapidamente in risposta alle nuove esigenze delle reti mobili di prossima generazione, assicurando una gestione efficace e dinamica delle xApp e contribuendo alla creazione di un ambiente O-RAN più aperto, interoperabile e adattabile.

5 Control Plane integrato per LoRa e RAN

L'evoluzione delle reti di telecomunicazioni ha portato alla necessità di integrare tecnologie eterogenee per supportare un'ampia gamma di applicazioni, dalla connettività a bassa latenza per dispositivi mobili al supporto di reti di sensori IoT. In questo contesto, l'integrazione tra 5G RAN (Radio Access Network) e LoRa (Long Range) rappresenta un'importante sfida e opportunità, specialmente per applicazioni IoT che richiedono basso consumo energetico e lunga portata.

L'architettura O-RAN (Open Radio Access Network) introduce un modello aperto e programmabile per il controllo della rete, consentendo l'implementazione di funzioni intelligenti attraverso componenti modulari come il Near-RT RIC (Near-Real-Time RAN Intelligent Controller). Grazie all'uso di xApp personalizzate, il Near-RT RIC può coordinare il traffico LoRa e il traffico 5G in maniera dinamica, ottimizzando l'utilizzo delle risorse di rete e migliorando la Qualità del Servizio (QoS).

L'integrazione tra reti 5G RAN e LoRa è fondamentale per colmare il divario tra reti a banda larga ad alta velocità e reti IoT a basso consumo. Mentre il 5G è progettato per garantire connessioni ad alta velocità e bassa latenza, LoRa si concentra su comunicazioni a lungo raggio e a basso consumo, caratteristiche essenziali per dispositivi IoT che devono operare per anni senza sostituzione della batteria. L'integrazione tra queste due tecnologie abilita nuovi scenari applicativi:

- Gestione intelligente delle risorse radio: L'uso del Near-RT RIC permette di ottimizzare lo scheduling tra i dispositivi LoRa e le reti 5G, riducendo le interferenze e massimizzando l'efficienza dello spettro radio.
- Coordinamento dinamico delle reti: Il Near-RT RIC può adattare i parametri operativi di LoRa (ad esempio, Spreading Factor, potenza di trasmissione) in base alle condizioni della rete 5G, garantendo un migliore utilizzo delle risorse radio.
- Interoperabilità tra reti eterogenee: Attraverso l'uso di un modello di servizio personalizzato (E2SM-LoRa), il Near-RT RIC può interfacciarsi con i gateway LoRa e controllare direttamente le loro operazioni, creando una gestione unificata delle reti IoT e 5G.

L'obiettivo di questa integrazione non è solo quello di consentire la coesistenza delle due tecnologie, ma anche di sfruttare le capacità di analisi e ottimizzazione del Near-RT RIC per migliorare l'efficienza complessiva della rete, eseguendo l'offloading computazionale delle ottimizzazioni più pesanti su una rete che dispone di maggiori risorse, come il 5G, e trasferendo le configurazioni ottimizzate alla rete LoRa.

Il caso d'uso analizzato in questo capitolo ha l'obiettivo di dimostrare come un Control Plane 5G RAN possa coordinarsi con dispositivi LoRa attraverso l'architettura O-RAN, utilizzando il Near-RT RIC. L'integrazione avviene tramite:

- L'uso del Near-RT RIC come unità di coordinamento centrale, capace di raccogliere KPI (Key Performance Indicators) da dispositivi LoRa e dalla rete 5G, fornendo un controllo dinamico dei parametri di comunicazione.
- Lo sviluppo di un E2 Service Model personalizzato (E2SM-LoRa), che permette al Near-RT RIC di interfacciarsi direttamente con i gateway LoRa, abilitando il monitoraggio e l'ottimizzazione delle comunicazioni.
- L'implementazione di xApp intelligenti, che analizzano i dati in tempo reale e regolano dinamicamente i parametri di LoRa e 5G, ad esempio modificando la potenza di trasmissione o lo scheduling delle risorse radio.
- La gestione efficiente delle risorse radio tra dispositivi LoRa e 5G, con l'obiettivo di ridurre le interferenze e migliorare il throughput della rete.

L'integrazione tra il Near-RT RIC e i dispositivi LoRa consente di ottenere un controllo più fine del traffico IoT, migliorando sia le prestazioni della rete che l'efficienza energetica dei dispositivi LoRa.

Questa integrazione può essere applicata a diversi scenari reali, nei quali i dispositivi LoRa necessitano di connettività avanzata e gestione intelligente attraverso un'infrastruttura 5G. Tra le applicazioni più rilevanti si trovano: Monitoraggio ambientale (qualità dell'aria, temperatura, umidità) tramite sensori LoRa gestiti dal Near-RT RIC; controllo del traffico e della mobilità urbana, con sensori IoT distribuiti per raccogliere dati in tempo reale; coordinamento delle comunicazioni tra sensori LoRa e droni 5G per la gestione agricola intelligente; sensori per il rilevamento precoce di incendi, inondazioni o terremoti, con allarmi inviati in tempo reale tramite 5G; sistemi di sicurezza basati su LoRa per il monitoraggio di aree sensibili. Questi casi d'uso dimostrano il valore dell'integrazione tra 5G e LoRa, che consente di combinare la capillarità e l'efficienza energetica di LoRa con la velocità e l'affidabilità delle reti 5G.

Questo capitolo descrive nel dettaglio il processo di integrazione di LoRa in O-RAN, partendo dalla definizione dell'architettura del sistema, passando attraverso la necessità di un service model personalizzato, fino alla descrizione dettagliata di E2SM-LoRa e della sua implementazione. Verranno illustrati i principi di funzionamento del modello, le scelte progettuali adottate e il codice sviluppato per la gestione della comunicazione tra il Near-RT RIC e la rete LoRa. L'obiettivo finale è dimostrare come sia possibile estendere l'architettura O-RAN per supportare reti LPWAN, aprendo la strada a una gestione unificata e intelligente delle reti IoT e 5G.

5.1 Architettura del sistema

L'integrazione della tecnologia LoRa nell'ecosistema O-RAN è stata progettata attraverso un'architettura modulare che consente la comunicazione tra il Near-RT RIC e i dispositivi LoRa End-Devices, passando attraverso un E2 Term personalizzato e un LoRa Gateway. L'obiettivo di questa architettura è abilitare un controllo più avanzato della rete LoRa, permettendo al Near-RT RIC di monitorare e configurare dinamicamente i dispositivi LoRa in tempo reale, sfruttando l'interfaccia E2 di O-RAN.

Tradizionalmente, nelle reti LoRaWAN, i pacchetti trasmessi dai dispositivi LoRa vengono ricevuti dal LoRa Gateway, che li inoltra a un LoRa Network Server (LNS). Il LNS è responsabile della gestione della rete, dell'ottimizzazione delle comunicazioni e della distribuzione dei dati verso le applicazioni finali. Tuttavia, in questa soluzione, l'infrastruttura è stata semplificata eliminando il LNS, permettendo una comunicazione diretta tra il LoRa Gateway e l'E2 Term attraverso il protocollo UDP. Questo approccio riduce la latenza nella configurazione dei dispositivi e migliora la compatibilità con l'architettura O-RAN. L'obiettivo principale di questa integrazione è consentire al Near-RT RIC di coordinare il traffico tra dispositivi LoRa e la rete 5G, migliorando l'allocazione delle risorse radio e garantendo una Qualità del Servizio (QoS) ottimale per applicazioni IoT.

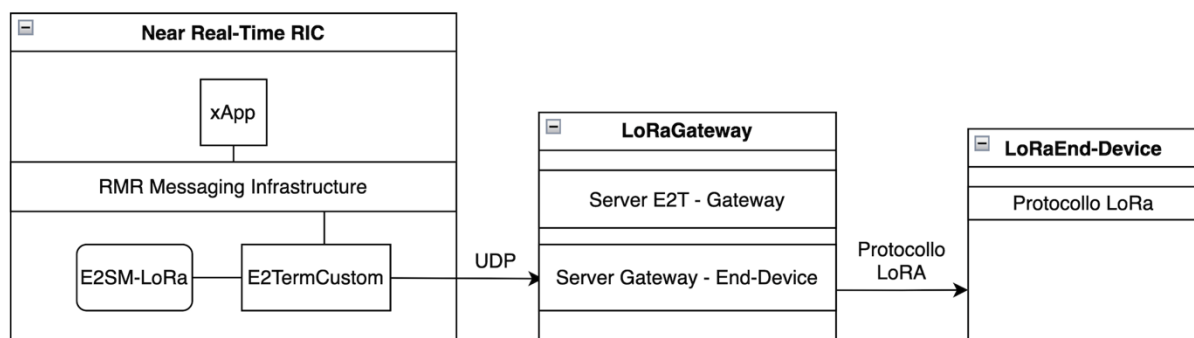


Figura 5.1 Architettura dell'integrazione tra LoRa e Near-RT RIC

L'architettura del sistema è composta da quattro elementi principali, ciascuno con un ruolo ben definito nel processo di comunicazione e gestione della rete LoRa:

- Near-RT RIC e xApp – Il controller centrale che raccoglie dati, invia comandi e gestisce le risorse di rete.
- E2 Termination Custom (E2TermCustom) – Il componente che traduce i messaggi tra Near-RT RIC e LoRa Gateway.
- LoRa Gateway – Il nodo che riceve comandi dal Near-RT RIC, li applica ai dispositivi LoRa e raccoglie i dati dai sensori.
- LoRa End-Devices – I dispositivi IoT finali che trasmettono dati ambientali e ricevono aggiornamenti di configurazione.

5.1.1 Near-RT RIC e xApp

L'integrazione della rete LoRa nell'architettura O-RAN ha richiesto l'estensione delle funzionalità del Near-RT RIC e lo sviluppo di una xApp dedicata alla gestione della rete LoRa. Il Near-RT RIC rappresenta un componente essenziale del framework O-RAN, progettato per ottimizzare e gestire dinamicamente le risorse radio con latenze estremamente ridotte, variabili da alcune decine di millisecondi fino a pochi secondi. A differenza del Non-RT RIC, che opera su scale temporali più lunghe per attività di ottimizzazione strategica, il Near-RT RIC è responsabile di decisioni rapide per adattare la rete in tempo quasi reale alle condizioni di traffico e alle esigenze operative.

L'integrazione di LoRa in questa architettura ha rappresentato una sfida, poiché la tecnologia LoRa non supporta nativamente l'interfaccia E2, utilizzata dal Near-RT RIC per interagire con gli elementi della rete radio. Per colmare questa lacuna, è stato sviluppato un E2 Term personalizzato, che consente di tradurre le istruzioni del Near-RT RIC in un formato compatibile con la rete LoRa e, viceversa, di raccogliere i dati provenienti dai dispositivi LoRa e restituirli al controller O-RAN. Questo sistema ha permesso di creare un'interfaccia di comunicazione bidirezionale, consentendo al Near-RT RIC di estendere le proprie capacità di gestione a una tecnologia originariamente non prevista nel modello O-RAN.

Un elemento chiave di questa architettura è la xApp, un'applicazione software eseguita direttamente all'interno del Near-RT RIC, responsabile dell'implementazione della logica di gestione della rete LoRa. La xApp funge da meccanismo centrale per il controllo e la gestione remota dei dispositivi LoRa, coordinando le operazioni necessarie per la configurazione, il monitoraggio e l'ottimizzazione della rete. Il suo funzionamento si articola in diverse fasi, che includono l'invio di richieste di configurazione ai dispositivi LoRa, la registrazione per ricevere le metriche di rete, l'analisi delle risposte provenienti dal LoRa Gateway e l'adattamento dinamico dei parametri di rete sulla base dei dati raccolti.

Una volta completata l'inizializzazione del contesto di comunicazione, la xApp è in grado di costruire e trasmettere un Control Request, un messaggio utilizzato per modificare in tempo reale i parametri operativi dei dispositivi LoRa. Questo processo consente alla xApp di adattare la configurazione della rete in base alle condizioni operative e ai requisiti di sistema, ottimizzando la qualità della trasmissione, l'efficienza energetica e la copertura del segnale.

L'integrazione tra Near-RT RIC e rete LoRa è resa più efficiente da un meccanismo di aggiornamenti periodici gestito dall'E2 Term. Una volta che la richiesta di registrazione è stata accolta, l'E2 Term raccoglie i dati dal LoRa Gateway e li inoltra alla xApp senza la necessità di interrogazioni manuali. Questo meccanismo riduce il carico sulle risorse di sistema e garantisce alla xApp un flusso costante di dati aggiornati per la gestione della rete.

Per assicurare una comunicazione efficace tra la xApp e la rete LoRa attraverso l'E2 Term, è stato sviluppato un modello di servizio personalizzato denominato E2SM-LoRa. Questo modello definisce un formato standardizzato per i messaggi scambiati tra il Near-RT RIC e l'E2 Term, garantendo la corretta interpretazione dei comandi da parte dei dispositivi LoRa e la trasmissione strutturata delle metriche di rete verso il Near-RT RIC.

La xApp implementa un ciclo operativo continuo che comprende l'inizializzazione del contesto di messaggistica RMR, la creazione e l'invio dei Control Request, l'attesa delle conferme di ricezione, la registrazione per il monitoraggio delle metriche e l'adattamento dinamico dei parametri di rete. Questo processo consente alla xApp di migliorare l'efficienza operativa della rete LoRa, garantendo una comunicazione stabile e ottimizzando la gestione della trasmissione dati all'interno di un'architettura altamente dinamica come quella O-RAN.

5.1.2 E2 Termination Custom

L'integrazione della rete LoRa nell'architettura O-RAN ha richiesto lo sviluppo di un componente software personalizzato, denominato E2 Termination Custom, progettato per fungere da intermediario tra il Near-RT RIC e la rete LoRa. Poiché LoRa non supporta nativamente l'interfaccia E2, standard dell'architettura O-RAN per la gestione e il controllo delle risorse radio, è stato necessario creare un sistema capace di tradurre i messaggi tra il Near-RT RIC e i dispositivi LoRa. Senza questo elemento, l'integrazione tra LoRa e O-RAN non sarebbe possibile, poiché le due tecnologie utilizzano protocolli di comunicazione completamente differenti e non compatibili tra loro.

Per superare questa incompatibilità, è stato sviluppato l'E2 Termination Custom, un componente software che consente al Near-RT RIC di interfacciarsi con la rete LoRa, permettendo la trasmissione dei comandi di controllo e la ricezione delle metriche di rete. Questo sistema si inserisce come elemento chiave tra il LoRa Gateway e l'architettura O-RAN, svolgendo la funzione di traduzione tra i due ambienti e garantendo un'integrazione efficace tra le due tecnologie. A differenza dell'E2 Termination standard, progettato per gestire la comunicazione tra il Near-RT RIC e i nodi radio tradizionali del 5G come O-DU e O-CU, l'E2 Termination Custom è stato sviluppato appositamente per elaborare e convertire i dati provenienti dai dispositivi LoRa in un formato compatibile con il protocollo E2AP. Questa trasformazione avviene attraverso l'adozione di un modello di servizio personalizzato, denominato E2SM-LoRa, che definisce il formato e la struttura dei messaggi scambiati tra il Near-RT RIC e la rete LoRa, garantendo che i comandi inviati dalla xApp siano interpretati correttamente dai dispositivi e che le metriche raccolte vengano trasmesse in modo conforme all'architettura O-RAN.

L'E2 Termination Custom ha il compito di gestire il flusso di comunicazione tra il Near-RT RIC e i dispositivi LoRa, assicurando che i messaggi vengano inoltrati nel formato corretto e che le risposte vengano elaborate e restituite tempestivamente alla xApp. Questo componente rimane costantemente in ascolto dei messaggi inviati dal Near-RT RIC attraverso il protocollo RMR, un'infrastruttura di messaggistica affidabile che consente il trasporto dei pacchetti all'interno dell'architettura O-RAN. Il ruolo dell'E2 Termination Custom si articola in due funzioni principali. Da un lato, si occupa della gestione delle configurazioni dei dispositivi LoRa, trasformando i comandi ricevuti dalla xApp in istruzioni eseguibili per i nodi LoRa, consentendo loro di adattare il proprio comportamento operativo in base alle nuove impostazioni definite dal Near-RT RIC. Dall'altro, raccoglie metriche di rete in tempo reale, permettendo alla xApp di monitorare continuamente lo stato della rete LoRa e di ottimizzare le impostazioni operative dei dispositivi in base alle condizioni effettive della rete.

L'integrazione della rete LoRa con l'architettura O-RAN non si limita alla sola gestione remota delle configurazioni dei dispositivi, ma include anche un sistema di monitoraggio continuo che consente alla xApp di ricevere aggiornamenti in tempo reale sulle condizioni della rete. Per evitare che la xApp debba interrogare manualmente il Near-RT RIC, l'E2 Termination Custom implementa un meccanismo basato sulle Subscription Requests, che automatizza il processo di raccolta e inoltro delle metriche di rete.

Questo sistema consente alla xApp di ricevere informazioni periodiche e aggiornate, riducendo il carico sulla rete e migliorando l'efficienza complessiva della comunicazione.

Attraverso questa architettura, il Near-RT RIC è in grado di ottimizzare dinamicamente la rete LoRa, garantendo che le configurazioni inviate dalla xApp siano applicate correttamente ai dispositivi, migliorando le prestazioni complessive del sistema e assicurando un'integrazione efficace tra l'architettura O-RAN e la tecnologia LoRa.

5.1.3 LoRa Gateway

L'integrazione del LoRa Gateway nell'architettura O-RAN è stata progettata per garantire una comunicazione diretta ed efficiente tra i dispositivi LoRa e il Near-RT RIC, eliminando la necessità di un LoRa Network Server (LNS). In una tipica architettura LoRaWAN, il gateway funge da ponte tra i dispositivi LoRa e il LNS, che gestisce il traffico di rete e la distribuzione delle configurazioni. Nell'implementazione sviluppata, tuttavia, il LoRa Gateway comunica direttamente con l'E2 Term, impiegando il protocollo UDP come meccanismo di trasporto per garantire una comunicazione a bassa latenza e ridurre l'overhead introdotto da protocolli più complessi come MQTT o HTTP.

Questa soluzione consente al LoRa Gateway non solo di ricevere le configurazioni provenienti dal Near-RT RIC, ma anche di applicarle direttamente sui dispositivi LoRa. Grazie a questa architettura, il gateway può aggiornare dinamicamente parametri fondamentali come la frequenza operativa, la potenza di trasmissione e lo spreading factor (SF), assicurando che i dispositivi funzionino nelle condizioni ottimali in base alle esigenze della rete e ai parametri definiti dal Near-RT RIC.

Il ruolo del LoRa Gateway è cruciale nell'architettura O-RAN poiché consente la configurazione remota e dinamica dei dispositivi LoRa senza la necessità di intervento manuale. Attraverso l'interazione con l'E2 Term, il gateway agisce come intermediario tra il Near-RT RIC e i dispositivi LoRa, traducendo e applicando le configurazioni ricevute per garantire un controllo efficiente della rete. Durante la fase di trasmissione dei comandi, il sistema monitora lo stato dell'operazione e registra i dettagli nei log, permettendo di tracciare l'intero processo di aggiornamento e garantendo la trasparenza nella gestione delle configurazioni.

Se la configurazione viene applicata con successo, il LoRa Gateway invia una conferma positiva all'E2 Term, notificando che il nodo LoRa ha aggiornato i propri parametri. In caso di problemi, invece, il sistema genera un messaggio di errore dettagliato che indica il motivo della mancata applicazione della configurazione. Questo meccanismo di feedback è essenziale affinché la xApp, in esecuzione nel Near-RT RIC, possa identificare eventuali problemi e ripetere l'operazione se necessario.

L'architettura implementata garantisce un'elevata efficienza nella gestione delle configurazioni grazie all'uso del protocollo UDP, che permette una comunicazione leggera e a bassa latenza tra l'E2 Term e il LoRa Gateway. Inoltre, l'impiego del formato JSON per la rappresentazione dei dati facilita l'integrazione con l'architettura O-RAN, offrendo una gestione delle configurazioni più flessibile e scalabile.

Grazie a questa soluzione, il LoRa Gateway è in grado di adattare dinamicamente la rete LoRa alle esigenze operative, migliorandone le prestazioni senza necessità di interventi manuali e garantendo un'integrazione efficiente tra O-RAN e LoRa.

5.1.4 LoRa End-Device

I LoRa End-Devices rappresentano il livello terminale della rete LoRa, essendo i dispositivi responsabili della trasmissione e ricezione dei dati attraverso il protocollo LoRa. Questi dispositivi, che includono

principalmente sensori e attuatori, sono progettati per operare in contesti IoT (Internet of Things), dove è essenziale garantire una comunicazione a lunga distanza mantenendo al minimo il consumo energetico. I dispositivi LoRa utilizzati in questa implementazione supportano diverse funzionalità avanzate che li rendono ideali per applicazioni IoT a lungo raggio.

1. Frequenza operativa e copertura del segnale

Uno degli elementi chiave della trasmissione LoRa è l'utilizzo della banda di 868 MHz, una frequenza specificamente destinata alle comunicazioni LoRa in Europa. Questo parametro risulta fondamentale per garantire un funzionamento efficace in scenari IoT, in quanto offre minore interferenza rispetto a bande più trafficate, come i 2.4 GHz, utilizzati da tecnologie come Wi-Fi e Bluetooth. Un altro vantaggio dell'utilizzo della banda 868 MHz è la sua capacità di garantire una maggiore copertura rispetto a frequenze più alte. Le onde radio trasmesse a questa frequenza hanno una migliore capacità di attraversare ostacoli e possono coprire distanze molto ampie, che in condizioni ottimali possono raggiungere diversi chilometri. Questo rende la tecnologia LoRa particolarmente efficace sia in ambienti outdoor, dove la copertura su lunghe distanze è fondamentale, sia in ambienti indoor, dove la propagazione del segnale deve essere garantita anche attraverso pareti e altre barriere fisiche.

2. Regolazione Dinamica della Potenza di Trasmissione

I dispositivi LoRa utilizzati in questa implementazione supportano una potenza di trasmissione regolabile fino a 22 dBm, permettendo così di adattare dinamicamente il livello di potenza in base alle esigenze operative. Un livello di potenza elevato, fino al massimo di 22 dBm, consente di estendere significativamente la copertura del segnale, rendendo possibile la comunicazione su lunghe distanze. Tuttavia, un aumento della potenza comporta anche un maggiore consumo energetico, riducendo così l'autonomia operativa del dispositivo. Al contrario, un livello di potenza inferiore consente di ridurre il consumo energetico, prolungando la durata della batteria del dispositivo, a scapito però della portata della trasmissione. Per ottimizzare questo compromesso tra efficienza energetica e copertura, il Near-RT RIC può modificare dinamicamente la potenza di trasmissione tramite la xApp, adattando le impostazioni in tempo reale in base alle condizioni della rete.

3. Gestione dello Spreading Factor (SF) e ottimizzazione della trasmissione

Un altro parametro fondamentale nella comunicazione LoRa è lo Spreading Factor (SF), che definisce la durata del simbolo trasmesso e incide direttamente sul compromesso tra portata del segnale e velocità di trasmissione. Un valore basso di SF (ad esempio 7 o 8) consente di ottenere una maggiore velocità di trasmissione, ma riduce la portata del segnale, rendendolo meno adatto a comunicazioni su lunghe distanze. Viceversa, un valore alto di SF (tra 11 e 12) permette di estendere la copertura, ma a scapito della velocità di trasmissione. L'architettura implementata consente di modificare dinamicamente lo SF, permettendo al Near-RT RIC di adattare le impostazioni del dispositivo attraverso il LoRa Gateway, in base alle condizioni della rete e ai requisiti operativi. Questa capacità di regolazione automatica assicura un bilanciamento ottimale tra affidabilità della trasmissione e consumo energetico, migliorando le prestazioni complessive della rete.

4. Modalità di Funzionamento: Comunicazione Sincrona e Asincrona

I dispositivi LoRa possono operare in diverse modalità di comunicazione: modalità sincrona dove il dispositivo trasmette i dati a intervalli regolari, inviando informazioni di telemetria al LoRa Gateway con una frequenza predefinita e modalità event-driven dove il dispositivo invia dati solo quando necessario, ad esempio in risposta a un evento specifico, come la rilevazione di un'anomalia o un cambiamento nelle condizioni ambientali. Questa flessibilità permette di adattare il comportamento del dispositivo alle esigenze dell'applicazione.

L'integrazione dei dispositivi LoRa End-Devices nell'architettura O-RAN è stata progettata per consentire una gestione dinamica e automatizzata delle loro configurazioni operative, migliorando l'efficienza della rete. Una volta ricevuta la configurazione trasmessa dal Near-RT RIC attraverso l'E2 Term e il LoRa Gateway, il dispositivo applica immediatamente le nuove impostazioni, aggiornando i parametri operativi in base alle istruzioni ricevute. L'aggiornamento può riguardare diversi aspetti chiave della trasmissione, come la modifica della potenza di trasmissione per ottimizzare la copertura del segnale o l'adattamento dello Spreading Factor, che permette di bilanciare la portata della trasmissione e la velocità di trasferimento dati. Grazie a questa capacità di auto-configurazione, il dispositivo LoRa può operare in condizioni ottimali senza interruzioni della comunicazione.

Dopo aver aggiornato la propria configurazione, il dispositivo inizia a trasmettere i dati di telemetria al LoRa Gateway utilizzando il modulo SX1262, che invia pacchetti di dati contenenti informazioni sullo stato della connessione e altre metriche di rete. Il LoRa Gateway riceve questi dati e li inoltra all'E2 Term, che si occupa di analizzarli e di trasmetterli al Near-RT RIC per l'elaborazione. Tra le informazioni raccolte, una delle metriche più significative è il Received Signal Strength Indicator (RSSI), che fornisce una misura della qualità del segnale ricevuto. Questo valore risulta essenziale per valutare l'efficacia della configurazione attuale e determinare se siano necessarie eventuali ottimizzazioni per garantire una comunicazione stabile ed efficiente.

L'analisi continua dei dati raccolti permette al Near-RT RIC di monitorare lo stato della rete e di individuare eventuali condizioni in cui sia necessario modificare i parametri operativi dei dispositivi LoRa. Se, ad esempio, viene rilevata una diminuzione della qualità del segnale, come un valore RSSI troppo basso, il sistema può generare un nuovo Control Request per adattare la configurazione del dispositivo e migliorare la connessione. Questo aggiornamento può consistere in un aumento della potenza di trasmissione per estendere la copertura del segnale o in una modifica dello Spreading Factor per ottimizzare il rapporto tra portata e velocità di trasmissione. Il nuovo messaggio di configurazione viene quindi inviato attraverso l'E2 Term, che lo inoltra al LoRa Gateway e, successivamente, al LoRa End-Device, avviando così un nuovo ciclo di aggiornamento dinamico della rete.

Grazie a questo meccanismo di monitoraggio e adattamento continuo, la rete LoRa è in grado di rispondere in tempo reale alle variazioni delle condizioni ambientali e operative, garantendo sempre la migliore qualità della connessione possibile. L'interazione tra il Near-RT RIC, l'E2 Term, il LoRa Gateway e i LoRa End-Devices consente di creare un sistema intelligente e reattivo, in cui la configurazione della rete può essere aggiornata costantemente senza richiedere interventi manuali. Questa architettura rende il sistema altamente scalabile ed efficiente, permettendo di gestire un elevato numero di dispositivi LoRa in modo automatico e ottimizzato, garantendo così un'integrazione efficace tra la rete LoRa e l'architettura O-RAN.

5.2 E2SM-LoRa: Necessità e Scopo

L'integrazione della tecnologia LoRa all'interno dell'architettura O-RAN ha richiesto la progettazione e implementazione di un Service Model personalizzato, denominato E2SM-LoRa, capace di colmare il divario tra le due tecnologie e consentire una gestione centralizzata dei dispositivi LoRa attraverso il Near-RT RIC. Poiché LoRa opera con un'architettura completamente diversa rispetto alle reti 5G RAN, caratterizzandosi per una trasmissione a bassa potenza e lungo raggio, l'adozione diretta dei modelli di servizio E2SM standard di O-RAN non era possibile.

A differenza del 5G, dove la gestione delle risorse radio è completamente dinamica e ottimizzata attraverso meccanismi di closed-loop control come E2SM RC (RAN Control) e E2SM CCC (Cell Configuration and Control), la rete LoRa utilizza una configurazione più statica, con parametri di trasmissione regolati principalmente tramite il LoRaWAN Adaptive Data Rate (ADR) o impostati manualmente a livello di LoRa Gateway. Questa incompatibilità ha reso necessario lo sviluppo di E2SM-LoRa, un modello di servizio personalizzato che permette al Near-RT RIC di monitorare la rete

LoRa e modificarne dinamicamente i parametri operativi, adattando la trasmissione dei dispositivi alle condizioni ambientali e ai requisiti di rete.

Per consentire questa interazione, è stato implementato un E2 Term personalizzato, che funge da intermediario tra l'infrastruttura O-RAN e la rete LoRa, traducendo i messaggi E2 in un formato compatibile con la comunicazione LoRa. Senza questo elemento, l'integrazione tra LoRa e O-RAN non sarebbe stata possibile, poiché i dispositivi LoRa non supportano nativamente l'interfaccia E2 e comunicano tipicamente attraverso un LoRa Network Server (LNS). L'implementazione di E2SM-LoRa ha permesso di bypassare la necessità di un LNS, stabilendo una comunicazione diretta tra il LoRa Gateway e l'E2 Term, con il protocollo UDP come meccanismo di trasporto, garantendo così una comunicazione leggera e a bassa latenza.

Un ulteriore motivo per cui è stato necessario sviluppare E2SM-LoRa riguarda il monitoraggio delle prestazioni della rete LoRa. Gli E2 Service Model standard, come E2SM KPM (Key Performance Measurements), sono progettati per raccogliere metriche di rete basate su indicatori tipici delle reti cellulari, come throughput, latenza e utilizzo della banda. Tuttavia, questi parametri non sono direttamente applicabili a LoRa, che utilizza metriche specifiche come RSSI (Received Signal Strength Indicator), SNR (Signal-to-Noise Ratio), Packet Delivery Ratio (PDR) e Time-on-Air. Per garantire un monitoraggio efficace della rete LoRa, è stato necessario progettare un meccanismo di Subscription & Reporting personalizzato, in grado di raccogliere ed elaborare informazioni pertinenti, trasmettendole al Near-RT RIC in un formato standardizzato per l'infrastruttura O-RAN.

E2SM-LoRa è stato inoltre progettato per introdurre un meccanismo di configurazione centralizzato, che consente al Near-RT RIC di controllare direttamente i dispositivi LoRa attraverso il LoRa Gateway, senza la necessità di interventi manuali. Mentre nelle implementazioni LoRaWAN tradizionali, la configurazione dei dispositivi è statica o aggiornata tramite il meccanismo ADR, con E2SM-LoRa diventa possibile modificare frequenza operativa, potenza di trasmissione e spreading factor (SF) in tempo reale, ottimizzando così l'uso delle risorse radio e migliorando le prestazioni complessive della rete.

Per garantire la compatibilità con l'architettura O-RAN, E2SM-LoRa è stato definito in ASN.1, adottando la stessa metodologia strutturale degli altri modelli di servizio E2SM. Grazie a questa scelta di design, il modello è scalabile e interoperabile, permettendo di espandere il sistema per supportare altre tecnologie LPWAN come NB-IoT e Sigfox. La modularità dell'implementazione consente di adattare E2SM-LoRa a nuovi scenari di rete, garantendo una gestione unificata e ottimizzata per diverse tipologie di dispositivi IoT.

L'integrazione di E2SM-LoRa nell'infrastruttura O-RAN rappresenta un progresso significativo nell'unificazione delle reti LPWAN con il 5G, permettendo al Near-RT RIC di raccogliere dati di rete dettagliati e di inviare comandi di configurazione in tempo reale. Questa architettura consente di superare i limiti imposti dall'architettura tradizionale di LoRaWAN, offrendo nuove opportunità per la gestione avanzata delle reti IoT e per l'ottimizzazione delle risorse radio.

Grazie all'implementazione di E2SM-LoRa, la rete LoRa può ora essere gestita con gli stessi principi di automazione e ottimizzazione delle reti 5G, garantendo un maggiore controllo sulle prestazioni della rete, la qualità della connessione e l'efficienza energetica. Il modello di servizio sviluppato non solo permette una gestione più avanzata della rete LoRa, ma crea anche una base solida per l'integrazione futura di altre tecnologie IoT all'interno dell'ecosistema O-RAN.

6 Implementazione

L'implementazione del sistema rappresenta la fase in cui le componenti teoriche e progettuali descritte nei capitoli precedenti vengono tradotte in soluzioni software e hardware operative. In questo capitolo vengono illustrate nel dettaglio le scelte tecniche adottate, le configurazioni necessarie e le modalità di interazione tra i vari elementi del sistema, con l'obiettivo di realizzare un'integrazione efficace tra l'architettura O-RAN e la rete LoRa.

L'infrastruttura implementata si compone di diverse componenti software e hardware, che operano in sinergia per abilitare la gestione dinamica dei dispositivi LoRa attraverso il Near-RT RIC. La progettazione è stata sviluppata per garantire un'architettura scalabile, modulare e conforme agli standard definiti dalla O-RAN Alliance, permettendo l'interoperabilità tra il sistema LoRa e il controller O-RAN attraverso l'interfaccia E2.

Il sistema è stato implementato seguendo una struttura suddivisa in più livelli, ciascuno dei quali svolge un ruolo specifico nella gestione della rete LoRa e nella sua integrazione con l'architettura O-RAN:

- **Hardware e Configurazione del LoRa Gateway e degli End-Devices:** La piattaforma hardware è basata su Raspberry Pi equipaggiati con moduli LoRa SX1262, configurati per ricevere e applicare i parametri operativi inviati dal Near-RT RIC.
- **Cluster RIC Control Plane:** Il Near-RT RIC è stato installato in un ambiente Kubernetes, con l'utilizzo del framework O-RAN Software Community per il deployment dei componenti di gestione.
- **Framework xApp e AppMgr:** La xApp, implementata utilizzando il framework ric-plt-xapp-frame-py, è responsabile della gestione dinamica della rete LoRa, permettendo l'invio di richieste di configurazione e la raccolta di metriche in tempo reale.
- **E2 Term Custom:** Il modulo software che agisce come intermediario tra il Near-RT RIC e il LoRa Gateway, traducendo i messaggi E2AP in un formato compatibile con la rete LoRa e garantendo il corretto instradamento delle configurazioni e delle metriche raccolte.
- **LoRa Gateway e End-Devices:** Componenti chiave per la trasmissione e ricezione dei dati, responsabili dell'applicazione delle configurazioni ricevute dall'E2 Term e dell'inoltro delle metriche di rete verso il Near-RT RIC.
- **Definizione del Service Model E2SM-LoRa:** La specifica del modello di servizio ASN.1, necessario per definire i messaggi di controllo e monitoraggio della rete LoRa all'interno dell'ecosistema O-RAN.

L'obiettivo di questa implementazione è garantire un'integrazione tra O-RAN e LoRa, consentendo al Near-RT RIC di ottimizzare la rete LoRa. Nei paragrafi successivi verranno analizzati nel dettaglio i componenti software e hardware utilizzati, le configurazioni necessarie per il loro corretto funzionamento e i meccanismi di comunicazione implementati tra le diverse entità del sistema.

6.1 Hardware Utilizzato

Per la realizzazione del LoRa Gateway e dei LoRa End-Devices, è stato scelto l'utilizzo di Raspberry Pi, mostrato nella Figura 6.1 entrambi equipaggiati con il modulo SX1262 868M LoRa HAT [25]. Questo modulo è progettato per supportare la comunicazione LoRa nella banda 868 MHz, largamente impiegata in Europa per reti LoRaWAN, e offre un controllo preciso sui parametri di trasmissione.

L'integrazione del SX1262 868M LoRa HAT con Raspberry Pi consente di regolare la potenza di trasmissione fino a 22 dBm e di impostare uno spreading factor variabile tra 7 e 12, permettendo di bilanciare in modo ottimale la copertura del segnale e la velocità di trasmissione dei dati. Grazie alla flessibilità offerta da questa configurazione, è possibile adattare il comportamento della rete alle specifiche esigenze applicative, migliorando l'affidabilità e l'efficienza della comunicazione.

Questa soluzione garantisce elevate prestazioni sia per il gateway, responsabile della gestione della rete e dell'instradamento dei pacchetti, sia per il dispositivo finale, che invia e riceve dati nell'ambito dell'applicazione IoT. L'uso di Raspberry Pi offre inoltre una maggiore configurabilità rispetto ad altri dispositivi embedded, consentendo di sviluppare e testare algoritmi avanzati per la gestione della rete, l'ottimizzazione dei parametri di comunicazione e il risparmio energetico.



Figura 6.1 Raspberry Pi con modulo SX1262 868M LoRa HAT utilizzato per il LoRa Gateway e il LoRa End-Device [25]

6.2 Configurazione del Cluster RIC Control Plane

L'implementazione del RIC Control Plane in un ambiente controllato richiede la creazione di un cluster Kubernetes, che rappresenta il nucleo dell'architettura O-RAN. Questo controllo centrale fornisce le funzionalità necessarie per orchestrare le xApps, gestire le risorse di rete e abilitare la comunicazione tra i vari componenti dell'infrastruttura O-RAN.

Per la configurazione del cluster è stato utilizzato Kind (Kubernetes IN Docker), una soluzione che consente di eseguire Kubernetes all'interno di container Docker, garantendo un ambiente leggero, facilmente replicabile e adatto allo sviluppo e al testing. Questa scelta permette di emulare un'infrastruttura di rete realistica pur mantenendo la flessibilità necessaria per il debugging e l'ottimizzazione del sistema.

Il repository ufficiale della O-RAN Software Community [26] fornisce gli strumenti necessari per il deployment del RIC Control Plane. Il processo di installazione è stato avviato clonando il repository:

```
git clone https://github.com/o-ran-sc/ric-plt-ric-dep.git
cd ric-plt-ric-dep/new-installer
```

Successivamente, è stato creato il cluster Kind, specificando il nome ric-cluster per identificare chiaramente l'ambiente Kubernetes dedicato al RIC Control Plane:

```
kind create cluster --name ric-cluster
```

Questa configurazione permette di avviare il RIC Control Plane in un ambiente containerizzato, fornendo una base stabile per l'integrazione delle xApp e dei moduli di gestione della rete LoRa. Il cluster Kubernetes così configurato consente inoltre di replicare scenari di rete reali, facilitando l'ottimizzazione e il debugging delle interazioni tra il Near-RT RIC, l'E2 Term, il LoRa Gateway e gli altri componenti dell'infrastruttura O-RAN.

6.3 Installazione del framework AppMgr

L'Application Manager (AppMgr) è un componente essenziale per la gestione delle xApps (e delle rApps) all'interno del Near-RT RIC. Questo framework semplifica l'onboarding, la distribuzione, l'aggiornamento e la gestione delle applicazioni nel cluster Kubernetes, garantendo un flusso di lavoro automatizzato e scalabile per l'integrazione delle xApps nell'architettura O-RAN.

L'installazione del framework AppMgr inizia con la clonazione del repository ufficiale della O-RAN Software Community, che contiene gli strumenti necessari per l'orchestrazione delle xApps:

```
git clone https://gerrit.o-ran-sc.org/r/ric-plt/appmgr
cd appmgr/xapp_orchestrator/dev/xapp_onboarder
```

Una volta posizionati nella directory corretta, è possibile eseguire il processo di onboarding delle xApps, specificando il percorso del file di configurazione e dello schema JSON:

```
CHART_REPO_URL=http://localhost:6873 dms_cli onboard --config-file-path
<path-to-app-config> --shcema_file_path <path-to-schema-json>
```

Dopo aver completato il processo di onboarding, si procede con il download del chart Helm necessario per la distribuzione dell'xApp nel cluster Kubernetes:

```
CHART_REPO_URL=http://localhost:6873 dms_cli download_helm_chart <chart-name><version>
```

Infine, l'xApp viene installata nel namespace ricxapp del cluster utilizzando Helm:

```
helm install <release-name> -n ricxapp <chart-name-chart-version.tgz>
```

Grazie a questa configurazione, il framework AppMgr consente di orchestrare in modo efficace le xApps all'interno del Near-RT RIC, facilitando la gestione del ciclo di vita delle applicazioni e garantendo una distribuzione dinamica e scalabile all'interno dell'architettura O-RAN.

6.4 Configurazione del framework xApp

Il framework ric-plt-xapp-frame-py [27], sviluppato nell'ambito del progetto O-RAN, è una libreria Python progettata per facilitare lo sviluppo di xApps destinate al Near-RT RIC. Questo framework fornisce una struttura modulare che semplifica la gestione delle applicazioni all'interno dell'architettura O-RAN, garantendo una comunicazione efficiente tramite il protocollo RMR (RIC Message Routing). L'obiettivo principale di questo framework è semplificare l'implementazione delle xApps, gestendo automaticamente la configurazione di base, la comunicazione con il RIC e il routing dei messaggi. Per iniziare l'integrazione della xApp, è necessario clonare il repository ufficiale del framework e configurare l'ambiente di sviluppo affinché possa interagire correttamente con il Near-RT RIC.

```
git clone https://github.com/o-ran-sc/ric-plt-xapp-frame-py
```

6.4.1 Implementazione xApp

Il primo passo nell'implementazione della xApp consiste nell'inizializzazione del contesto di comunicazione, un'operazione essenziale affinché l'applicazione possa interagire con gli altri elementi del sistema. Come si può notare dalla Figura 6.2, la configurazione della comunicazione è gestita dal metodo `initialize_rmr(self)`, che avvia il RIC Message Router (RMR) e stabilisce il contesto di messaggistica, verificando che sia pronto per ricevere e inviare messaggi. L'operazione viene eseguita attraverso l'istanza del contesto RMR utilizzando il comando `rmr.rmr_init()`, specificando la porta su cui la comunicazione avrà luogo. Poiché il contesto di messaggistica potrebbe non essere immediatamente disponibile, il metodo prevede una verifica ciclica con `rmr.rmr_ready(self.rmr_ctx)`, attendendo finché l'ambiente non diventa completamente operativo. Solo dopo questa fase, la xApp è in grado di inviare configurazioni ai dispositivi LoRa e ricevere metriche di rete.

```
def initialize_rmr(self):
    """Inizializza il contesto RMR."""
    logging.info("Inizializzazione del contesto RMR...")
    self.rmr_ctx = rmr.rmr_init(str(self.rmr_port).encode(), rmr.RMR_MAX_RCV_BYTES, rmr.RMRFL_NONE)
    if not self.rmr_ctx:
        raise RuntimeError("Errore nell'inizializzazione del contesto RMR.")
    while rmr.rmr_ready(self.rmr_ctx) == 0:
        logging.info("RMR non ancora pronto. Aspettando...")
        time.sleep(1)
    logging.info("RMR pronto.")
```

Figura 6.2 Inizializzazione del contesto RMR nella xApp

Il processo di trasmissione del Control Request si suddivide in due fasi fondamentali: la generazione del payload contenente le informazioni di configurazione e la successiva trasmissione del messaggio attraverso l'E2 Term.

```
def create_e2sm_message(self):
    config = self.load_config()
    return self.e2sm_schema.encode("LoRaConfigControlRequest", {
        "loraConfigHeader": {
            "loraRequestID": 1234,
            "functionID": 1,
            "configType": "configuration",
        },
        "loraConfigMessage": {
            "freq": config["freq"],
            "power": config["power"],
            "addr": config["addr"],
            "sf": config["sf"],
        },
    })
```

Figura 6.3 Generazione del messaggio di configurazione E2SM-LoRa

La generazione del payload avviene nel metodo `create_e2sm_message(self)`, come mostrato nella Figura 6.3, che costruisce un messaggio conforme al formato E2SM-LoRa. Questo formato è stato appositamente sviluppato per garantire la compatibilità con l'architettura O-RAN, fornendo una struttura chiara e standardizzata per la trasmissione delle configurazioni. All'interno del messaggio vengono definiti parametri chiave che determinano il comportamento del dispositivo LoRa, tra cui la frequenza operativa, espressa in MHz, che stabilisce il canale di comunicazione da utilizzare, la potenza di trasmissione, espressa in dBm, che influenza la portata del segnale, l'indirizzo del dispositivo LoRa,

che identifica univocamente il nodo all'interno della rete, e il fattore di spreading (SF), un parametro che regola il compromesso tra distanza di copertura e velocità di trasmissione. Valori più elevati di SF consentono di estendere la copertura a distanze maggiori, riducendo però il throughput disponibile.

Il metodo `create_e2sm_message(self)` recupera i valori di configurazione dalle variabili d'ambiente attraverso la chiamata `self.load_config()`. Una volta ottenuti i valori necessari, il metodo li organizza in una struttura dati conforme al protocollo E2AP, assicurando che il messaggio rispetti le specifiche O-RAN. Questo processo facilita l'interpretazione del messaggio da parte degli elementi della rete, garantendo un'interoperabilità efficace all'interno dell'architettura.

Dopo aver generato il payload con le informazioni di configurazione, la xApp procede con la costruzione del Control Request, operazione svolta all'interno del metodo `create_control_request(self)` come si può osservare dalla Figura 6.4. Questo metodo crea un messaggio di controllo che include, oltre al payload precedentemente generato, un'intestazione denominata `ricControlHeader`, contenente informazioni essenziali per identificare il messaggio. Tra queste informazioni figurano il `ricRequestorID`, che identifica la richiesta all'interno del sistema, il `ranFunctionID`, che specifica la funzione di rete a cui è destinato il messaggio, e il `ricControlType`, che indica la tipologia di comando da eseguire. Il messaggio completo, costituito dall'intestazione e dal payload, viene poi codificato utilizzando

`self.e2ap_schema.encode("RICControlRequestMessage", control_request)`, assicurando che il formato rispetti gli standard previsti per la comunicazione con l'E2 Term.

```
def create_control_request(self):
    """Crea un messaggio RICControlRequest."""
    control_request = {
        "ricControlHeader": {
            "ricRequestorID": 1234,
            "ranFunctionID": 1,
            "ricControlType": 0,
        },
        "ricControlMessage": self.create_e2sm_message(),
    }
    return self.e2ap_schema.encode("RICControlRequestMessage", control_request)
```

Figura 6.4 Creazione del messaggio di controllo `RICControlRequest`

Una volta codificato il messaggio di Control Request, questo viene trasmesso all'E2 Term, che provvede a inoltrarlo al LoRa Gateway affinché la nuova configurazione venga applicata al dispositivo LoRa di destinazione. Se il processo si conclude con successo, il dispositivo aggiorna i propri parametri e opera secondo le nuove impostazioni, mentre l'E2 Term restituisce un messaggio di conferma, chiamato Control Acknowledge, che viene successivamente analizzato dalla xApp per verificare l'esito dell'operazione.

Una volta generato il payload contenente la configurazione desiderata, la xApp costruisce il messaggio di Control Request all'interno del metodo `send_control_request(self)`, mostrato nella figura 6.5. Qui viene definita la struttura del messaggio, che include un'intestazione con informazioni di identificazione e il payload generato in precedenza. Il messaggio viene poi codificato utilizzando `self.e2ap_schema.encode()`, che lo trasforma in un formato compatibile con la trasmissione RMR. La xApp alloca quindi un buffer di messaggistica RMR, nel quale inserisce il messaggio codificato, e assegna un identificativo di tipo per far sì che l'E2 Term possa riconoscerlo correttamente. A questo punto, il messaggio viene inviato utilizzando il metodo `rmr.rmr_send_msg()`, che gestisce l'effettiva trasmissione del Control Request attraverso il sistema RMR.

```

def send_control_request(self):
    try:
        # Crea il payload della configurazione (E2SM message)
        config_payload = self.create_e2sm_message()
        # Costruisce il messaggio di Control Request includendo il payload con la configurazione
        control_request = {
            "ricControlHeader": {
                "ricRequestorID": 1234,
                "ranFunctionID": 1,
                "ricControlType": 0,
            },
            "ricControlMessage": config_payload,
        }
        encoded_request = self.e2ap_schema.encode("RICControlRequestMessage", control_request)
        sbuf = rmr.rmr_alloc_msg(self.rmr_ctx, len(encoded_request))
        if not sbuf:
            logging.error("Errore: Impossibile allocare il messaggio RMR per il Control Request.")
            return (False, config_payload)
        payload_ptr = cast(sbuf[0].payload, POINTER(c_char * len(encoded_request)))
        for i, byte in enumerate(encoded_request):
            payload_ptr.contents[i] = byte
        sbuf.contents.mtype = 12050 # Control Request
        sbuf.contents.len = len(encoded_request)
        result = rmr.rmr_send_msg(self.rmr_ctx, sbuf)

        if result and result.contents.state == rmr.RMR_OK:
            logging.info(f"Control Request inviato con successo.")
            return True
        else:
            logging.error(f"Errore durante l'invio del Control Request. Stato: {result.contents.state}")
            return False
    finally:
        if sbuf:
            rmr.rmr_free_msg(sbuf)

```

Figura 6.5 Invio del Control Request tramite RMR

Dopo l'invio del messaggio, la xApp attende una risposta sotto forma di Control Acknowledge, un messaggio di conferma inviato dall'E2 Term che indica se l'operazione è stata eseguita correttamente o se si sono verificati errori. Se la risposta ricevuta conferma l'applicazione della configurazione, la xApp considera completata la procedura di aggiornamento; in caso contrario, registra l'errore nei log e può decidere di inviare nuovamente la richiesta o eseguire un'operazione di fallback.

Parallelamente alla configurazione dei dispositivi, la xApp deve anche monitorare lo stato della rete LoRa in tempo reale. A tal fine, invia una Subscription Request attraverso il metodo `create_subscription_request(self)`, mostrato nella Figura 6.6, specificando all'E2 Term quali metriche desidera ricevere.

L'uso del RIC Message Router (RMR) è cruciale per garantire un trasferimento di messaggi efficiente e affidabile. RMR offre una gestione automatica del routing dei messaggi, evitando la necessità di configurazioni manuali e migliorando la scalabilità del sistema. Inoltre, il supporto per la messaggistica asincrona consente alla xApp di inviare comandi e ricevere risposte senza bloccare il proprio flusso operativo, ottimizzando il tempo di esecuzione delle operazioni. Un altro aspetto fondamentale è la gestione degli errori, che permette al sistema di rilevare automaticamente eventuali problemi di trasmissione e di attivare meccanismi di retry per assicurare la consegna corretta dei messaggi.

```

def create_subscription_request(self):
    """Crea un messaggio RICSubscriptionRequest."""
    subscription_request = {
        "ricRequestID": {
            "ricRequestorID": 1234,
            "ricInstanceID": 1
        },
        "ranFunctionID": 1,
        "ricSubscriptionDetails": {
            "ricEventTriggerDefinition": b"event_trigger_payload",
            "ricActionToBeSetupList": [
                {
                    "ricActionID": 1,
                    "ricActionType": 0,
                    "ricActionDefinition": b"action_definition_payload",
                    "ricSubsequentAction": {
                        "ricSubsequentActionType": 0,
                        "ricTimeToWait": 1
                    }
                }
            ]
        }
    }
    return self.e2ap_schema.encode("RICSubscriptionRequest", subscription_request)

```

Figura 6.6 Creazione di un Subscription Request per la xApp

6.5 Implementazione E2TermCustom

L'E2 Term Custom è il componente responsabile dell'integrazione tra il Near-RT RIC e la rete LoRa, fungendo da intermediario per la trasmissione di comandi di controllo e la raccolta di metriche di rete. Poiché l'architettura O-RAN e LoRa utilizzano protocolli di comunicazione differenti, l'E2 Term si occupa di tradurre i messaggi E2AP in un formato compatibile con il protocollo LoRa e viceversa. Questa operazione garantisce la compatibilità tra i due sistemi e consente alla xApp di monitorare e gestire i dispositivi LoRa in modo efficiente.

L'implementazione dell'E2 Term Custom si basa su due meccanismi fondamentali che consentono alla xApp di interagire con la rete LoRa in modo efficiente e scalabile. Da un lato, il metodo *handle_subscription_request(self, sbuf)* gestisce le Subscription Requests, permettendo alla xApp di registrarsi per ricevere aggiornamenti periodici sulle metriche della rete LoRa. Questo processo assicura che le informazioni aggiornate vengano trasmesse automaticamente con la frequenza stabilita, riducendo il carico sulla rete e migliorando la reattività del sistema. Dall'altro, il metodo *handle_control_request(self, sbuf)* è responsabile della gestione dei Control Requests, consentendo alla xApp di inviare configurazioni specifiche ai dispositivi LoRa. Grazie a questo meccanismo, la xApp può adattare dinamicamente i parametri di trasmissione e comunicazione dei dispositivi, garantendo un'ottimizzazione continua in base alle condizioni ambientali e ai requisiti operativi.

Entrambi i meccanismi vengono orchestrati dal metodo *run(self)* come mostrato in Figura 6.7, che rappresenta il ciclo operativo dell'E2 Term. Questo metodo rimane in ascolto dei messaggi in arrivo tramite il RIC Message Router (RMR), instradandoli ai rispettivi gestori a seconda del tipo di richiesta ricevuta. In particolare, quando viene intercettata una Subscription Request, il messaggio viene inoltrato a *handle_subscription_request(self, sbuf)*, mentre nel caso di una Control Request, la gestione viene affidata a *handle_control_request(self, sbuf)*. Questo approccio garantisce un'elaborazione efficiente e in tempo reale delle richieste, permettendo alla rete LoRa di essere gestita dinamicamente attraverso l'infrastruttura O-RAN.

```

def run(self):
    """Avvia il ciclo di ricezione dei messaggi."""
    while self.running:
        try:
            sbuf = rmr.rmr_rcv_msg(self.rmr_ctx, None)
            if sbuf and sbuf.contents.state == rmr.RMR_OK:
                if sbuf.contents.mtype == 12000: # Subscription Request
                    self.handle_subscription_request(sbuf)
                elif sbuf.contents.mtype == 12050: # Control Request
                    self.handle_control_request(sbuf)
        except Exception as e:
            logging.error(f"Errore nel ciclo principale: {e}")
            time.sleep(0.1)

```

Figura 6.7 Loop di ascolto e gestione dei messaggi RMR nell'E2 Term

Quando la xApp desidera ricevere informazioni periodiche sulla qualità della comunicazione LoRa, invia un Subscription Request, che viene elaborato dall'E2 Term attraverso il metodo `handle_subscription_request(self, sbuf)`. Questo metodo gestisce l'intera procedura, dalla decodifica del messaggio alla registrazione della richiesta per la raccolta dati. Se tutto il processo si conclude correttamente, la xApp può considerare attivato il monitoraggio e iniziare a ricevere aggiornamenti periodici. In caso di errore, l'E2 Term registra un messaggio nei log, facilitando il debugging e la correzione del problema.

Come mostrato nella Figura 6.8, l'elaborazione della richiesta prevede l'estrazione del payload del messaggio, che viene eseguita attraverso `self.extract_payload(sbuf)`. Una volta isolati i dati effettivi della richiesta, il messaggio viene decodificato nel formato E2AP, utilizzando `self.e2ap_schema.decode("RICSubscriptionRequestMessage", payload_data)`. Questo passaggio è fondamentale per interpretare correttamente i parametri specificati dalla xApp, in modo da registrare la richiesta all'interno dell'E2 Term e avviare la procedura di raccolta dati.

```

def handle_subscription_request(self, sbuf):
    """Gestisce un Subscription Request."""
    try:
        payload_data = self.extract_payload(sbuf)
        decoded_request = self.e2ap_schema.decode("RICSubscriptionRequest", payload_data)
        logging.info(f"Subscription Request ricevuto: {decoded_request}")

        # Estrai request_id e ran_function_id
        request_id = decoded_request["ricRequestID"]
        ran_function_id = decoded_request["ranFunctionID"]

        # Invia una risposta al Subscription Request
        self.send_subscription_ack(sbuf, request_id, ran_function_id)
    except Exception as e:
        logging.error(f"Errore durante la gestione del Subscription Request: {e}")

```

Figura 6.8 Gestione della Subscription Request nell'E2 Term

Durante la decodifica, l'E2 Term estrae due parametri chiave: il Request ID, che identifica in modo univoco la richiesta di sottoscrizione, e il RAN Function ID, che specifica la funzione di rete coinvolta nella raccolta dei dati. Queste informazioni vengono salvate internamente per consentire all'E2 Term di associare le risposte alle richieste corrispondenti. Una volta registrata la richiesta, l'E2 Term avvia il processo di raccolta periodica delle metriche, comunicando con il LoRa Gateway per ottenere dati

aggiornati sulle condizioni della rete. Il LoRa Gateway è responsabile della misurazione di parametri critici.

Dopo aver ricevuto le metriche aggiornate dal LoRa Gateway, l'E2 Term provvede a inoltrarle alla xApp, garantendo che le informazioni siano trasmesse con la frequenza specificata nella Subscription Request. Questo sistema permette alla xApp di monitorare lo stato della rete LoRa in modo continuo ed efficiente, senza dover generare richieste ripetute, riducendo il carico sulla rete e migliorando l'efficienza del processo.

Una volta che la richiesta è stata elaborata correttamente, l'E2 Term invia una Subscription Response alla xApp, confermando l'avvenuta registrazione e indicando che le metriche richieste saranno fornite secondo la frequenza stabilita. Questo passaggio è cruciale per garantire alla xApp che il processo di monitoraggio sia stato attivato correttamente e che le informazioni verranno trasmesse in tempo reale senza necessità di ulteriori interventi. Per fare ciò, utilizza il metodo `self.send_subscription_ack(sbuf, request_id, ran_function_id)`, esposto nella Figura 6.9, che costruisce e trasmette un messaggio Subscription Acknowledge,

```
def send_subscription_ack(self, sbuf, request_id, ran_function_id):
    try:
        response_message = {
            "ricRequestID": request_id,
            "ranFunctionID": ran_function_id,
            "ricActionAdmittedList": [{"ricActionID": 1}],
            # Puoi aggiungere ricActionNotAdmittedList se necessario
        }
        encoded_response = self.e2ap_schema.encode("RICSubscriptionResponse", response_message)

        response_sbuf = rmr.rmr_alloc_msg(self.rmr_ctx, len(encoded_response))
        if not response_sbuf:
            logging.error("Errore: Impossibile allocare il messaggio RMR per il Subscription Response.")
            return

        payload_ptr = cast(response_sbuf[0].payload, POINTER(c_char * len(encoded_response)))
        for i, byte in enumerate(encoded_response):
            payload_ptr.contents[i] = byte

        response_sbuf.contents.mtype = 12001 # Tipo di messaggio RICSubscriptionResponse
        response_sbuf.contents.len = len(encoded_response)

        result = rmr.rmr_send_msg(self.rmr_ctx, response_sbuf)
        if result and result.contents.state == rmr.RMR_OK:
            logging.info("Subscription Response inviato con successo.")
        else:
            logging.error(f"Errore durante l'invio del Subscription Response. Stato: {result.contents.state if result else 'None'}")
    except Exception as e:
        logging.error(f"Errore durante l'invio del Subscription Response: {e}")
    finally:
        if response_sbuf:
            rmr.rmr_free_msg(response_sbuf)
```

Figura 6.9 Invio della Subscription Response nell'E2 Term

Grazie a questa struttura, il sistema di Subscription Requests consente alla rete LoRa di essere monitorata in modo continuo, garantendo al Near-RT RIC e alla xApp un accesso costante a metriche aggiornate, fondamentali per ottimizzare le prestazioni della comunicazione e migliorare l'affidabilità della rete.

Parallelamente alla gestione delle richieste di sottoscrizione, l'E2 Term è responsabile del trattamento dei Control Requests inviati dalla xApp. Quando riceve una richiesta di configurazione, utilizza *handle_control_request(self, sbuf)* come mostrato nella Figura 6.10, per estrarre i parametri di configurazione inviati dalla xApp. Dopo aver decodificato il messaggio, il sistema riformatta i dati in JSON, compatibile con il protocollo LoRa, e li trasmette al LoRa Gateway utilizzando UDP. Una volta applicata la configurazione, il LoRa Gateway invia una risposta di conferma, che viene elaborata dall'E2 Term e riconvertita in un messaggio E2AP, per poi essere trasmessa al Near-RT RIC, notificando alla xApp l'esito dell'operazione.

```
def handle_control_request(self, sbuf):
    """Gestisce un Control Request."""
    try:
        payload_data = self.extract_payload(sbuf)
        decoded_request = self.e2ap_schema.decode("RICControlRequestMessage", payload_data)
        e2sm_payload = decoded_request["ricControlMessage"]

        # Decodifica il messaggio E2SM
        decoded_e2sm = self.e2sm_schema.decode("LoRaConfigControlRequest", e2sm_payload)
        config = decoded_e2sm["loraConfigMessage"]
        logging.info(f"Configurazione ricevuta: {config}")

        # Inoltra la configurazione a LoRa1
        self.send_to_loraGateway(config)

        # Invia l'ACK
        self.send_control_ack(decoded_request["ricControlHeader"])
    except Exception as e:
        logging.error(f"Errore durante la gestione del Control Request: {e}")
```

Figura 6.10 Gestione del Control Request nell'E2 Term

Quando l'E2 Term riceve un messaggio RMR contenente una richiesta di configurazione proveniente dal Near-RT RIC, avvia il processo di decodifica e traduzione dei dati affinché possano essere compresi e applicati dai dispositivi LoRa. Poiché l'architettura O-RAN e la rete LoRa utilizzano protocolli differenti, è necessario un passaggio intermedio per garantire che le informazioni contenute nella richiesta siano trasformate in un formato compatibile con la tecnologia LoRa. La prima operazione eseguita dall'E2 Term è l'estrazione del payload del messaggio ricevuto, che viene gestita dal metodo *self.extract_payload(sbuf)*. Questo passaggio consente di isolare i dati contenuti nel messaggio RMR, eliminando eventuali informazioni di trasporto superflue.

Una volta ottenuto il payload, il messaggio viene decodificato utilizzando il metodo *self.e2ap_schema.decode("RICControlRequestMessage", payload_data)*, che permette di interpretare il contenuto della richiesta secondo il protocollo E2AP, lo standard utilizzato all'interno dell'architettura O-RAN per la comunicazione tra il Near-RT RIC e i nodi radio. All'interno del messaggio decodificato, l'E2 Term individua la sezione denominata *ricControlMessage*, che contiene i parametri operativi da applicare al dispositivo LoRa. Questi parametri includono: Frequenza operativa (MHz), Potenza di trasmissione (dBm), Spreading Factor (SF), Indirizzo del dispositivo LoRa.

Dopo aver estratto i parametri, l'E2 Term procede alla conversione dei dati in un formato comprensibile per il LoRa Gateway. Poiché la comunicazione con i dispositivi LoRa non avviene tramite l'interfaccia E2, ma piuttosto attraverso protocolli più tradizionali, l'E2 Term riformatta il messaggio in un JSON strutturato e lo trasmette al LoRa Gateway utilizzando il protocollo UDP. Questo passaggio è cruciale, in quanto consente al Near-RT RIC di comunicare con dispositivi che originariamente non erano stati progettati per integrarsi con l'architettura O-RAN.

```
def send_to_loraGateway(self, config):
    """Invia la configurazione a LoRa1 tramite UDP."""
    try:
        payload = json.dumps(config).encode("utf-8")
        self.udp_socket.sendto(payload, (self.lora1_ip, self.lora1_port))
        logging.info(f"Configurazione inviata a LoRa1 ({self.lora1_ip}:{self.lora1_port}): {config}")
    except Exception as e:
        logging.error(f"Errore nell'invio della configurazione a LoRa1: {e}")
```

Figura 6.11 Invio della Configurazione al LoRa Gateway tramite UDP

Come mostrato nella Figura 6.11, l'invio del messaggio JSON avviene attraverso il metodo `send_to_loraGateway(config)`, che organizza i dati in una struttura JSON conforme alle specifiche del LoRa Gateway. Una volta generato il messaggio nel formato corretto, il metodo utilizza un socket UDP per trasmetterlo al gateway, garantendo una comunicazione a bassa latenza e con un overhead minimo. La scelta di utilizzare il protocollo UDP (User Datagram Protocol) per la comunicazione tra E2 Term e LoRa Gateway è stata dettata da diversi fattori. Bassa latenza: UDP è un protocollo connectionless, che non introduce ritardi dovuti alla gestione di connessioni persistenti. Overhead ridotto: A differenza di TCP, UDP non utilizza meccanismi di controllo del flusso e ritrasmissione, riducendo il carico di rete. Compatibilità con la natura di LoRa: Le trasmissioni LoRa sono di tipo best-effort, quindi l'utilizzo di un protocollo leggero come UDP è più appropriato rispetto a protocolli più complessi come MQTT o HTTP.

6.6 Implementazione LoRaGateway

L'implementazione software del LoRa Gateway è stata sviluppata in Python e si avvale di diversi componenti fondamentali:

- **Socket UDP:** Utilizzati per la ricezione e l'invio di messaggi in formato JSON, garantendo così una comunicazione rapida e a bassa latenza.
- **Librerie LoRa (SX1262):** Permettono di interfacciarsi con l'hardware del modulo LoRa, facilitando la configurazione dei dispositivi.
- **Gestione degli Errori e Logging:** Queste funzionalità sono essenziali per assicurare l'affidabilità dell'intero sistema, consentendo di monitorare le operazioni e di effettuare il debugging in caso di problemi.

Per garantire la gestione dinamica della rete, il LoRa Gateway rimane costantemente in ascolto sulla porta UDP predefinita, pronto a ricevere messaggi di configurazione inviati dall'E2 Term. Quando una xApp, in esecuzione nel Near-RT RIC, decide di modificare i parametri operativi di un nodo LoRa, invia un Control Request all'E2 Term, il quale si occupa di tradurlo in un messaggio JSON compatibile con la rete LoRa e di trasmetterlo al LoRa Gateway utilizzando il protocollo UDP. Il messaggio ricevuto contiene i parametri da applicare al dispositivo, come la frequenza operativa, la potenza di trasmissione, il fattore di spreading (SF) e l'indirizzo del nodo destinatario.

Una volta ricevuto un pacchetto di configurazione, il LoRa Gateway avvia un flusso di elaborazione strutturato per garantire la corretta applicazione dei nuovi parametri. Questa operazione è orchestrata dal metodo `server()`, come mostrato nella Figura 6.12, il quale rappresenta il cuore del sistema di gestione delle configurazioni. Il metodo inizializza il server UDP e lo pone in stato di ascolto continuo sulla porta assegnata. Non appena un messaggio arriva dall'E2 Term, il gateway lo acquisisce e lo decodifica dal formato JSON, estraendo i parametri da applicare al dispositivo LoRa. Questa fase è critica, poiché un'eventuale anomalia nel formato del messaggio potrebbe compromettere la trasmissione della configurazione. Per questo motivo, il LoRa Gateway esegue una verifica della validità del messaggio e, in caso di errori, genera una risposta negativa da inviare all'E2 Term, evitando così che configurazioni corrotte vengano applicate ai dispositivi.

```

def serve():
    # Inizializza il modulo LoRa (gateway)
    lora_device = lora.sx126x()
    service = LoraConfigService(lora_device)

    # Configura il server UDP per ricevere le configurazioni dal E2Term
    server_address = ("", 50052)
    udp_server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    udp_server.bind(server_address)

    logging.info("LoRa UDP Server (gateway) in esecuzione su porta 50052...")

    try:
        while True:
            data, client_address = udp_server.recvfrom(1024)
            logging.info(f"UDP packet ricevuto da {client_address}")
            try:
                config_data = json.loads(data.decode('utf-8'))
                response = service.forward_configuration(config_data)

                logging.info(f"Configurazione inoltrata")
            except json.JSONDecodeError as e:
                response = {"status": f"error: Invalid JSON - {str(e)}"}

            response_data = json.dumps(response).encode('utf-8')
            udp_server.sendto(response_data, client_address)

    except KeyboardInterrupt:
        logging.info("Server UDP interrotto.")
    finally:
        udp_server.close()

```

Figura 6.12 Gestione del Server UDP nel LoRa Gateway

Dopo aver ricevuto il messaggio di configurazione dall'E2 Term, il LoRa Gateway avvia il processo di elaborazione della richiesta e di aggiornamento dei parametri del dispositivo LoRa corrispondente. Questa fase è fondamentale perché rappresenta il momento in cui le nuove impostazioni, definite dal Near-RT RIC, vengono effettivamente applicate ai nodi della rete LoRa. Il gateway, grazie alla sua interfaccia hardware compatibile con la tecnologia LoRa, è in grado di convertire i parametri ricevuti in comandi eseguibili dal dispositivo e di trasmetterli in modalità wireless.

Il primo passo in questo processo è gestito dal metodo *forward_configuration()*, il quale riceve i dati precedentemente decodificati e li prepara per l'invio al dispositivo LoRa, introducendo un breve ritardo di sincronizzazione, solitamente inferiore al secondo, per garantire che il dispositivo LoRa sia pronto a ricevere e applicare i nuovi parametri.

A questo punto, il metodo *send_to_loraED()* entra in azione per eseguire la trasmissione vera e propria della configurazione, come mostrato nella Figura 6.13. Il messaggio ricevuto dall'E2 Term viene prima trasformato in un payload compatibile con il modulo LoRa SX1262, utilizzato dal LoRa Gateway per la comunicazione con i dispositivi della rete. Una volta che il messaggio è pronto, *send_to_loraED()* lo invia tramite interfaccia radio, assicurandosi che il dispositivo riceva e interpreti correttamente i nuovi parametri.

```

def forward_configuration(self, config_data):
    try:
        time.sleep(0.05)
        self.send_to_loraED(config_data) # Inoltrare la configurazione al dispositivo LoRa
        logging.info(f"Configurazione inoltrata a loraED ")
        return {"status": "success"}
    except Exception as e:
        return {"status": f"error: {str(e)}"}

def send_to_loraED(self, config_data):
    try:
        payload = json.dumps(config_data).encode("utf-8") + b"<END>"
        self.lora_device.send(payload)
        logging.info(f"Configurazione inoltrata a loraED: {config_data}")
    except Exception as e:
        logging.error(f"Errore nell'invio della configurazione a loraED: {e}")

```

Figura 6.13 Inoltro e Trasmissione della Configurazione ai Dispositivi LoRa

6.7 Implementazione LoRa End-Device

La comunicazione tra i dispositivi LoRa End-Device e il Near-RT RIC avviene attraverso una serie di passaggi ben definiti che garantiscono la corretta applicazione delle configurazioni, la trasmissione dei dati di telemetria e l'adattamento dinamico dei parametri operativi in base alle condizioni della rete. Questo processo consente di ottimizzare le prestazioni della rete LoRa, garantendo un equilibrio tra efficienza energetica e qualità della connessione.

Il primo passaggio fondamentale è la ricezione della configurazione dal LoRa Gateway. Il dispositivo LoRa End-Device rimane costantemente in ascolto per intercettare eventuali aggiornamenti inviati dal gateway attraverso il modulo LoRa SX1262. Come mostrato nella Figura 6.14, questa operazione è gestita dal metodo *listen(self)*, che avvia un ciclo di ricezione continua dei messaggi provenienti dal LoRa Gateway. Quando un messaggio viene ricevuto, il metodo lo registra nei log di sistema, permettendo al dispositivo di elaborare la configurazione ricevuta e applicare le nuove impostazioni operative.

Il messaggio ricevuto contiene i parametri operativi aggiornati, tra cui la frequenza operativa, la potenza di trasmissione e lo Spreading Factor (SF). Questi parametri sono stati precedentemente determinati dal Near-RT RIC, che li ha calcolati in base alle condizioni attuali della rete e ai requisiti operativi stabiliti dalla xApp. Dopo la ricezione, il LoRa End-Device esegue l'aggiornamento della propria configurazione interna, adattandosi alle nuove condizioni operative e garantendo un funzionamento ottimale della rete.

```

def listen(self):
    logging.info("Lora0 in ascolto di messaggi LoRa...")
    while True:
        message = self.lora_device.receive()
        if message:
            logging.info(f"Messaggio ricevuto da LoraGateway: {message}")

```

Figura 6.14 Ricezione dei messaggi da LoRa Gateway nel LoRa End-Device

6.8 Implementazione dell'E2SM Personalizzato

Come mostrato nella Figura 6.15, l'E2SM-LoRa è stato definito utilizzando il linguaggio ASN.1, lo standard per i modelli di servizio in O-RAN, per abilitare la comunicazione tra il Near-RT RIC e la rete LoRa, consentendo il monitoraggio e la configurazione dinamica dei dispositivi LoRa attraverso l'interfaccia E2. Il modello è composto da due componenti principali:

- **LoRaConfigControlRequest**: definisce il messaggio utilizzato per inviare configurazioni ai dispositivi LoRa. Questo messaggio include un'intestazione contenente informazioni di identificazione e un payload con i parametri operativi da applicare ai dispositivi.
- **LoRaConfigControlAcknowledge**: è il messaggio di conferma inviato in risposta a un Control Request. Serve a notificare al Near-RT RIC l'esito dell'applicazione della configurazione, specificando se l'operazione ha avuto successo o meno.

```
E2SM-LoRa-Definitions DEFINITIONS AUTOMATIC TAGS ::= BEGIN

-- Top-level messages for LoRa configuration
LoRaConfigControlRequest ::= SEQUENCE {
    loraConfigHeader      LoRaConfigHeader,
    loraConfigMessage      LoRaConfigMessage,
    callProcessID          OCTET STRING OPTIONAL
}

-- Header for the LoRa configuration message
LoRaConfigControlAcknowledge ::= SEQUENCE {
    loraRequestID          INTEGER (0..65535),
    outcome                 ENUMERATED { success(0), failure(1) }
}

LoRaConfigHeader ::= SEQUENCE {
    loraRequestID          INTEGER (0..65535),
    functionID             INTEGER (0..65535),
    configType             ENUMERATED { configuration(0), update(1) }
}

-- Message containing the LoRa configuration
LoRaConfigMessage ::= SEQUENCE {
    freq                   INTEGER (840..950), -- Frequenza in MHz
    addr                   INTEGER (0..255),   -- Indirizzo del dispositivo
    power                  INTEGER (0..30),    -- Potenza in dBm
    sf                     INTEGER (7..12)     -- Spreading Factor
}

END
```

Figura 6.15 Definizione ASN.1 del Service Model E2SM-LoRa

L'intestazione del messaggio, rappresentata da **LoRaConfigHeader**, include il campo **loraRequestID**, che identifica univocamente la richiesta, **functionID**, che specifica la funzione di rete coinvolta, e **configType**, che distingue tra una nuova configurazione e un aggiornamento di una configurazione esistente.

Il payload di configurazione, definito in `LoRaConfigMessage`, contiene i parametri operativi essenziali per i dispositivi LoRa, ovvero:

- `freq`: la frequenza operativa in MHz, variabile tra 840 e 950 MHz.
- `addr`: l'indirizzo del dispositivo LoRa, identificato da un valore numerico compreso tra 0 e 255.
- `power`: la potenza di trasmissione espressa in dBm, regolabile tra 0 e 30 dBm per adattare il consumo energetico e la copertura del segnale.
- `sf`: il Spreading Factor, un parametro che incide sulla portata del segnale e la velocità di trasmissione, con valori compresi tra 7 e 12.

Grazie a questa struttura, il modello E2SM-LoRa consente una gestione efficiente della configurazione dei dispositivi LoRa, permettendo al Near-RT RIC di inviare istruzioni precise e ricevere conferme affidabili sull'esito dell'operazione. L'uso del linguaggio ASN.1 garantisce inoltre un'integrazione coerente con il framework O-RAN, facilitando l'implementazione del modello all'interno dell'infrastruttura di rete esistente.

6.9 xApp Deployment: onboarding

L'onboarding della xApp nel Near-RT RIC viene effettuato utilizzando `dms_cli` in combinazione con Helm, garantendo una gestione strutturata e controllata del deployment. Questo processo assicura che la xApp venga registrata, validata e distribuita correttamente all'interno di un cluster Kubernetes, seguendo le specifiche definite per l'ecosistema O-RAN.

L'onboarding si basa sui file `descriptor.json` e `schema.json`, che definiscono rispettivamente i dettagli della xApp e la struttura della sua configurazione.

- Il file `descriptor.json` contiene informazioni essenziali come il nome della xApp, la versione, l'immagine Docker utilizzata, le risorse assegnate e le porte di comunicazione.
- Il file `schema.json` definisce le regole di validazione per i parametri di configurazione della xApp, la struttura dei dati scambiati e i campi necessari per la codifica e decodifica dei messaggi tra E2 Termination (E2T) e Near-RT RIC. Questo assicura che la xApp possa interagire correttamente con il sistema O-RAN rispettando le specifiche dell'interfaccia E2AP e E2SM-LoRa.

L'onboarding della xApp avviene attraverso `dms_cli`, che si occupa della registrazione e della gestione dei pacchetti Helm. Il processo segue questi passaggi:

- **Registrazione della xApp:** La xApp viene registrata nel Near-RT RIC tramite `dms_cli onboard`, specificando il file `descriptor.json` e il file `schema.json` per la validazione della configurazione:
`CHART_REPO_URL=http://localhost:6873 dms_cli onboard --config-file-path <path-to-app-config> --schema_file_path <path-to-schema-json>`
- **Download del pacchetto Helm:** Una volta registrata, la xApp viene associata a un pacchetto Helm che può essere scaricato con `dms_cli download_helm_chart`. Questo comando recupera il pacchetto contenente tutti i manifesti Kubernetes necessari per il deployment della xApp:
`CHART_REPO_URL=http://localhost:6873 dms_cli download_helm_chart <chart-name> <version>`
- **Installazione della xApp con Helm:** Il pacchetto Helm viene infine installato nel namespace `ricxapp` utilizzando `helm install`. Questo step avvia il deployment della xApp all'interno del cluster Kubernetes, rendendola operativa nel Near-RT RIC: *`helm install <release-name> -n ricxapp <chart-name-chart-version.tgz>`*

Dopo l'installazione, è possibile verificare lo stato della xApp per assicurarsi che sia stata correttamente registrata e avviata nel sistema. Questo controllo può essere effettuato utilizzando Helm, che mostra la lista delle applicazioni attive nel namespace, oppure direttamente tramite Kubernetes, monitorando lo stato dei pod con il comando: *kubectl get pods -n ricxapp*

L'intero processo di deployment della xApp avviene all'interno di un cluster Kubernetes, il cui ruolo nella gestione delle applicazioni containerizzate è stato discusso in dettaglio nel Capitolo 4. In quel capitolo, sono stati approfonditi i concetti di orchestrazione con Kubernetes, l'uso di Helm per semplificare il deployment e la gestione delle applicazioni, e l'importanza della scalabilità dei microservizi in ambienti cloud-native. Il deployment della xApp si basa su questi principi, sfruttando Kubernetes per la gestione delle risorse e Helm per l'automazione della configurazione e dell'installazione.

Se in futuro è necessario aggiornare la xApp con nuove configurazioni, Helm consente di effettuare un upgrade senza dover disinstallare e reinstallare l'applicazione. Questo approccio facilita la gestione dinamica della xApp, consentendo modifiche senza interruzioni del servizio: *helm upgrade <release-name> -n ricxapp <chart-name-chart-version.tgz>*

Infine, se la xApp deve essere rimossa dal sistema, è possibile utilizzare Helm per disinstallarla, eliminando sia la registrazione nel Near-RT RIC che il relativo deployment Kubernetes: *helm uninstall <release-name> -n ricxapp*

L'onboarding della xApp con *dms_cli* e Helm rappresenta un metodo efficiente e modulare per gestire la sua distribuzione, garantendo un'integrazione semplice e scalabile nell'ecosistema O-RAN. L'adozione di Kubernetes e Helm, come discusso nel Capitolo 4, fornisce un'infrastruttura affidabile per il deployment e la gestione di applicazioni cloud-native, assicurando flessibilità ed efficienza operativa. Inoltre, l'uso di *dms_cli* introduce un livello di controllo aggiuntivo sul ciclo di vita della xApp, standardizzando il processo di registrazione, validazione e distribuzione all'interno del Near-RT RIC.

7 Test e Risultati Sperimentali

In questo capitolo vengono presentati i test sperimentali condotti per analizzare le prestazioni del sistema in termini di latenza e tasso di errore in diverse condizioni operative. L'obiettivo di questa fase di valutazione è comprendere il comportamento della rete e dei dispositivi coinvolti nel processo di trasmissione dati, identificando eventuali colli di bottiglia e limitazioni strutturali che potrebbero influenzare l'efficienza della comunicazione.

L'analisi delle prestazioni è stata eseguita mediante due approcci distinti, ciascuno finalizzato a valutare il sistema in condizioni operative differenti:

- Gradual Load Test, in cui la frequenza di trasmissione è stata aumentata progressivamente con intervalli temporizzati di 30 secondi tra un incremento e l'altro. Questa metodologia ha permesso di osservare come il sistema si adatti a un carico crescente, fornendo indicazioni sulla sua capacità di gestione delle risorse in condizioni di traffico controllato.
- Stress Test, in cui la frequenza di trasmissione è stata aumentata senza intervalli di attesa, sottoponendo il sistema a un carico elevato e costante. Questo test è stato progettato per individuare eventuali limiti strutturali e verificare fino a che punto il sistema è in grado di mantenere prestazioni accettabili prima di entrare in uno stato di saturazione.

I risultati di questi test sono rappresentati attraverso grafici dettagliati, che illustrano l'andamento della latenza media e la ripartizione in funzione della frequenza dei messaggi inviati. L'analisi dei dati consente di individuare tendenze chiave, come l'aumento progressivo della latenza, l'insorgere di fluttuazioni nei tempi di risposta e il punto in cui la perdita di pacchetti diventa significativa.

Questa valutazione sperimentale è fondamentale per ottimizzare le prestazioni del sistema, poiché consente di comprendere le reali capacità operative della rete e dei dispositivi coinvolti. I dati ottenuti permettono di evidenziare i limiti della tecnologia LoRa in scenari di traffico elevato, suggerendo possibili strategie di ottimizzazione per migliorare l'affidabilità e la scalabilità del sistema.

7.1 Fase di Configurazione

Durante la fase di configurazione, sono stati condotti una serie di test per misurare la latenza in tre diverse fasi, al fine di comprendere come il sistema rispondesse all'invio sequenziale di un numero variabile di configurazioni. L'obiettivo principale di questi test era valutare l'impatto della quantità di configurazioni inviate sulla latenza complessiva del sistema, individuando eventuali perdite di pacchetti o inefficienze nei processi di comunicazione tra i diversi componenti della rete.

Per eseguire le misurazioni, sono stati considerati tre passaggi fondamentali nella trasmissione dei dati: il primo riguarda la comunicazione tra il componente E2TermCustom e il LoRaGateway, ovvero il tempo impiegato per il trasferimento delle configurazioni da un nodo di elaborazione avanzato al Gateway responsabile della gestione della rete LoRa. Il secondo aspetto analizzato è il tempo di computazione all'interno dello stesso LoRaGateway, cioè il tempo necessario per processare le configurazioni ricevute prima di poterle inoltrare verso il dispositivo di destinazione. Infine, il terzo scenario riguarda la comunicazione dal LoRaGateway al LoRaEnd-Device, ossia la fase in cui le configurazioni elaborate vengono effettivamente inviate ai dispositivi finali LoRa, che rappresentano l'ultima destinazione nel processo di configurazione della rete.

Per valutare le prestazioni del sistema in condizioni di carico variabile, i test sono stati eseguiti in due differenti configurazioni: la prima ha previsto l'invio di 10 configurazioni, mentre nella seconda sono state inviate 100 configurazioni in parallelo, i test sono stati effettuati 20 volte, al fine di osservare come l'incremento del numero di richieste potesse influenzare i tempi di latenza complessivi. I risultati

ottenuti hanno mostrato un aumento significativo della latenza media al crescere del numero di configurazioni, evidenziando un comportamento esponenziale nella degradazione delle prestazioni.

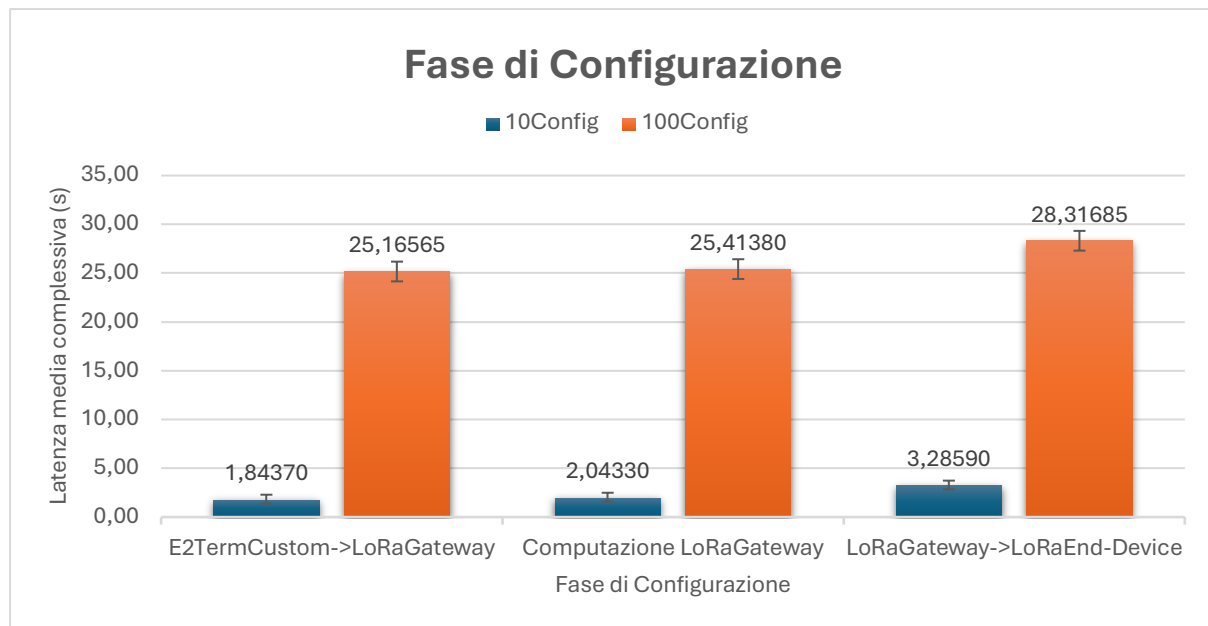


Figura 7.1 Fase di Configurazione: Confronto tra 10 e 100 Configurazioni

Un aspetto fondamentale da considerare nei risultati è che, nella fase di configurazione, il tempo di latenza è stato calcolato come il tempo intercorso tra l'invio del primo pacchetto e la ricezione dell'ultimo pacchetto. Poiché il processo di trasmissione è sequenziale e il protocollo LoRa impiega del tempo a trasferire ogni pacchetto dal LoRaGateway al LoRaEnd-Device, il tempo di arrivo dell'ultimo pacchetto include il tempo di elaborazione di tutte le configurazioni precedenti, causando un effetto cumulativo sulla latenza totale. Questo spiega perché i valori misurati risultano particolarmente elevati, in quanto l'aumento del numero di configurazioni implica un accumulo progressivo dei tempi di elaborazione e trasmissione lungo l'intero flusso dati.

Nella prima fase del test, in cui sono state inviate 10 configurazioni, il tempo medio necessario per il trasferimento delle configurazioni da E2TermCustom a LoRaGateway è risultato pari a 1,84370 secondi, mentre per la computazione all'interno del LoRaGateway la latenza media è stata di 2,04330 secondi. Infine, il tempo necessario per la trasmissione delle configurazioni dal LoRaGateway al LoRaEnd-Device è stato misurato in 3,28590 secondi.

Tuttavia, quando il numero di configurazioni inviate è stato aumentato a 100, i tempi di latenza hanno subito un incremento drastico. Il tempo medio per il trasferimento da E2TermCustom a LoRaGateway è salito a 25,16565 secondi, mostrando un aumento considerevole rispetto alla configurazione precedente. La computazione all'interno del LoRaGateway ha subito un incremento analogo, raggiungendo una latenza media di 25,41380 secondi, suggerendo un sovraccarico significativo nelle operazioni di elaborazione interna del gateway. Anche la fase finale del processo, ovvero l'invio dei dati dal LoRaGateway al LoRaEnd-Device, ha evidenziato un incremento importante della latenza, che ha raggiunto i 28,31685 secondi.

L'analisi dei dati evidenzia che il sistema risente in modo significativo dell'aumento del numero di configurazioni inviate, mostrando un incremento della latenza con un andamento quasi lineare. Questo comportamento indica che, all'aumentare del carico, l'invio simultaneo di più pacchetti può rallentare il sistema, causando un accumulo di richieste in attesa di elaborazione. Di conseguenza, si possono

verificare fenomeni di congestione nei nodi di calcolo e trasmissione, compromettendo l'efficienza complessiva della rete.

Al fine di migliorare la gestione della trasmissione e ridurre il problema della latenza cumulativa, è stata adottata una soluzione con un thread dedicato nel LoRaGateway. Questa soluzione prevede l'implementazione di una coda interna che permette al gateway di:

- Ricevere i pacchetti da E2TermCustom in ingresso senza interrompere il processo di trasmissione.
- Gestire in parallelo l'invio verso LoRaEnd-Device, evitando che il tempo di attesa di un pacchetto ritardi la trasmissione dei successivi.

L'adozione di questa strategia mira a ottimizzare il flusso dati, separando la gestione della ricezione dalla trasmissione e riducendo l'effetto cumulativo della latenza. Questo approccio dovrebbe migliorare l'efficienza del sistema, specialmente nelle condizioni di carico elevato testate nel Gradual Load Test e nello Stress Test, con lo scopo di ridurre la congestione e migliorare la capacità del sistema di scalare con l'aumento del numero di pacchetti inviati.

7.2 Gradual Load Test: Analisi della Latenza e la Ripartizione della Trasmissione

Il Gradual Load Test è stato progettato per studiare l'andamento della latenza media, tempo di trasmissione e la ripartizione dei pacchetti in condizioni controllate, simulando un incremento progressivo della frequenza di invio dei pacchetti dalla xApp. Questa metodologia consente di valutare il comportamento del sistema in condizioni di carico crescente, osservando il punto in cui iniziano a emergere problemi di latenza e perdita di pacchetti.

La trasmissione dei pacchetti è stata gestita con un meccanismo temporizzato, in cui il numero di messaggi è stato aumentato gradualmente, partendo da 1 fino a 100 messaggi. Per evitare il sovraccarico della rete e garantire che tutti i pacchetti avessero il tempo necessario per raggiungere il LoRaEnd-Device, tra un incremento e l'altro è stato introdotto un intervallo di 30 secondi. Questo approccio ha consentito di valutare la risposta del sistema nel tempo, riducendo il rischio di congestione immediata e permettendo un'analisi più dettagliata delle prestazioni.

L'analisi ha preso in considerazione due fasi principali della trasmissione:

- E2TermCustom → LoRaGateway (tramite UDP).
- LoRaGateway → LoRaEnd-Device (tramite modulo LoRa).

L'analisi della latenza nella prima fase (Figura 7.2) della trasmissione mostra un comportamento altamente variabile, con una latenza generalmente contenuta nei primi intervalli ma soggetta a oscillazioni irregolari con l'aumento della frequenza di invio dei pacchetti. Sebbene inizialmente i valori rimangano nell'ordine di pochi millisecondi, all'aumentare della frequenza si osservano fluttuazioni più marcate, caratterizzate da picchi improvvisi che indicano momentanei rallentamenti nella trasmissione.

Il grafico della Figura 7.2, mostra che, nonostante la frequenza di trasmissione aumenti progressivamente, la latenza non cresce in modo lineare, ma è invece caratterizzata da un andamento oscillante, con momenti di congestione alternati a recuperi improvvisi delle prestazioni. Questo comportamento indica che il LoRaGateway può assorbire il traffico crescente in modo dinamico, ma in alcuni casi la gestione delle risorse computazionali può introdurre ritardi temporanei prima di tornare a livelli normali.

La comunicazione tra E2TermCustom e LoRaGateway avviene tramite UDP, anche se il protocollo UDP non garantisce che i pacchetti mandati vengano ricevuti dal destinatario, nei test effettuati non si è osservata nessuna perdita.

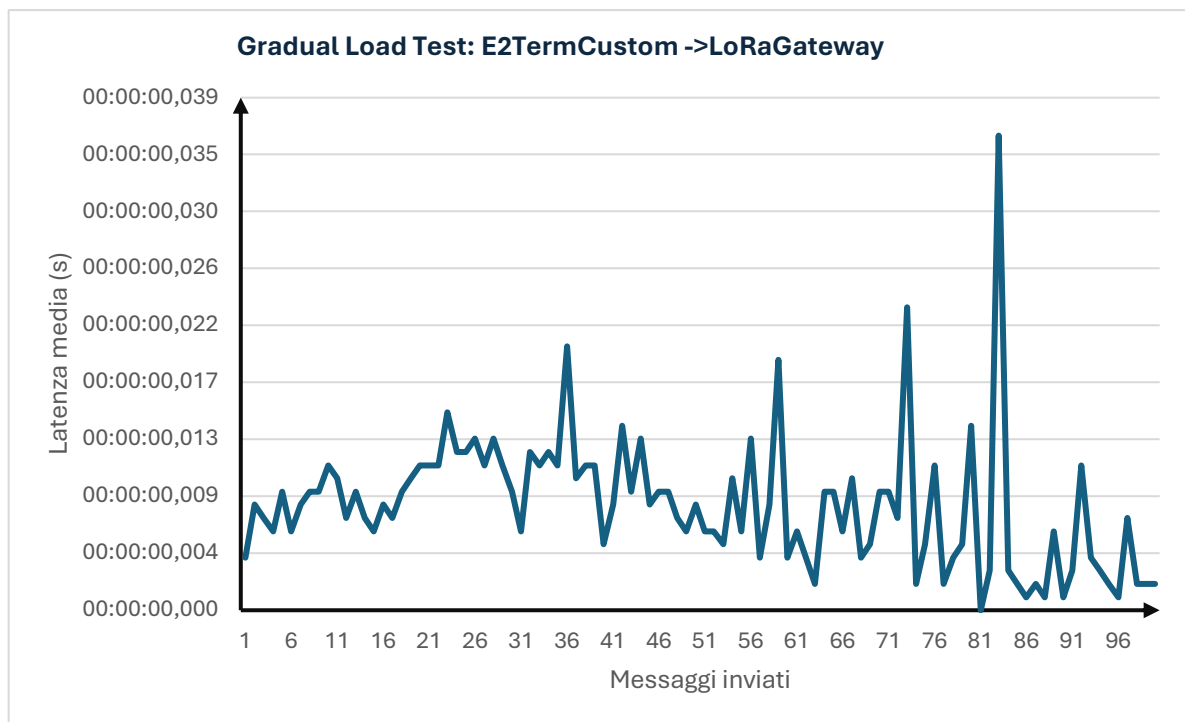


Figura 7.2 Latenza Media nella Comunicazione E2TermCustom → LoRaGateway (Gradual Load Test)

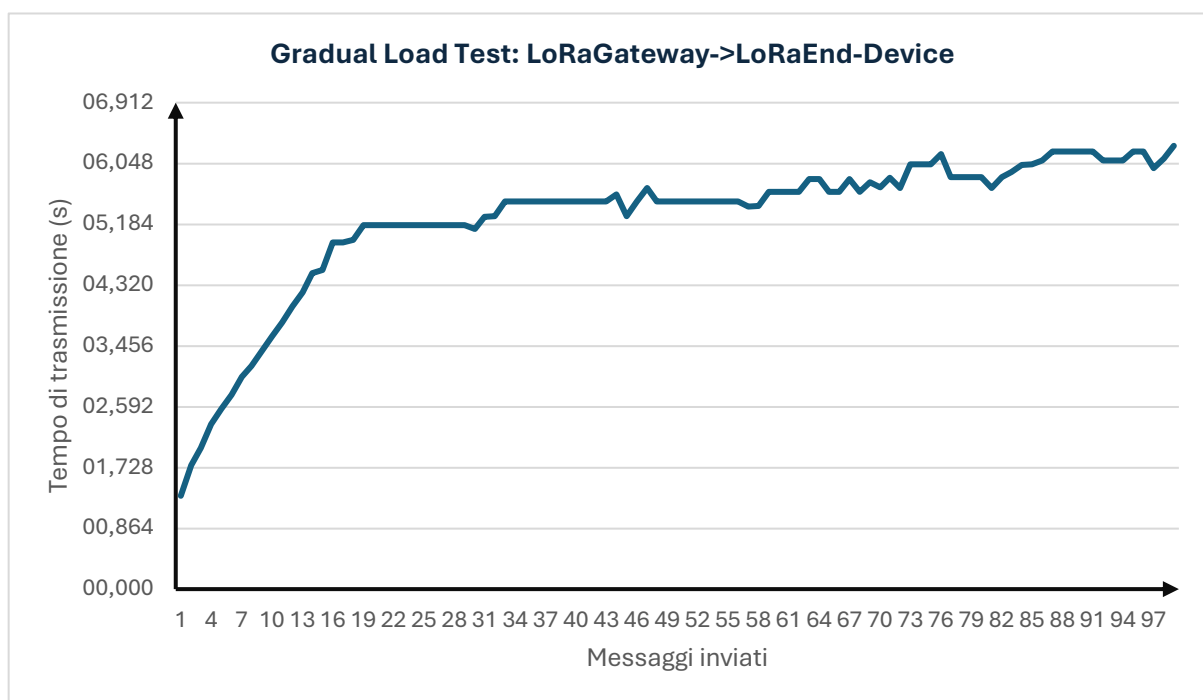


Figura 7.3 Tempo di trasmissione nella Comunicazione LoRaGateway → LoRaEnd-Device (Gradual Load Test)

Utilizzando il thread nel LoRaGateway, nella Figura 7.3 e Figura 7.4 si osserva un aumento regolare fino a una soglia critica, oltre la quale il tempo di trasmissione si stabilizza su in un range di 5-6 secondi. Questo comportamento indica che il LoRaEnd-Device raggiunge il suo limite massimo di capacità di ricezione. Indipendentemente da ulteriori incrementi nella frequenza di invio, il dispositivo non riesce a elaborare un numero maggiore di pacchetti rispetto a quello che la tecnologia LoRa gli consente di gestire.

L'andamento osservato nei grafici suggerisce che la latenza complessiva non è causata da inefficienze nel LoRaGateway, che trasmette correttamente tutti i pacchetti ricevuti tramite UDP, né da una gestione inefficiente della coda di trasmissione, ma piuttosto dai limiti fisici imposti dal modulo LoRa. La natura stessa di questa tecnologia introduce ritardi strutturali, che diventano sempre più evidenti con l'aumento della frequenza di trasmissione. Dopo una certa soglia, il sistema raggiunge un plateau in cui la latenza rimane elevata ma relativamente costante, con solo lievi variazioni. Questo fenomeno evidenzia il collo di bottiglia principale della comunicazione tra LoRaGateway e LoRaEnd-Device, determinato dalla capacità massima del dispositivo finale.

Come mostrato nella Figura 7.4, l'analisi della ripartizione dei pacchetti conferma il comportamento osservato nei tempi di latenza. Nei primi intervalli di trasmissione, il numero di pacchetti persi è nullo, dimostrando che il sistema è in grado di gestire senza difficoltà un traffico ridotto. Tuttavia, con l'aumentare della frequenza di invio, il numero di pacchetti persi inizia a crescere in modo costante e progressivo. Il grafico mostra chiaramente che il numero di pacchetti persi aumenta quasi linearmente, mentre il numero di pacchetti ricevuti rimane relativamente stabile dopo un certo punto. Questo comportamento indica che il LoRaEnd-Device raggiunge un limite strutturale, oltre il quale non è in grado di ricevere ed elaborare un numero maggiore di pacchetti, indipendentemente dalla frequenza di invio.

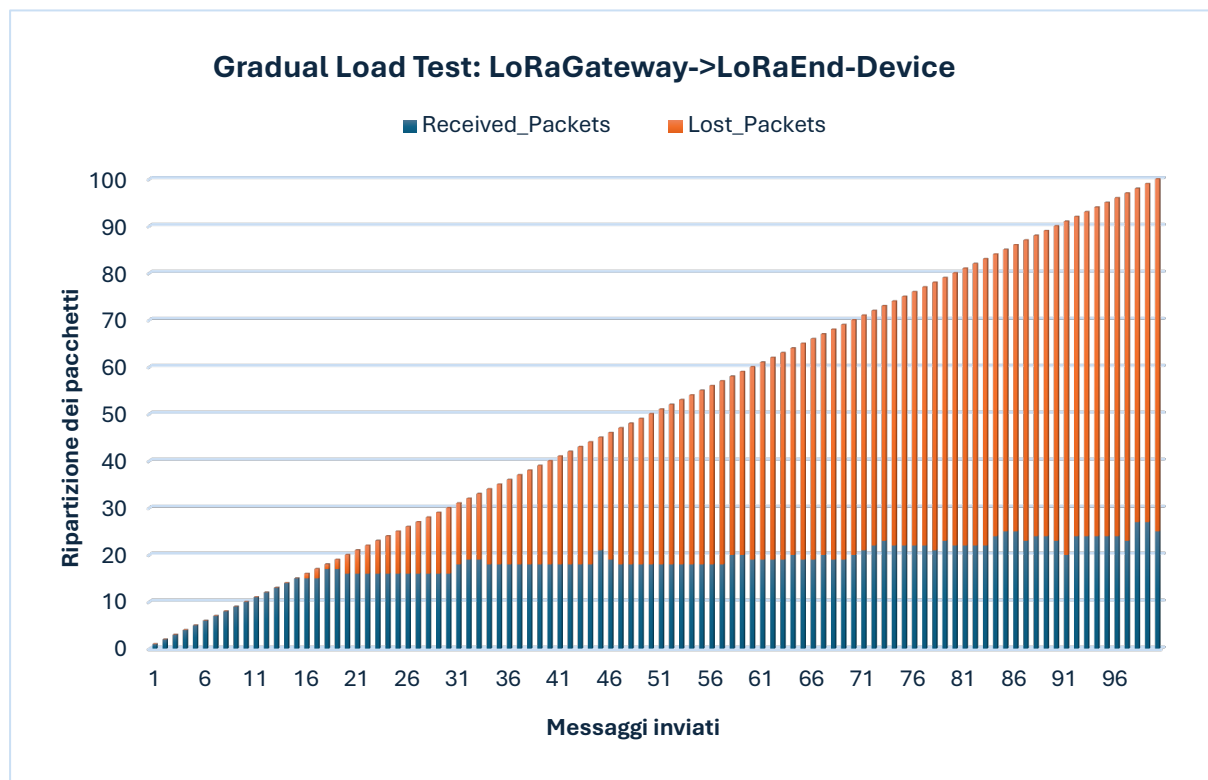


Figura 7.4 Analisi della Ripartizione dei pacchetti (Gradual Load Test)

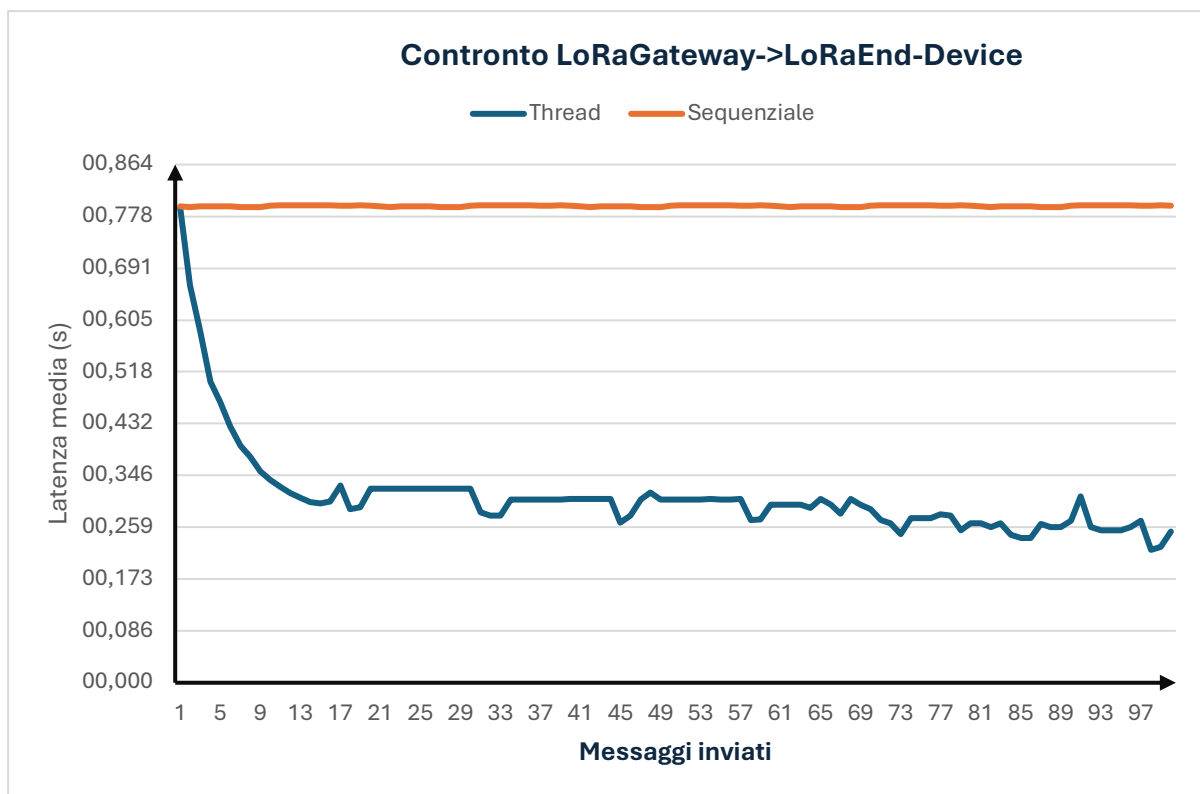


Figura 7.5 Confronto tra invio sequenziale e threading nella latenza LoRaGateway → LoRaEnd-Device

Dalla Figura 7.5 si può osservare il confronto tra l'invio sequenziale e l'utilizzo del thread, analizzando esclusivamente i pacchetti effettivamente ricevuti dal LoRaEnd-Device. L'adozione del thread ha significativamente ridotto il tempo di trasmissione dei pacchetti da parte del LoRaGateway, stabilizzandosi intorno ai 300 ms per pacchetto.

È interessante notare che, quando viene inviato un solo pacchetto, la latenza media è identica in entrambi i casi (sequenziale e thread). Questo conferma che il tempo minimo di trasmissione di un singolo pacchetto è determinato dalle caratteristiche intrinseche della tecnologia LoRa. Tuttavia, con l'aumentare del numero di pacchetti inviati, l'uso del thread permette di ridurre significativamente la latenza rispetto alla trasmissione sequenziale, che invece si mantiene costantemente elevata.

Questo approccio risulta vantaggioso fino all'invio simultaneo di un certo numero di pacchetti. Superata questa soglia si verifica un effetto collaterale: un numero crescente di pacchetti non viene ricevuto correttamente. Ciò indica che l'elevata frequenza di invio può portare alla saturazione del LoRaEnd-Device, compromettendo l'affidabilità della trasmissione.

7.3 Stress Test: Analisi della Latenza e la Ripartizione della Trasmissione

Lo Stress Test è stato progettato per misurare la capacità del sistema di gestire un carico intensivo e continuo di messaggi, simulando una situazione in cui la frequenza di invio viene incrementata rapidamente senza pause prolungate. A differenza del Gradual Load Test, in cui la frequenza veniva aumentata con intervalli di 30 secondi per permettere ai pacchetti in coda di essere elaborati prima di aumentare il traffico. Nello Stress Test il sistema è stato sottoposto a un incremento costante della frequenza di invio ogni secondo, senza alcun ritardo tra un incremento e l'altro. Questo approccio ha permesso di osservare il comportamento del sistema in condizioni di traffico elevato e prolungato, identificando ulteriori colli di bottiglia e limiti strutturali nella gestione della rete.

L'analisi della latenza tra E2TermCustom e LoRaGateway mostra una tendenza alla stabilizzazione, con valori che si attestano tra 35 e 40 millisecondi, come illustrato nella Figura 7.6.

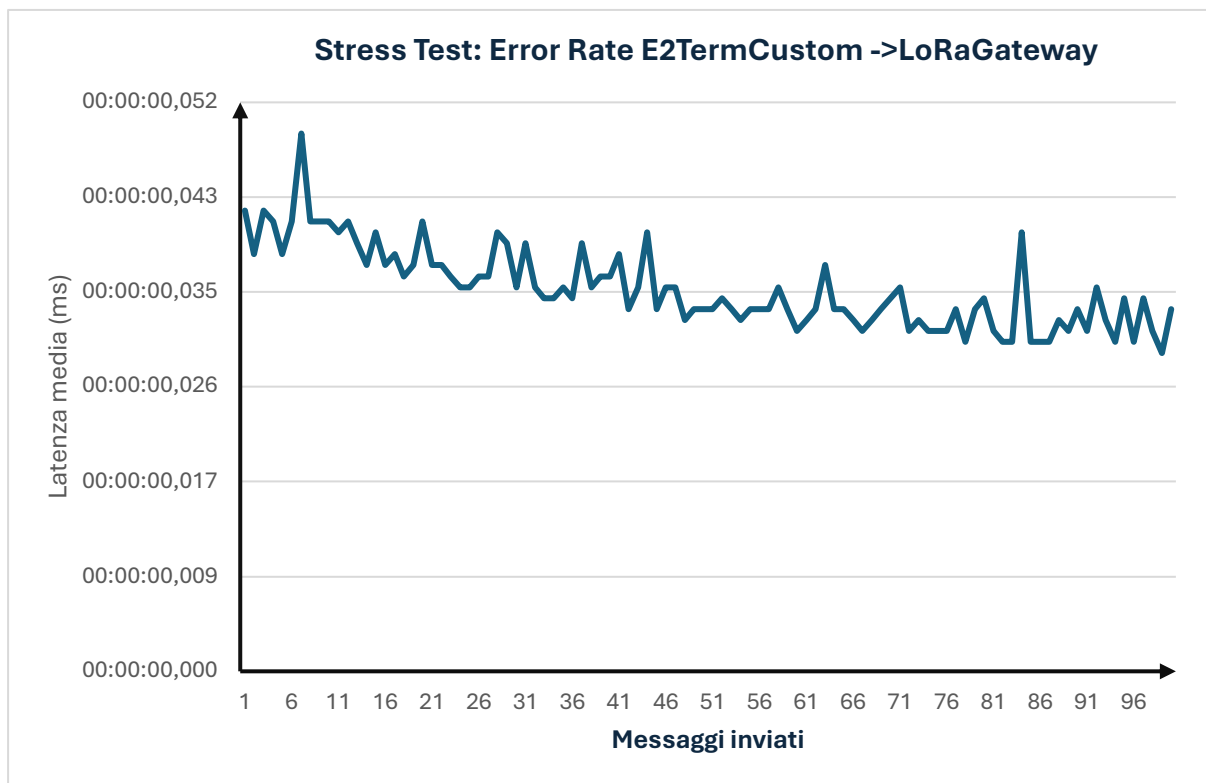


Figura 7.6 Latenza Media nella Comunicazione E2TermCustom → LoRaGateway (Stress Test)

Tuttavia, si osservano variazioni nei tempi di risposta, con picchi occasionali che suggeriscono momentanei rallentamenti dovuti alla saturazione delle risorse computazionali del LoRaGateway. A differenza di quanto accade nel Gradual Load Test, in cui il sistema ha il tempo di adattarsi all'aumento del carico, qui la crescita continua della frequenza senza pause genera fluttuazioni più marcate.

Poiché la comunicazione tra E2TermCustom e LoRaGateway avviene tramite UDP, non si osserva perdita di pacchetti in questa fase. Tuttavia, il tempo di risposta del LoRaGateway potrebbe essere influenzato dalla sua capacità di processare in tempo reale il numero crescente di pacchetti ricevuti.

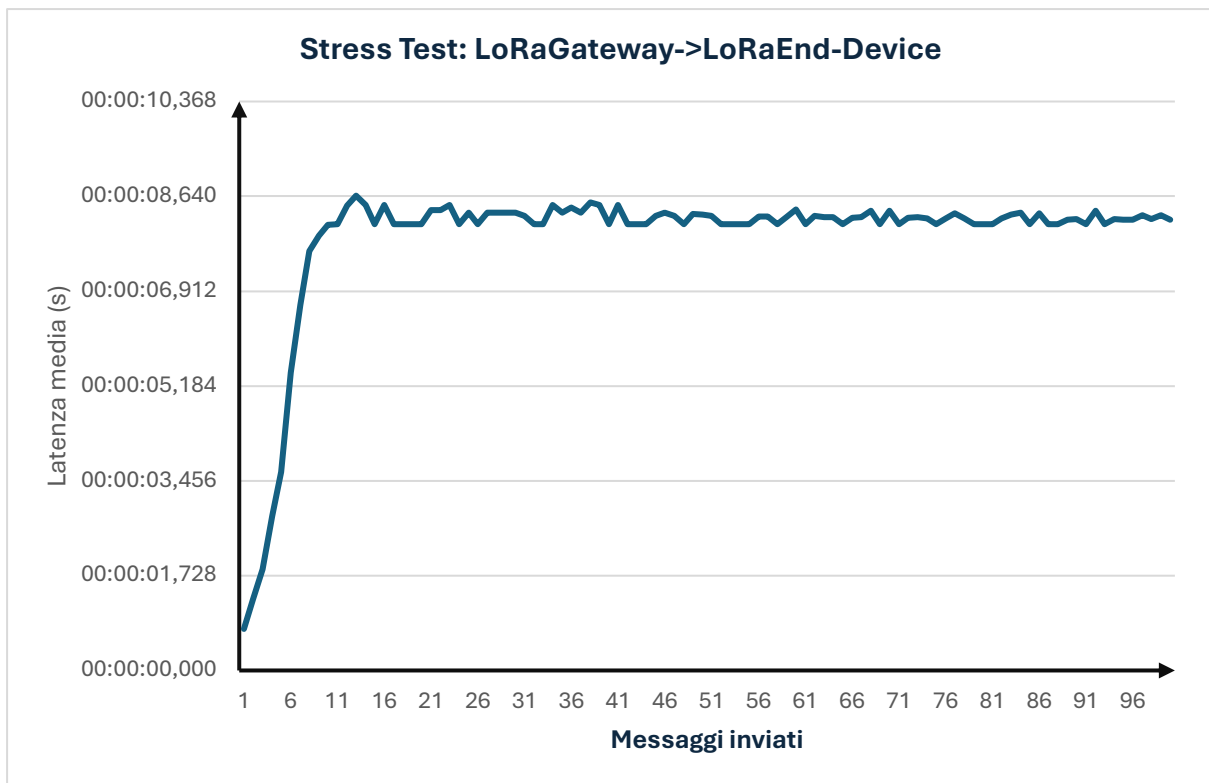


Figura 7.7 Latenza Media nella Comunicazione LoRaGateway → LoRaEnd-Device (Stress Test)

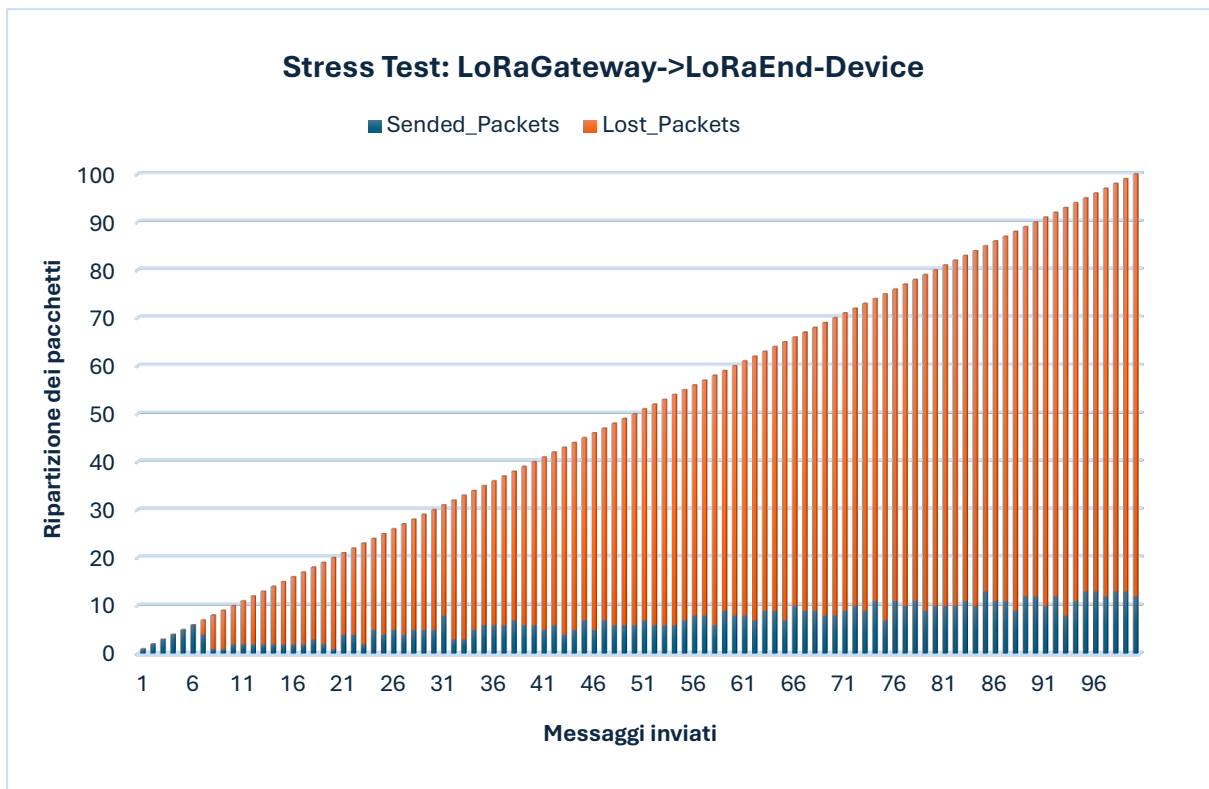


Figura 7.8 Analisi della ripartizione dei pacchetti (Stress Test)

Come nel Gradual Load Test, anche nella comunicazione tra LoRaGateway e LoRaEnd-Device si osserva che il sistema, una volta raggiunta la capacità massima di gestione delle trasmissioni, entra in una fase di saturazione (Figure 7.7 e 7.8). Oltre questo limite, il tempo di elaborazione rimane costante, indipendentemente da ulteriori incrementi della frequenza di invio.

Tuttavia, nello Stress Test, la soglia di ricezione dei pacchetti risulta inferiore rispetto al Gradual Load Test, a causa della velocità di trasmissione del LoRaGateway, che supera di gran lunga la capacità del LoRaEnd-Device di elaborare i messaggi. Inoltre, l'assenza del tempo di attesa di 30 secondi tra un batch e l'altro ha contribuito a un accumulo più rapido dei pacchetti in ingresso, accelerando la saturazione e aumentando il tasso di perdita.

Nei primi intervalli di trasmissione, con una frequenza di invio relativamente bassa, la perdita di pacchetti è trascurabile, poiché il LoRaEnd-Device riesce a gestire il traffico senza difficoltà. Tuttavia, con l'aumentare della frequenza, il numero di pacchetti persi cresce rapidamente, indicando che il dispositivo ha raggiunto il suo limite massimo di capacità.

Questo comportamento è una conseguenza delle limitazioni intrinseche della tecnologia LoRa, progettata per trasmissioni a basso consumo energetico e bassa velocità di trasferimento dati. La modulazione a lungo raggio e la banda stretta comportano tempi di trasmissione elevati, che portano a fenomeni di congestione man mano che i pacchetti si accodano.

Nel Gradual Load Test, la trasmissione avveniva con intervalli regolari, permettendo al sistema di assorbire il carico in modo progressivo, con perdite che aumentavano solo quando la capacità del LoRaEnd-Device veniva superata. Al contrario, nello Stress Test, l'invio continuo e ininterrotto provoca un accumulo di pacchetti che il dispositivo non riesce a processare, causando un aumento esponenziale delle perdite.

Conclusioni e sviluppi futuri

Nel corso di questa tesi è stata affrontata l'integrazione tra il 5G RAN e la rete LoRa, due tecnologie eterogenee che, sebbene nate per esigenze differenti, possono essere complementari in scenari avanzati di telecomunicazione. L'obiettivo principale del lavoro è stato lo sviluppo di un Control Plane unificato, capace di gestire in modo intelligente la configurazione e il monitoraggio della rete LoRa attraverso il Near-RT RIC (Near-Real Time RAN Intelligent Controller) dell'architettura O-RAN.

Nei primi capitoli sono state esposte le tecnologie e le architetture di riferimento. In particolare, è stato introdotto il concetto di O-RAN e il ruolo chiave del Near-RT RIC nella gestione dinamica delle risorse di rete nel 5G. Successivamente, è stata illustrata l'architettura delle reti LPWAN, con un approfondimento sulla tecnologia LoRa e il suo utilizzo nei sistemi IoT.

Nella fase centrale dell'elaborato, è stato sviluppato un modello di integrazione tra 5G O-RAN e LoRa, basato sull'implementazione di una xApp per il Near-RT RIC. Questa xApp permette di configurare e monitorare dinamicamente la rete LoRa, garantendo un migliore controllo delle risorse e una gestione più efficiente della rete. Inoltre, è stato introdotto un modello di servizio personalizzato, denominato E2SM-LoRa, che definisce il formato dei messaggi per l'interazione tra il RIC e i LoRa Gateway.

Per valutare l'efficacia della soluzione proposta, sono stati condotti test sperimentali su diversi scenari di carico di rete. I risultati hanno evidenziato che il sistema è in grado di gestire in modo efficiente il traffico per frequenze di invio moderate. Tuttavia, all'aumentare della densità di trasmissione, la rete LoRa tende a saturarsi rapidamente, causando un incremento della latenza e del tasso di errore. Il Gradual Load Test ha mostrato che un incremento progressivo della frequenza di trasmissione consente al sistema di adattarsi, mentre lo Stress Test ha evidenziato un rapido degrado delle prestazioni in condizioni di traffico elevato. Una delle principali criticità individuate riguarda la gestione delle code di trasmissione nei LoRa Gateway, che attualmente operano con un modello multi-threading.

Le prospettive future di ricerca si concentreranno su:

- L'ottimizzazione delle strategie di allocazione delle risorse, mediante l'uso di modelli predittivi basati su Machine Learning per analizzare il comportamento della rete LoRa e adattare dinamicamente i parametri di configurazione. La xApp potrà sfruttare algoritmi predittivi per anticipare le condizioni di traffico e regolare parametri come SF (Spreading Factor), BW (Bandwidth), potenza di trasmissione e duty cycle, ottimizzando così le prestazioni del sistema.
- L'integrazione con strumenti avanzati di gestione delle reti LoRaWAN, come ChirpStack, che potrebbe offrire un miglior controllo della registrazione e autenticazione dei dispositivi, l'ottimizzazione dei parametri di trasmissione e una gestione più flessibile delle policy di rete tramite il Near-RT RIC.
- Il miglioramento delle strategie di gestione del traffico nei LoRa Gateway per ridurre la latenza e la perdita di pacchetti in scenari di elevata densità di trasmissione.

L'approccio presentato in questa tesi rappresenta un primo passo verso un'infrastruttura di telecomunicazione più flessibile e adattiva, capace di integrare reti eterogenee in un ecosistema unificato. Questo modello può aprire nuove prospettive per l'ottimizzazione delle reti IoT e delle reti cellulari di prossima generazione, contribuendo alla realizzazione di un sistema di gestione delle risorse più efficiente, scalabile e intelligente. In sintesi, questa tesi ha dimostrato che un Control Plane basato su Near-RT RIC e xApp può migliorare significativamente la gestione della rete LoRa, consentendo un'integrazione più efficace tra reti cellulari e LPWAN. Tuttavia, permangono alcune sfide aperte, in particolare legate alla scalabilità e alla gestione del traffico elevato.

Indice delle figure

Figura 1.1 Architettura dell'O-RAN: interazione tra SMO, RIC e componenti della rete di accesso radio [4]	9
Figura 1.2 Architettura del Near-RT RIC: componenti, gestione della messaggistica e interazioni con SMO, Non-RT RIC e nodi E2 [2]	10
Figura 1.3 Interazione tra Near-RT RIC e nodi E2: terminazione E2, gestione delle xApp e controllo delle funzioni RAN [4]	12
Figura 1.4 Architettura del framework SMO e Non-RT RIC: gestione delle rApp, infrastruttura di messaggistica e interfacce di terminazione [1]	15
Figura 1.5 E2 Interface: Struttura dell'E2AP e incapsulamento degli E2SM [4]	17
Figura 1.6 Procedura di sottoscrizione e gestione degli eventi tra Near-RT RIC ed E2 Node [1]	17
Figura 1.7 Funzionalità e protocolli dell'interfaccia O1 per la gestione e supervisione della rete (MnS) [1]	20
Figura 1.8 Panoramica dell'interfaccia A1 [1]	21
Figura 1.9 Flusso di comunicazione tra xApp, Near-RT RIC e E2 Node [2]	23
Figura 2.1 Evoluzione delle generazioni di rete mobile: dal 1G al 5G [7]	27
Figura 2.2 Ecosistema dell'Internet of Things (IoT): interconnessione di dispositivi e servizi https://www.federprivacy.org/informazione/sapresti-rispondere/cosa-significa-internet-of-things	29
Figura 2.3 Architettura a strati del 5G IoT: livelli funzionali e tecnologie abilitanti [6]	30
Figura 3.1 Architettura di rete LoRaWAN: interazione tra dispositivi finali, gateway, network server e application server [8]	38
Figura 3.2 Struttura dello stack di protocollo LoRaWAN [14]	40
Figura 3.3 Architettura ChirpStack per la gestione delle reti LoRaWAN [18]	41
Figura 4.1 Struttura a livelli per l'esecuzione delle xApp su Kubernetes, con pod Docker e Near-RT RIC su VM o server bare metal [2]	45
Figura 4.2 Dockerfile per il deployment di una xApp [2]	45
Figura 4.3 Distribuzione e gestione di una xApp con Helm Chart [2]	49
Figura 4.4 Ciclo di vita delle xApp in Kubernetes [2]	50
Figura 4.5 Onboarding, installazione e disinstallazione delle xApp nel Near-RT RIC con dms_cli [2]	51

Indice dei listati

Figura 5.1 Architettura dell'integrazione tra LoRa e Near-RT RIC	54
Figura 6.1 Raspberry Pi con modulo SX1262 868M LoRa HAT utilizzato per il LoRa Gateway e il LoRa End-Device [25].....	62
Figura 6.2 Inizializzazione del contesto RMR nella xApp	64
Figura 6.3 Generazione del messaggio di configurazione E2SM-LoRa.....	64
Figura 6.4 Creazione del messaggio di controllo RICControlRequest.....	65
Figura 6.5 Invio del Control Request tramite RMR	66
Figura 6.6 Creazione di un Subscription Request per la xApp.....	67
Figura 6.7 Loop di ascolto e gestione dei messaggi RMR nell'E2 Term	68
Figura 6.8 Gestione della Subscription Request nell'E2 Term.....	68
Figura 6.9 Invio della Subscription Response nell'E2 Term.....	69
Figura 6.10 Gestione del Control Request nell'E2 Term	70
Figura 6.11 Invio della Configurazione al LoRa Gateway tramite UDP.....	71
Figura 6.12 Gestione del Server UDP nel LoRa Gateway.....	72
Figura 6.13 Inoltro e Trasmissione della Configurazione ai Dispositivi LoRa	73
Figura 6.14 Ricezione dei messaggi da LoRa Gateway nel LoRa End-Device.....	73
Figura 6.15 Definizione ASN.1 del Service Model E2SM-LoRa	74
Figura 7.1 Fase di Configurazione: Confronto tra 10 e 100 Configurazioni	78
Figura 7.2 Latenza Media nella Comunicazione E2TermCustom → LoRaGateway (Gradual Load Test)	80
Figura 7.3 Tempo di trasmissione nella Comunicazione LoRaGateway → LoRaEnd-Device (Gradual Load Test).....	80
Figura 7.4 Analisi della Ripartizione dei pacchetti (Gradual Load Test)	81
Figura 7.5 Confronto tra invio sequenziale e threading nella latenza LoRaGateway → LoRaEnd-Device.....	82
Figura 7.6 Latenza Media nella Comunicazione E2TermCustom → LoRaGateway (Stress Test)	83
Figura 7.7 Latenza Media nella Comunicazione LoRaGateway → LoRaEnd-Device (Stress Test)	84
Figura 7.8 Analisi della ripartizione dei pacchetti (Stress Test).....	84

Bibliografia

- [1] M. Polese, L. Bonati, S. D'Oro, S. Basagni and T. Melodia, "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges," in *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 1376-1411, Secondquarter 2023, doi: 10.1109/COMST.2023.3239220
- [2] J. F. Santos, A. Huff, D. Campos, K. V. Cardoso, C. B. Both and L. A. DaSilva, "Managing O-RAN Networks: xApp Development From Zero to Hero," in *IEEE Communications Surveys & Tutorials*, doi: 10.1109/COMST.2025.3539687
- [3] U. M. M, G. Sadashivappa and R. Palepu, "Integration of RIC and xApps for Open -Radio Access Network for Performance Optimization," *2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS)*, Bangalore, India, 2023, pp. 1-4, doi: 10.1109/CSITSS60515.2023.10334159.
- [4] S. Marinova and A. Leon-Garcia, "Intelligent O-RAN Beyond 5G: Architecture, Use Cases, Challenges, and Opportunities," in *IEEE Access*, vol. 12, pp. 27088-27114, 2024, doi: 10.1109/ACCESS.2024.3367289.
- [5] H. -T. Thieu, V. -Q. Pham, A. Kak and N. Choi, "Demystifying the Near-real Time RIC: Architecture, Operations, and Benchmarking Insights," *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, Hoboken, NJ, USA, 2023, pp. 1-8, doi: 10.1109/INFOCOMWKSHPS57453.2023.10225852.
- [6] S. F. Ahmed *et al.*, "Toward a Secure 5G-Enabled Internet of Things: A Survey on Requirements, Privacy, Security, Challenges, and Opportunities," in *IEEE Access*, vol. 12, pp. 13125-13145, 2024, doi: 10.1109/ACCESS.2024.3352508.
- [7] G. Gkagkas, D. J. Vergados and A. Michalas, "The advantage of the 5G network for enhancing the Internet of Things," *2021 6th South-East Europe Design Automation, Computer Engineering, Computer Networks and Social Media Conference (SEEDA-CECNSM)*, Preveza, Greece, 2021, pp. 1-7, doi: 10.1109/SEEDA-CECNSM53056.2021.9566239.
- [8] F. Cuomo, M. Campo, A. Caponi, G. Bianchi, G. Rossini and P. Pisani, "EXPLoRa: Extending the performance of LoRa by suitable spreading factor allocations," *2017 IEEE 13th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob)*, Rome, Italy, 2017, pp. 1-8, doi: 10.1109/WiMOB.2017.8115779.
- [9] J. Navarro-Ortiz, S. Sendra, P. Ameigeiras and J. M. Lopez-Soler, "Integration of LoRaWAN and 4G/5G for the Industrial Internet of Things," in *IEEE Communications Magazine*, vol. 56, no. 2, pp. 60-67, Feb. 2018, doi: 10.1109/MCOM.2018.1700625.
- [10] F. S. D. Silva *et al.*, "A Survey on Long-Range Wide-Area Network Technology Optimizations," in *IEEE Access*, vol. 9, pp. 106079-106106, 2021, doi: 10.1109/ACCESS.2021.3079095.
- [11] J. Zhang *et al.*, "Enabling Blind Area Coverage for the Smart Grids: Integrating Energy-efficient LoRa Technologies in the 5G," *2021 6th International Conference on Intelligent Computing and Signal Processing (ICSP)*, Xi'an, China, 2021, pp. 952-956, doi: 10.1109/ICSP51882.2021.9408706.
- [12] Y. Chen, Y. A. Sambo, O. Onireti, S. Ansari and M. A. Imran, "LoRaWAN-5G Integrated Network with Collaborative RAN and Converged Core Network," *2022 IEEE 33rd Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC)*, Kyoto, Japan, 2022, pp. 1-5, doi: 10.1109/PIMRC54779.2022.9977480.

- [13] A. Lavric and A. I. Petrariu, "LoRaWAN communication protocol: The new era of IoT," *2018 International Conference on Development and Application Systems (DAS)*, Suceava, Romania, 2018, pp. 74-77, doi: 10.1109/DAAS.2018.8396074.
- [14] Workgroup, T. M. (2015). LoRaWAN™ What is it? A technical overview of LoRa® and LoRaWAN™
- [15] Arratia-Urbe, B. A., Rosas-Olivos, E. S., Tavares De Araujo Cesariny Calafate, C. M., Cano, J. C., Cecilia-Canales, J. M., & Manzoni, P. (2024). AILoRa: Empowering environmental intelligence through an advanced LoRa-based IoT solution. *Computer Communications*, 218, 44-58.
- [16] T. Lin, C. Baohao, G. Weiming, Z. Guoyi and W. Long, "Deployment and Performance Verification of 5G Smart Grid Based on LoRa," *2021 4th International Conference on Information Communication and Signal Processing (ICICSP)*, Shanghai, China, 2021, pp. 538-542, doi: 10.1109/ICICSP54369.2021.9611908.
- [17] ChirpStack, "API Documentation," <https://www.chirpstack.io/docs/chirpstack/api/>
- [18] ChirpStack, "Architecture Overview" <https://www.chirpstack.io/docs/architecture.html>
- [19] IoT Factory, "Best LoRaWAN Network Servers," <https://iotfactory.eu/products/software-platform/best-lorawan-network-servers/>
- [20] Dusun IoT, "How to Choose the Proper LoRaWAN Gateway," <https://www.dusuniot.com/blog/how-to-choose-the-proper-lorawan-gateway/>
- [21] ChirpStack, "System Requirements for ChirpStack," <https://www.chirpstack.io/docs/chirpstack/requirements.html>
- [22] Docker, "Docker" <https://www.docker.com>
- [23] Kubernetes, "Kubernetes Official Documentation," <https://kubernetes.io>
- [24] HackMD, "E2 Interface and xApp" https://hackmd.io/@RaffieWinata/H1_g-fAlA
- [25] WAVESHARE, "SX1262 868M LoRa HAT," https://www.waveshare.com/wiki/SX1262_868M_LoRa_HAT
- [26] GitHub, "O-RAN SC RIC Deployment," <https://github.com/o-ran-sc/ric-plt-ric-dep/>
- [27] GitHub, "O-RAN SC" <https://github.com/o-ran-sc/ric-plt-xapp-frame-py>
- [28] O-RAN SC, "RIC Subscription Manager Documentation," <https://docs.o-ran-sc.org/projects/o-ran-sc-ric-plt-submgr/>
- [29] GitHub, "O-RAN SC RIC E2 Interface," <https://github.com/o-ran-sc/ric-plt-e2/>
- [30] O-RAN Alliance, "O-RAN: Open Radio Access Network," <https://www.o-ran.org>
- [31] O-RAN Software Community, "O-RAN SC Official Website," <https://o-ran-sc.org>
- [32] Rose, K., Eldridge, S., & Chapin, L. (2015). The internet of things: An overview. *The internet society (ISOC)*, 80(15), 1-53.